open-media-drc

Digital Room Correction media chain for Linux and FreeBSD — User and Administrator Manual

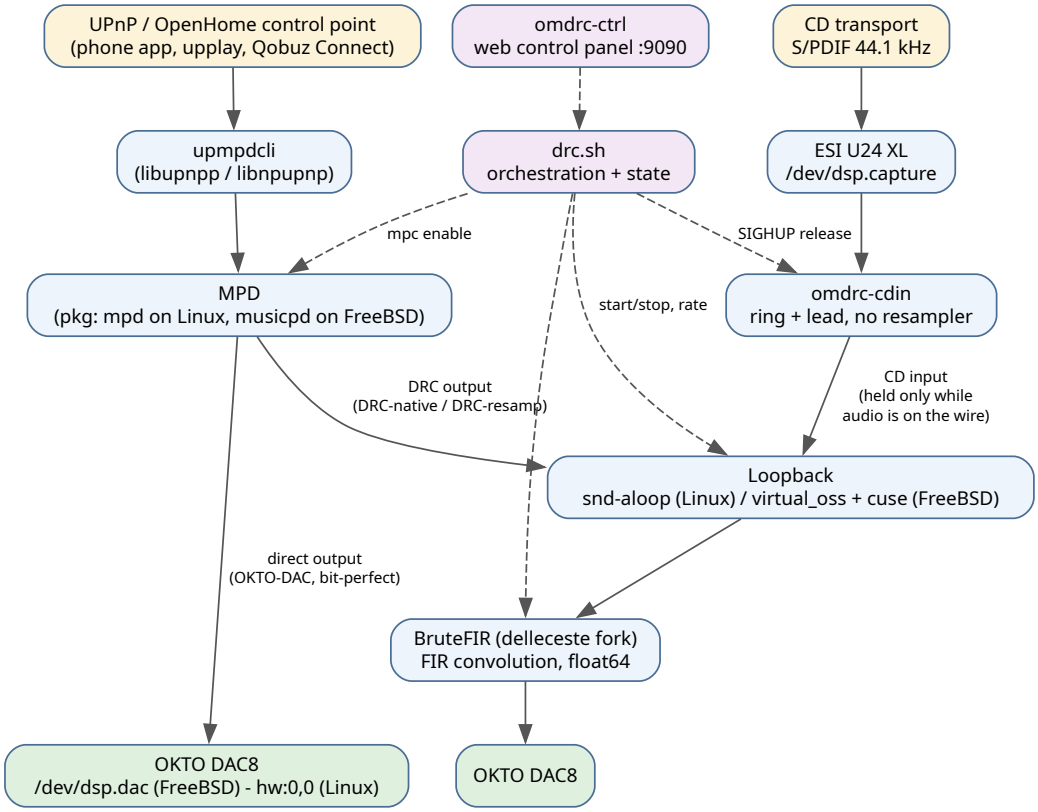
Generated from the repository Markdown documentation

v0.93.0-18-g1106ca6-dirty (2026-08-28)

1 Introduction

open-media-drc is the complete software stack of a headless, high-quality music and video playback appliance with **Digital Room Correction (DRC)**. It runs on both **Linux** (reference: Arch) and **FreeBSD** (reference: 15.1 on an Intel NUC), driving an OKTO RESEARCH DAC8 STEREO USB DAC.

The core idea: audio from any source — UPnP/OpenHome streaming (Qobuz), local files, Blu-ray discs, network streams — is routed through **BruteFIR**, a fast FIR convolution engine, which applies room-correction filters designed with Room EQ Wizard (REW) and DRC tooling, before reaching the DAC. When correction is off, the chain collapses to a verified **bit-perfect** direct path.



The full audio playback chain, from control point to DAC. Solid arrows carry audio; dashed arrows are control.

The design principles that shape everything in the repository:

- **Saved user intent plus actual device state drive DRC.** A present DAC and saved power-on intent mean DRC up at the saved source/rate/design; an absent DAC means transient teardown without losing that intent; saved power-off means direct output. Boot, hotplug, and manual requests reconcile this rule.
- **Only two resting states exist**: DRC fully up (BruteFIR processing), or direct output. A failed start rolls back — there is no resting state where the loopback runs without BruteFIR.
- **Run-from-repo.** The live configuration files and scripts are symlinks into the git checkout wherever possible; `git pull` is the whole update path. Only "deploy glue" parsed at early boot (systemd units, udev rules, rc.d scripts, devd rules) is *copied* into system paths, because those are parsed before a separately mounted `/home` is available.
- **Verify, don't assume.** The repository ships tooling to *prove* the chain is bit-perfect (a USB wire tap), to detect and classify glitches, and to monitor the whole chain from a phone.

1.1 Repository map

Path	Contents
`drc.sh`, `drc-status.sh`	DRC orchestration and status
`configs/<geometry>/`	Per-rate BruteFIR configurations
`filters/<geometry>/<rate>/`	Raw FIR filter coefficients (FLOAT64_LE)
`mpd/`	MPD configuration templates (`mpd.conf.in` Linux, `musicpd.conf.in` FreeBSD)
`etc/`	Service glue: `systemd/`, `modules-load.d/` (Linux); `rc.d/`, `devd/` (FreeBSD)
`omdrc-ctrl/`	Web control panel (Flask)
`cdin/`	`omdrc-cdin`, the CD / S-PDIF capture bridge (FreeBSD only)
`video/`	mpv playback launchers + phone web remote
`browser-nodrc/`	Browser launchers that temporarily bypass DRC
`scripts/`	Filter conversion, headroom, verification helpers
`tests/`	Bit-perfect test signal
`freebsd-uaudio-patch/`	FreeBSD kernel `uaudio(4)` patches (Appendix A)
`freebsd-virtual-oss-patch/`	FreeBSD `virtual_oss` / `cuse` patches (Appendix A)
`kodi-virtual-oss-patch/`	Kodi OSS-sink enumeration patch (Appendix A)
`doc/`	Measurement plots, verification and glitch docs, this manual

2 Components

The playback chain is assembled from the following components, in signal order.

2.1 upmpdcli, libupnpp, libnpupnp (UPnP/OpenHome front-end)

`upmpdcli` turns MPD into a UPnP/OpenHome media renderer, so any control point (a phone app, `upplay`, Qobuz Connect) can drive playback. It is built from source, bottom-up, as three standard meson projects: **libnpupnp** (UPnP base library; needs libcurl, libmicrohttpd, expat), **libupnpp** (C++ wrapper), then **upmpdcli** itself (needs jsoncpp, libmpdclient). The optional Qobuz plugin needs python3 with `requests`. Prebuilt packages exist (FreeBSD port `upmpdcli`, Arch AUR); source builds are used here to track upstream.

2.1.1 A disconnected wired port makes it invisible

Do not give an Ethernet port a static address in `rc.conf` on a box that is sometimes wired and sometimes wireless. That single line is the whole bug.

libupnpp chooses its interface once, at startup, taking the first that is UP+RUNNING+MULTICAST *and has an address*. On FreeBSD `em(4)` keeps `RUNNING` set with no carrier, so a statically configured wired port stays fully qualified with no cable in it:

```
ifconfig_em0="inet 192.168.1.9 netmask 255.255.255.0"   # applied regardless
em0: flags=8843<UP,BROADCAST,RUNNING,...>  status: no carrier
```

upmpdcli then binds the dead port and its SSDP advertisements never leave the host. The failure is unusually quiet: upmpdcli **starts, connects to MPD and keeps driving it**, so `ps` and the panel's renderer switch both report it healthy — only BubbleUPnP, Kazoo and every other control point stop listing it. A box whose wired port *used to be* the live one breaks this way without being touched: move it to Wi-Fi and the stale static address is still there, still first in the selection, and now dead.

The tell is one line in `/tmp/upmpdcli.log`:

```
LibUPnP: Using IPV4 192.168.1.9 port 49152     <- not the address you serve on
```

sometimes accompanied by the harder failure:

```
sendAvertisement failed: UpnpSendAdvertisement :-100: UPNP_E_INVALID_HANDLE
Device would not start
```

which is intermittent — the HTTP side can come up (the description is served) while multicast never works.

The fix is in `rc.conf`, not in `upmpdcli.conf`:

```
ifconfig_em0="DHCP"      # NOT "inet 1.2.3.4 ..."
```

An unplugged DHCP interface never gets a lease, so it has no address, so libupnpp skips it and picks the interface that is actually connected — with `upnpiface` left unset, and nothing to edit when you move between Ethernet and Wi-Fi. Plugging the cable back in still works: FreeBSD's own `/etc/devd/dhclient.conf` starts `dhclient` on `LINK_UP`.

Verified on this box: with the static address removed, upmpdcli's automatic selection lands on the live interface by itself, and `upnp:rootdevice` and `ssdp:all` both return the renderer.

Pinning `upnpiface` also works but is the wrong tool — the pinned value goes stale the moment the box changes network, which is exactly the situation this arises in. Keep it as a last resort for a genuinely ambiguous host.

2.1.2 Leave `defaultrouter` unset when any interface uses DHCP

`rc.d/routing` installs `defaultrouter` **unconditionally at boot**, with no regard for whether that gateway is reachable. On a box that is sometimes wired and sometimes wireless it therefore does the wrong thing exactly when it matters: with no cable, the default route still points at the wired gateway and Wi-Fi's DHCP-supplied route is overridden.

With both interfaces on DHCP the connected one supplies the default route by itself, which is the behaviour wanted. Leave `defaultrouter` commented out.

```
# /etc/rc.conf
ifconfig_em0="DHCP"
wlans_iwm0="wlan0"
ifconfig_wlan0="WPA  DHCP"
#defaultrouter="192.168.1.1"      # leave unset; DHCP provides it
```

Note that `synchronous_dhclient` defaults to `NO`, so an unplugged DHCP interface does not delay boot: `dhclient` is started by devd when a link actually comes up.

2.2 MPD — Music Player Daemon

The actual player. Installed **from the OS package**:

> **Naming: MPD is packaged as `mpd` on Linux, but as `musicpd` on FreeBSD.** The FreeBSD package/port is `audio/musicpd`, the daemon binary is `musicpd`, the service is `service musicpd ...`, and the bundled command-line client is `musicpc` (aliasing `mpc`). Everywhere this manual says "MPD", read `mpd` on Linux and `musicpd` on FreeBSD.

MPD must have the **soxr** resampler and the ALSA (Linux) / OSS (FreeBSD) output plugins enabled — both stock packages do. MPD is configured with three outputs (section): the direct DAC output and two DRC loopback outputs.

2.3 The CD / S-PDIF capture bridge

The second source, and the only live one: a CD transport's S/PDIF output, captured through an ESI U24 XL and written into the same loopback MPD uses. On FreeBSD this is `omdrc-cdin`, a purpose-written daemon: a `memcpy` through a ring whose *lead* absorbs the permanent few-ppm difference between the disc's crystal and the DAC's — no resampler, so the bit-perfect claim survives the CD path too —

and it holds the loopback only while audio is actually on the wire. The source-selection policy is nevertheless exclusive: the CD-input control remembers and disables MPD's audible output before the bridge starts, then restores that exact output after it stops. This is explicit on FreeBSD because `virtual_oss` would otherwise mix both writers. On Linux the same job is `alsaloop(1)` from `alsa-utils`, steering `snd-aloop`'s rate-shift control instead of a hand-rolled ring; its single-substream loopback enforces the same exclusivity at the device level. Full treatment, including the ESI configuration both platforms depend on, in chapter .

2.4 The loopback: snd-aloop (Linux) / virtual_oss (FreeBSD)

BruteFIR needs to receive the player's audio. That is done with a loopback device MPD plays into and BruteFIR reads from:

- **Linux**: the `snd-aloop` ALSA kernel module (loaded via `etc/modules-load.d/`), pinned to the DAC's clock with the `timer_source` module parameter (`etc/modprobe.d/`) rather than its default free-running hrtimer — otherwise the loopback is a second, independent drift pair on top of whatever else is clocked against the DAC (section).
- **FreeBSD**: **`virtual_oss`**, a userland OSS mixing/routing daemon from the base system, which creates character devices through the **`cuse(3)`** kernel facility. `drc.sh` starts it per rate with a play node (`/dev/dsp.play`, where MPD writes) and a synchronized loopback node (`/dev/dsp.loop`, where BruteFIR reads): the `-L` loopback. The `cuse` kernel module must be loaded.

2.5 BruteFIR (delleceste fork)

The convolution engine, built from **`github.com/delleceste/brutefir`** — a fork of Anders Torger's classic BruteFIR adding FreeBSD OSS fixes (`bfio_oss` fragment-size fix, `brutefir_loopback -L` fix, and a passthrough-config default). It processes audio entirely in **float64**, convolving the per-rate `L.raw`/`R.raw` FIR filters and writing to the DAC. Requires FFTW3 in both single and double precision; ALSA on Linux (OSS support is built in on FreeBSD). Built with CMake; modules install to `/usr/local/lib/brutefir`.

2.6 open-media-drc proper: drc.sh and the configuration tree

`drc.sh` is the single control point of the DRC pipeline: it starts and stops BruteFIR and the loopback, selects the MPD output, primes the DAC on rate changes, serializes concurrent runs, records persistent state, and rolls back to direct output on failure. Section covers it in full.

2.7 omdrc-ctrl — web control panel

A lightweight Flask web app serving a mobile-friendly, dark, touch-first control panel on the LAN (default port 9090). It exposes DRC control buttons, full audio-chain health monitoring with a bit-perfect verdict, a live spectrum analyzer, and DRC filter-response charts. Section .

2.8 video/ — mpv playback and the phone web remote

Blu-ray discs, DVDs, files and streams played through mpv with audio routed through the same DRC chain, controlled either from KDE Connect (MPRIS) or from a dedicated phone web remote (the mpv-only Kodi replacement). Section .

2.9 browser-nodrc — DRC bypass for browsers

Web browsers cannot easily route into the DRC loopback, and BruteFIR holds the DAC single-open while DRC runs. `browser-nodrc/` provides one launcher per browser (Firefox, Chromium, Chrome) that snapshots the DRC state, runs `drc.sh off`, launches the browser in the foreground, and restores the exact pre-launch state from an EXIT trap — even if the browser crashes.

3 Installation

3.1 Dependencies

Build tools for all from-source components: a C/C++ compiler, **meson + ninja** (upmpdcli stack), **cmake** (BruteFIR fork, omdrc-ctrl), **pkg-config**, and git.

Component	Runtime deps	FreeBSD pkg	Arch pacman
libnpupnp 6.3.0	libcurl, libmicrohttpd, expat	`curl libmicrohttpd expat2`	`curl libmicrohttpd expat`
libupnpp 1.0.4	libnpupnp + the above	(above)	(above)
upmpdcli 1.9.17	libupnpp, jsoncpp, libmpdclient	`jsoncpp libmpdclient`	`jsoncpp libmpdclient`
upmpdcli Qobuz plugin	python3 + requests	`python3 py311-requests`	`python python-requests`
MPD	soxr resampler + ALSA/OSS output	**`musicpd`**	**`mpd`**
BruteFIR (fork)	FFTW3 single+double; ALSA on Linux	`fftw3 fftw3-float`	`fftw alsa-lib`
Loopback	—	`virtual_oss` (+ cuse)	`snd-aloop` kernel module
omdrc-ctrl	python3, flask>=2.3, markdown>=3.5, numpy>=1.21 (optional)	`py311-flask py311-Markdown py311-numpy`	`python-flask python-markdown python-numpy`

3.2 Build and install order

1. The upmpdcli stack (bottom-up; each a standard meson project):

```
for p in libnpupnp-6.3.0 libupnpp-1.0.4 upmpdcli-1.9.17; do
  cd ~/Downloads/$p
  meson setup build --prefix=/usr/local
  ninja -C build
  sudo ninja -C build install
done
sudo ldconfig 2>/dev/null || true   # Linux: refresh the linker cache
```

2. MPD — from the OS package:

```
sudo pkg install musicpd      # FreeBSD
sudo pacman -S mpd            # Arch Linux
```

3. BruteFIR — from the fork:

```
git clone https://github.com/delleceste/brutefir ~/Downloads/brutefir
cd ~/Downloads/brutefir
cmake -B build && cmake --build build
sudo cmake --install build    # modules -> /usr/local/lib/brutefir
```

4. open-media-drc (DRC engine + web UIs + DAC hotplug) — the classical CMake build:

```
git clone --recursive https://github.com/delleceste/open-media-drc ~/DRC/open-media-drc
cd ~/DRC/open-media-drc
cp host.cmake.sample host.cmake   # AUDIO_USER (defaults to the invoking user),
$EDITOR host.cmake                #   GEOMETRY, GEOMETRIES, OMDRC_SITE_DATA_DIRS,
                                  #   MUSIC_DIR, VIDEO_DIR, OMDB_API_KEY
mkdir build && cd build
cmake .. -C ../host.cmake
make
sudo make install                 # -> $PREFIX (default /usr/local)
```

`host.cmake` is read only by `-C`, and only for cache entries that do not exist yet. A plain `cmake ..` configures the whole project from the built-in defaults, and adding `-C` afterwards cannot repair that build directory — CMake skips an initial-cache assignment whose entry is already set. The symptom is silent: everything configures, with the wrong values. The project therefore detects it, because `host.cmake` sets a marker the check looks for:

```
CMake Warning at CMakeLists.txt:50 (message):
  host.cmake exists but this build directory was not initialised from it, so
  the built-in defaults are in effect.  Adding -C now will not help; start a
  fresh build directory:

      rm -rf build && mkdir build && cd build && cmake -C ../host.cmake ..
```

A `host.cmake` copied from a version of the sample that predates the marker will trip this warning; add the one line the sample carries near the top.

`host.cmake` (the successor to the old `config.env`) is the single source of box-specific values; CMake renders every config from it and installs the DRC engine (`drc.sh` behind the `omdrc` / `omdrc-status` wrappers), the site data (brutefir configs + impulse-response filters for `GEOMETRY` and every set listed in `GEOMETRIES`, each looked up along `OMDRC_SITE_DATA_DIRS` and reported at configure time), both web UIs (omdrcctrl :9090 as a **system** service running as the audio user; omdrcvideo :9080 as a **`--user`** service, since it drives the desktop-session mpv), and the DAC-hotplug glue. The install prints the OS-specific enable steps and the one or two files that must be copied into `/etc` (next section).

The older `./install.sh` rendered the `*.in` templates in place and ran everything straight from the checkout (`git pull` = update). That run-from-repo mode still works but is superseded by the CMake install, which now covers the whole DRC stack — engine, DAC-hotplug glue, both web UIs, and the MPD + upmpdcli renderer configs/units. `install.sh` remains only for the desktop glue not yet in CMake: the `browser-nodrc .desktop` launcher entries and the Linux `snd-aloop` module-load. (The video mpv-idle autostart entry moved to CMake: `make install` puts it under `$PREFIX/share/omdrcvideo/autostart/` on both OSes, `make user-install` links it into `~/.config/autostart/`.)

5. BruteFIR defaults — BruteFIR reads float precision, partition size and the I/O devices from `~/.config/BruteFIR/brutefir_defaults.conf`. The per-rate configs deliberately leave their `input`/`output` blocks empty and inherit devices from this file, so it **must** be deployed:

```
mkdir -p ~/.config/BruteFIR
cp brutefir_defaults.linux.conf ~/.config/BruteFIR/brutefir_defaults.conf  # Linux/ALSA
cp brutefir_defaults.conf       ~/.config/BruteFIR/brutefir_defaults.conf  # FreeBSD/OSS
```

If this file is missing, BruteFIR (>= 1.1) silently auto-generates a broken fallback `~/.brutefir_defaults`, every start fails with *"Parse error: path not set ... module 'file'"*, and `drc.sh` rolls back to direct output — so DRC never comes up at boot.

6. Renderer runtime state — MPD, upmpdcli and qobuzconnect2mpd all write persistent state (database, pid, cache, tokens) and drop logs into `/tmp`. On a real host install `make install` runs `scripts/prepare-renderer-runtime.sh` (wired in by `cmake/install-renderer-runtime.cmake.in`, and again from `cmake/renderers.cmake`) to converge every one of those paths onto `AUDIO_USER` — the same user BruteFIR and MPD already share for the audio devices — instead of leaving a reinstall, a user rename, or qobuzconnect2mpd's own dedicated service account to produce mismatched ownership that is discovered only when a renderer fails to start. The script takes `AUDIO_USER`, `AUDIO_HOME`, the qobuzconnect2mpd state directory and the runtime/log directory, refuses a handful of dangerously broad roots, and skips (rather than chowns) an unexpected symlink under `/tmp`. It is skipped entirely under `DESTDIR` packaging, since that is staging rather than the target host.

3.3 Files that must live in /etc

The CMake install copies everything into `$PREFIX` (default `/usr/local`); the update path is `git pull` followed by `cmake --build build && sudo cmake --install build`, not an in-place edit of the checkout. Two files are read *before* `$PREFIX` is in the relevant search path, so they are handled specially:

- **Linux udev rule** — udev only scans `/etc/udev/rules.d` and `/usr/lib/udev`, never `/usr/local/lib/udev`. The install places `99-usb-audio-drc.rules` under `$PREFIX/lib/udev/rules.d` and prints the one-line copy into `/etc/udev/rules.d` (then `udevadm control --reload`).
- **MPD `/etc` drop-in** — overriding the distro `mpd` unit's `User=mpd` requires a real file in `/etc/systemd/system/mpd.service.d/` (a drop-in in `/etc` beats one in `/usr/lib`; see the caveat below).

Everything else stays under `$PREFIX` and needs no `/etc` copy: the systemd units (`$PREFIX/lib/systemd/{system,user}`, which systemd *does* scan), the FreeBSD rc.d scripts and devd rule (`$PREFIX/etc/{rc.d,devd}`, scanned natively), `drc.sh`, the filters and configs. The BruteFIR defaults still go to `~/.config/BruteFIR/` (below).

3.4 Linux specifics

3.4.1 The MPD User= drop-in caveat (Arch)

The Arch mpd package ships a systemd drop-in (`/usr/lib/systemd/system/mpd.service.d/00-arch.conf`) setting `User=mpd`. Drop-ins always apply *on top of* the main unit, so a full unit override at `/etc/systemd/system/mpd.service` **cannot** override that `User=` — it silently loses. The repo therefore ships a **counter-drop-in** (`etc/systemd/system/mpd.service.d/open-media-drc.conf`, rendered by `install.sh`) that sets `User=@AUDIO_USER@` and the repo config path; a drop-in in `/etc` beats one in `/usr/lib`. It must be a **real file copied into `/etc`, not a symlink** (early-boot parse, see above). Do not create a full `/etc/systemd/system/mpd.service`.

3.4.2 USB DAC hotplug (udev + systemd)

File	Installed to	Purpose
`99-usb-audio-drc.rules`	`/etc/udev/rules.d/`	Triggers the service on DAC plug/unplug
`etc/systemd/system/drc-usb-audio.service`	`/etc/systemd/system/`	Starts/stops DRC
`mpd.service.d/open-media-drc.conf`	`/etc/systemd/system/mpd.service.d/`	MPD user/config drop-in

The udev rule matches any USB sound-card control device and pulls in `drc-usb-audio.service` (`Type=oneshot`, `RemainAfterExit=yes` so the several `controlC*` events of one plug never start duplicate BruteFIR instances). `ExecStart` is `drc.sh restore` after a 1 s settle; `ExecStop` is `drc.sh stop`. Because udev synthesizes ADD events for already-present devices at boot, the same service covers boot and hotplug.

Manual control: `sudo systemctl start|stop drc-usb-audio.service`, `journalctl -fu drc-usb-audio.service`.

3.5 FreeBSD installation and lifecycle

This section is the authoritative FreeBSD installation and init-system reference. It covers every FreeBSD init or devd artifact owned by this repository. Port templates and their rendered copies are one logical script, so they appear in one row rather than as duplicates.

3.5.1 Installation procedure

1. Install runtime packages and load cuse.

```
pkg install bash brutefir virtual_oss musicpd mpc
sysrc kld_list+="cuse"
kldload cuse
```

Install the optional controller/video Python dependencies and renderer packages for the features actually used. The login audio user must be a member of the groups that grant access to sound and USB devices. BruteFIR must never be started as root: an interactive `drc.sh` could not stop a root-owned instance.

2. Render or install the project.

For a run-from-repository installation, set `AUDIO_USER`, `AUDIO_HOME`, `PREFIX`, and the media paths in `config.env`, then run:

```
./install.sh
```

For the package layout, use the FreeBSD port under `freebsd/audio/open-media-drc`. The installed `omdrc_audio` script points at `/usr/local/libexec/omdrc/drc.sh`; the run-from-repository script points at this checkout.

The early-boot files must be regular copies in the system paths, not symlinks into a possibly separate `/home` filesystem:

```
install -m 755 etc/rc.d/omdrc_audio /usr/local/etc/rc.d/omdrc_audio
install -m 644 etc/devd/omdrc-audio.conf /usr/local/etc/devd/omdrc-audio.conf
sh scripts/prepare-musicpd-rc-conf-dir.sh \
  /usr/local/etc/rc.conf.d/musicpd
install -m 644 etc/rc.conf.d/musicpd/omdrc_audio \
  /usr/local/etc/rc.conf.d/musicpd/omdrc_audio
```

FreeBSD's `rc.subr` permits `rc.conf.d/musicpd` to be either a single file or a directory. The preparation helper makes the directory form. If the path was already a file, it is moved unchanged, with its mode preserved, to `musicpd/00-local.conf`; then the independent `omdrc_audio` fragment is installed beside it. The helper is idempotent. The direct Make installer and CMake installer call it automatically, while the FreeBSD package performs the same conversion in its `PRE-INSTALL` script before package extraction. This avoids both an upgrade failure from `mkdir` on a file and the loss or rewriting of an administrator's existing MPD settings.

Copy the other enabled rc.d scripts from the table below in the same way. After an update, refresh these copies. Runtime configurations may remain symlinked to the checkout once their containing filesystem is available.

Remove obsolete lifecycle files after the new service is installed and tested:

```
# Historical names; they must not coexist with omdrc_audio.
# /usr/local/etc/rc.d/{drc_usb_audio,brutefir_drc,omdrc_sndlink}
# /usr/local/etc/devd/omdrc-sndlink.conf
# /usr/local/libexec/omdrc-hotplug
```

3. Install the user-owned BruteFIR defaults and MPD configuration.

```
install -d -o AUDIO_USER -g AUDIO_GROUP /home/AUDIO_USER/.config/BruteFIR
install -m 644 brutefir_defaults.conf \
  /home/AUDIO_USER/.config/BruteFIR/brutefir_defaults.conf
```

Merge the three named outputs from `mpd/musicpd.conf.in`: `OKTO-DAC`, `DRC-native`, and `DRC-resamp`. All FreeBSD physical output paths must use `/dev/dsp.dac`; do not substitute a numbered `/dev/dsp0`.

4. Configure the lifecycle in rc.conf.

A core installation with the controller and renderer restore service uses:

```
musicpd_enable="YES"
musicpd_config="/home/giacomo/open-media-drc/mpd/musicpd.conf"

omdrc_audio_enable="YES"
omdrc_audio_user="giacomo"
omdrc_audio_dac="0x152a:0x88c5"       # strongly recommended with >1 card
omdrc_audio_capture="ESI U24XL"       # omit when CD input is unused
omdrc_audio_capture_recsrc="auto"

omdrc_renderer_enable="YES"
upmpdcli_enable="NO"
qobuzconnect2mpd_enable="NO"
qobuzconnect2mpd_user="giacomo"
qobuzconnect2mpd_group="giacomo"
qobuzconnect2mpd_homedir="/var/db/qobuzconnect2mpd"

omdrcctrl_enable="YES"
omdrcctrl_user="giacomo"
omdrcvideo_enable="YES"
omdrcvideo_user="giacomo"
```

Remove any numbered default-device assignment such as `hw.snd.default_unit=0` from `/etc/sysctl.conf`. The global OID survives a re-attach, but its **value is a pcm unit number**, so only the role resolver can know the correct value. Other genuinely global sound tunables remain in `/etc/sysctl.conf`.

For CD input, also set `omdrc_cdin_enable="YES"` and `omdrc_cdin_user="giacomo"`. The detailed CD knobs are documented in section .

These are all project lifecycle key families:

Key family	Purpose
`musicpd_enable`, `musicpd_config`	MPD boot and configuration
`omdrc_audio_enable`, `omdrc_audio_user`, `omdrc_audio_drcsh`, `omdrc_audio_statussh`	master audio lifecycle and user boundary
`omdrc_audio_dac`, `omdrc_audio_capture`	stable card identities
`omdrc_audio_dac_sysctls`, `omdrc_audio_capture_sysctls`, `omdrc_audio_capture_recsrc`	per-role settings reapplied after every attach
`omdrc_audio_rundir`, `omdrc_audio_lockfile`, `omdrc_audio_statefile`	root boot-lifetime device transaction state
`omdrc_cdin_*`	optional CD bridge, fully listed in section

Key family	Purpose
`omdrc_renderer_enable`, `omdrc_renderer_prefix`, `omdrc_renderer_script`, `omdrc_renderer_statedir`	restore the last selected renderer
`upmpdcli_enable`, `upmpdcli_user`, `upmpdcli_homedir`, `upmpdcli_config`, `upmpdcli_pidfile`, `upmpdcli_logfile`, `upmpdcli_flags`	UPnP renderer worker
`omdrcctrl_enable`, `omdrcctrl_user`, `omdrcctrl_env`, `omdrcctrl_pidfile`, `omdrcctrl_logfile`	web controller
`omdrcvideo_enable`, `omdrcvideo_user`, `omdrcvideo_env`, `omdrcvideo_pidfile`, `omdrcvideo_logfile`	video web remote

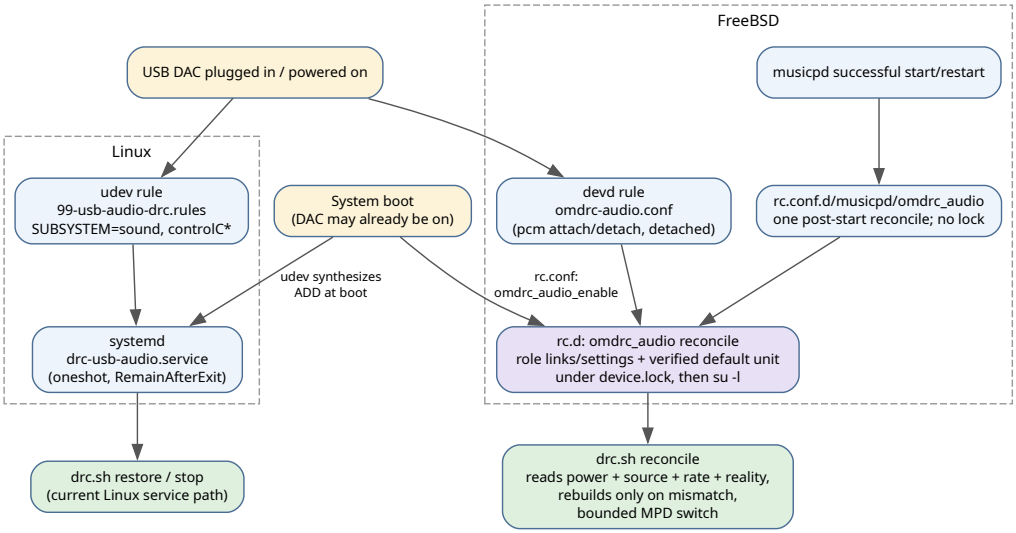
The old `drc_usb_audio_*`, `brutefir_drc_*`, and `omdrc_sndlink_*` families are accepted only as one-release migration fallbacks by `omdrc_audio`. Copy their local values to the new keys and remove them. Enabling an old copied script creates a second lifecycle owner and is unsupported.

5. Validate ordering and activate.

```
rcorder /etc/rc.d/* /usr/local/etc/rc.d/* | \
  egrep 'devd$|omdrc_audio$|musicpd$|omdrc_cdin$|omdrc_renderer$'
service devd restart
service omdrc_audio roles
service omdrc_audio status
service omdrc_audio reconcile
sysctl hw.snd.default_unit       # must equal the pcm unit reported as dac
mpc outputs                     # desired output enabled after musicpd starts
```

`omdrc_audio` requires `devd`, so the cold-plug scan runs after devd is listening. A card that finishes attaching after the scan produces a pcm event. The service deliberately does not require MPD: a slow MPD must not prevent the physical DRC chain from becoming healthy.

3.5.2 Complete FreeBSD init and devd inventory



Linux's current udev/systemd edge and the redesigned FreeBSD devd/rc.d level reconciliation.

Script/configuration	rcorder relation or event	Function	Enable directly?
`musicpd`	`REQUIRE: mixer LOGIN avahi_daemon`	Starts MPD with the repository's FreeBSD configuration	Yes
`rc.conf.d/musicpd/omdrc_audio`	successful `musicpd start_postcmd`	Issues one bounded audio reconcile after MPD is actually available	No; sourced by `musicpd`
`omdrc_audio`	`REQUIRE: FILESYSTEMS devd; shutdown`	Single owner of card roles and DRC lifecycle	Yes
`omdrc_cdin`	`REQUIRE: omdrc_audio; shutdown`	Optional continuous S/PDIF capture bridge	Yes, only with CD input
`omdrc_renderer`	`REQUIRE: NETWORKING FILESYSTEMS musicpd; shutdown`	Restores whichever renderer the UI last selected	Yes
`upmpdcli`	`REQUIRE: NETWORKING FILESYSTEMS musicpd; shutdown`	UPnP/OpenHome worker controlled by `omdrc_renderer`	No when renderer restore is used
`omdrcctrl`	`REQUIRE: NETWORKING LOGIN; shutdown`	Starts the web controller as the audio user	Yes when installed
`omdrcvideo`	`REQUIRE: NETWORKING LOGIN; shutdown`	Starts the video web remote; it does not start mpv	Yes when installed
`omdrc-audio.conf`	devd `pcm[0-9]+` attach and detach	Detaches one level-triggered `omdrc_audio reconcile` request	Installed in devd; no rcvar

3.5.2.1 musicpd

The project `musicpd` script selects the rendered `musicpd_config`, asks rc.subr to derive the pidfile from that file, and launches the FreeBSD `musicpd` binary. MPD drops to the user/group declared inside its configuration. It owns no DRC transition and no project lock. The dependency token is consistently `musicpd`: both `omdrc_renderer` and `upmpdcli` require the name this script actually provides.

3.5.2.2 rc.conf.d/musicpd/omdrc_audio

This is a service-specific `rc.subr` configuration fragment, not another daemon and not another init service. It sets `musicpd`'s `start_postcmd` to a small function that runs only after MPD has started successfully:

```
omdrc_musicpd_poststart()
{
    checkyesno omdrc_audio_enable 2>/dev/null || return 0
    /usr/sbin/service omdrc_audio reconcile ||
        warn "musicpd: omdrc_audio reconcile failed; retry it manually"
    return 0
}
```

All supported installation paths install this fragment: the FreeBSD CMake branch, the direct Make target, and the port/package. This is required because `omdrc_audio` normally runs before `musicpd`: the physical chain can be healthy while its bounded MPD selection remains pending. The successful MPD start is the factual readiness event that triggers one later reconcile. The hook itself has no lock, wait, daemon, or reverse call to `musicpd`.

An existing single-file `rc.conf.d/musicpd` cannot coexist at the same path with the fragment directory. Before installing the hook, it is therefore migrated to `rc.conf.d/musicpd/00-local.conf`. `rc.subr` sources regular files from the directory in lexical order, so local settings remain active and the hook is added without editing them.

The trigger exists because `omdrc_audio` deliberately does **not** require or wait for MPD. At early boot it can construct and verify virtual_oss and BruteFIR first; the final bounded `mpc` output switch then reports `pending` if MPD is still unavailable. A successful MPD start is the earliest factual readiness signal, so the hook retries once at that event rather than guessing with a delay, polling forever, or adding a readiness gate to the physical chain. The same hook runs after a manual `service musicpd restart`, which is another point at which MPD may have forgotten or disabled its outputs.

The edge is strictly one-way:

```
musicpd successful start
  -> service omdrc_audio reconcile
  -> roles transaction (device.lock, then release)
  -> drc.sh reconcile (drc.lock, bounded mpc)
```

No `omdrc_audio` path starts or restarts `musicpd`, so there is no recursive service cycle. The fragment takes no lock and creates no background process. The ordinary reconcile path supplies all serialization and deadlines. A reconcile failure is reported but the hook returns success: MPD is already running

and must not be falsely reported as failed because an optional audio route needs later operator attention. When `omdrc_audio_enable` is not enabled, the hook is a no-op, preserving the master switch.

3.5.2.3 omdrc_audio

`omdrc_audio` replaces the former three-service/helper chain. Its rcvar, `omdrc_audio_enable`, is the only master switch. Its verbs are:

Verb	Meaning
`start`	boot cold-plug role pass, then full reconcile
`roles`	root-only role links/settings transaction, no DRC transition
`reconcile`	role transaction, release device lock, then user DRC reconcile
`stop`	transient teardown; preserve desired power, rate, design, and source
`status`	show role resolution and actual chain status

The service absorbs, rather than discards, two essential old responsibilities. From `brutefir_drc` it preserves `su -l`, so HOME and the login-class PATH are correct and BruteFIR remains owned by the same user who runs interactive commands. From `drc_usb_audio` it preserves the master rcvar and boot/hotplug entry point. It does not preserve the unreliable `/var/run/drc_usb_audio.active` marker; actual processes, configuration paths, rates, nodes, role links, and saved intent are authoritative.

Role publication is an atomic temporary-file rename to `/var/run/omdrc/audio.roles`. Link changes use relative targets:

```
/dev/dsp.dac       -> dspN
/dev/mixer.dac     -> mixerN
/dev/dsp.capture   -> dspM
/dev/mixer.capture -> mixerM
```

A pre-existing non-symlink at a role name is never overwritten. A detach pass removes only project-owned symlinks whose role is now unfilled. An explicit USB ID, optional serial, or description match wins; automatic DAC ranking is reported loudly when several playback candidates make the result a guess. There is no numbered-device fallback.

The same serialized role transaction owns the kernel's bare-OSS default. It sets `hw.snd.default_unit` to `DAC_UNIT` with the absolute base-system `/sbin/sysctl`, reads the value back, logs a change or failure, and makes `service omdrc_audio status` fail visibly when the readback does not match the resolved DAC. This setting matters to applications outside the project that open `/dev/dsp`; project components themselves continue to use `/dev/dsp.dac`. A silent best-effort write is insufficient: on a two-card box, leaving the default at pcm0 can route an unrelated application into the ESI capture interface even while BruteFIR correctly holds the DAC on pcm1.

Do not put `hw.snd.default_unit=N` in `/etc/sysctl.conf`. That file runs near the beginning of rc, before USB attach order and roles are known. A fixed number therefore creates two owners and can encode yesterday's enumeration. The role pass is the correct location because it already holds `device.lock`, has selected the card by stable identity, and runs again after each pcm attach/detach. It adds no lock and no independent race window.

3.5.2.4 omdrc_cdin

`omdrc_cdin` runs as the configured audio user. Internally it holds the capture role but opens its playback destination only while audio is present. At the source-policy level, however, a running bridge is exclusive: the controller remembers and disables MPD's audible output before Start and restores it after Stop, because `virtual_oss` would otherwise mix both sources.

Before replacing `virtual_oss`, `drc.sh` stops the bridge and waits to a fixed deadline for its process to exit. Process exit is the release acknowledgement; it does not depend on a logfile that may have been rotated or unlinked. Failure aborts CUSE teardown and attempts to restore MPD's direct output. After a successful chain transition, a bridge that was running is restarted with `onestart` so a panel-started instance survives even when its rcvar is off. The service's `release` extra command still sends SIGHUP for direct diagnostic use, but it is no longer the transition handshake.

3.5.2.5 omdrc_renderer

`omdrc_renderer` reads `last_renderer` and starts exactly one renderer: upmpdcli or qobuzconnect2mpd. It retains the selection at shutdown. Both worker rcvars remain `NO`; the helper deliberately uses `onestart`/`onestop`. Enabling a worker independently races the owner and may leave two front-ends driving MPD.

3.5.2.6 upmpdcli

The repository's `upmpdcli` script is a worker service. It supplies the audio user's HOME and a PATH containing `/usr/local/bin`, creates the user-owned pid directory, and captures inherited plugin stderr for the controller's log view. Its `REQUIRE` token is `musicpd`, not the Linux package name `mpd`.

3.5.2.7 omdrcctrl

`omdrcctrl` runs the Flask controller as the configured non-root user. Its pre-command creates user-writable pid/log directories; its environment sets HOME, PATH, DISPLAY, and optionally the shared `OMDRC_STATE_DIR`. It reads `/var/run/omdrc/audio.roles` without spawning a status command. The DRC panel includes a distinct `CD input (44.1 kHz)` action.

Its service identity is invariant across callers: `/var/run/omdrcctrl/omdrcctrl.pid` always names the `daemon(8)` supervisor. The former non-root branch silently selected `${TMPDIR:-/tmp}/omdrcctrl-USER.pid`; consequently an ordinary status probe reported the root-started service as stopped, and `onestart` could attempt a second instance. That branch was removed rather than hidden behind a custom `pgrep` status command. rc.subr already avoids a redundant `su` when the caller is the configured service user. `daemon -M 0644` makes the canonical PID readable for diagnostics while the containing directory and service lifecycle remain system-owned.

3.5.2.8 omdrcvideo

`omdrcvideo` starts only the video HTTP/API process. The persistent idle mpv belongs to the graphical login session and is not launched by rc. The rc script sets the non-root user, PATH, pidfile, and logfile and never participates in audio locking.

It follows the same one-service/one-pidfile rule at `/var/run/omdrcvideo/omdrcvideo.pid`. Neither service derives identity from `TMPDIR` or the invoking UID. FreeBSD rc.d remains the system lifecycle interface (`sudo service ... start|stop|restart`); a development process must use a distinct port and direct launcher, not masquerade as a second instance of the system service.

3.5.2.9 omdrc-audio.conf

The only project devd configuration matches `pcm[0-9]+` at attach and detach:

```
attach 100 {
    device-name "pcm[0-9]+";
    action "/usr/sbin/daemon -f /usr/sbin/service omdrc_audio reconcile";
};
```

The detach rule has the identical action. Matching `pcm` is a safety boundary. A UAC2 device exposes several USB interfaces but one sound card; matching USB class events caused several lifecycle runs for one plug. A broad USB detach rule could also react to a keyboard or storage device. At the pcm event, the kernel has allocated the unit and created its OSS nodes, so role resolution has facts to inspect and does not need a settle sleep.

3.5.2.10 What devd serializes — and what it does not

The current FreeBSD 15.1 source implements a direct action in `sbin/devd/devd.cc::my_system()`. It forks a child which executes `/bin/sh -c command`, while the devd parent waits for that exact child PID with `wait4()`. Consequently, a genuinely synchronous rule such as:

```
action "/usr/sbin/service omdrc_audio reconcile";
```

would block devd until that complete reconcile returned. If a detach arrived while an attach reconcile was running, the detach event would remain pending; after the first action returned, devd would execute the second. The second reconcile would not be lost and the two devd-originated service calls would not overlap under this implementation.

That is an implementation fact, not an API promise. `devd.conf(5)` documents that `action` names a command to execute, but it does not promise synchronous execution, non-overlap, or the present `fork()`/`wait4()` process model. Correctness must distinguish the FreeBSD 15.1 source behavior from a supported interface which could survive a future devd implementation change.

Our installed action deliberately changes the process lifetime:

```
devd
  -> sh -c "daemon -f service omdrc_audio reconcile"
       -> daemon launcher
            -> detached service omdrc_audio reconcile
```

devd waits for its exact shell child. The shell waits for the immediate `daemon` command, but `daemon` returns after creating the detached descendant; neither the shell nor devd follows that descendant and waits for the actual reconcile. In FreeBSD, `daemon` itself always performs the detachment and its `-f` option means **close inherited file descriptors** — it is not a "foreground" or "detach" switch as similarly named tools may use on other systems.

The resulting timing can be:

```
pcm attach A: devd starts daemon A -> reconcile A detaches -> launcher A exits
pcm attach B: devd starts daemon B -> reconcile B detaches -> launcher B exits
                                      reconcile A and B can now overlap
```

Thus devd still serializes the short top-level launcher commands, but it does not serialize their detached audio workers. This is intentional: a full reconcile may wait on bounded locks, MPD, BruteFIR, virtual_oss/CUSE, and hardware verification. Running it inline would freeze the machine-wide event loop and delay unrelated USB, network, input, storage, and ACPI events.

The device lock makes overlapping workers safe. It is acquired *before* each worker scans pcm state, so a waiter does not later publish a snapshot collected before it waited. For example:

```
A takes device.lock; scans ESI only
OKTO attaches; B starts and waits
A publishes capture-only state; releases device.lock
B takes device.lock; scans ESI + OKTO; publishes both roles and links
```

The inverse detach case is equally important. Without the lock, an old pre-detach scan could recreate `/dev/dsp.dac` after a newer worker removed it. With the lock, the later waiter scans after acquiring the lock and therefore observes the final device tree. Every request reconstructs complete level state; no attach or detach edge is interpreted directly as an instruction to start or stop DRC.

A detached **rejecting singleton** is not equivalent to either the synchronous model or this queued-lock model. Consider a pidfile wrapper which exits when a worker already exists:

```
A starts and scans: DAC present
DAC detaches
B's top-level devd action runs, but its singleton refuses to start reconcile B
A publishes its older "DAC present" result
```

devd did not lose the detach event: it successfully executed B's launcher. The application-level singleton discarded the reconciliation requested by that event. A correct coalescing singleton would need to set a pending/dirty flag and force A to run another complete scan before exiting. That reintroduces a marker, wake-up protocol, and worker lifecycle while the scan/update transaction still needs serialization. The present lock queues the waiter instead of rejecting it, so the second pass performs a fresh scan.

Authoritative references are the FreeBSD 15.1 `devd.cc` implementation, `devd(8)`, `devd.conf(5)`, `daemon(8)`, and `lockf(1)`.

3.5.3 Locking, concurrency, and bounded waits

Only two locks remain, with a strict non-nesting rule:

Lock	Owner	Protects	Lifetime
`/var/run/omdrc/device.lock`	root `omdrc_audio roles`	role discovery, four links, sysctls, recsrc, role publication	one short role transaction
`STATE_DIR/drc.lock`	audio-user `drc.sh`	saved intent reads/writes and the physical chain transition	one mutating DRC command

Both FreeBSD calls use `lockf -k -s -t ...`. `-k` keeps one inode after release; lockf(1) specifically recommends it for concurrent callers because unlink/recreate mode cannot guarantee waiter ordering. File existence does not mean locked. `/var/run` is correct for root boot-lifetime device state; the DRC lock belongs beside persistent user state, and its path is never derived from TMPDIR. A desktop session and a boot login shell therefore cannot silently choose different locks.

The order is:

```
devd or rc
  -> omdrc_audio roles       [take device.lock; update facts; release]
  -> su -l AUDIO_USER
  -> drc.sh reconcile        [take drc.lock; converge; release]

musicpd successful start
  -> the same omdrc_audio reconcile path (the hook owns no lock)
```

No path takes `drc.lock` and then asks for `device.lock`, and `omdrc_audio` releases `device.lock` before entering the user reconciler. This removes the former three-lock nesting. Several simultaneous pcm events may wait for the short role transaction, then wait for `drc.lock`; the first repairs state and later calls become no-ops. Lock waits have finite timeouts.

The apparent simplification “devd serializes, so delete the locks” is therefore incorrect for four independent reasons:

1. `daemon -f` intentionally releases devd before the real worker finishes.
2. Boot rc, the MPD post-start hook, administrators, the UI, and direct `drc.sh` commands are not serialized by devd.
3. A rejecting asynchronous singleton can discard the reconcile requested by a busy attach/detach event even though devd delivered and executed the top-level action; a queued level rescan must observe the final device tree.
4. The manuals do not guarantee the current internal `wait4()` behavior.

Removing `daemon` would make long audio waits compromise the entire device subsystem. Replacing the queued lock with debounce plus a dirty marker would add a marker, timer, and worker lifecycle while the scan/update transaction would still need serialization. Combining the locks would hold the short root device transaction across the long audio-user chain transition and recreate cross-privilege lock coupling. The implemented two non-nested locks are thus the simplest safe design, not leftover scaffolding.

The command under FreeBSD’s `lockf file command` is supervised by the lockf process. The lock is released when the orchestration command exits; BruteFIR and virtual_oss do not become lifecycle-lock owners. Linux uses an explicit flock descriptor and closes it in daemon children; the Linux audit is treated separately.

External waits are bounded while `drc.lock` is held. Every `mpc` call goes through a timeout wrapper. A slow or unavailable MPD does not block boot and does not prevent a verified physical chain from becoming active; the pending output selection is logged, and `musicpd`'s successful-start hook supplies the specific next reconcile that retries it. There is no sleep or indefinite MPD readiness loop. Service calls used to restart the CD bridge also have a deadline and use non-interactive sudo. BruteFIR startup, exit, virtual_oss readiness, DAC warm-up, verification, and CD release all use explicit poll caps.

Syslog is supplementary evidence, not lifecycle state. During the incident, syslogd still held a bound but unlinked `/var/run/log` socket; existing connected processes could retain descriptors while new `logger` calls failed silently. `omdrc_audio` therefore publishes factual role state directly and `drc.sh` appends to persistent `STATE_DIR/drc.log`; neither decides anything from syslog. Repair/restart syslogd before using log absence as evidence. The project audit found no broad `/var/run` deletion, and FreeBSD `cleanvar` explicitly excludes the `log` and `logpriv` socket names, so the unlink cause remains a separate system issue rather than an attributed project action.

3.5.4 Reconcile state and reboot persistence

The reconciler compares saved intent with physical reality:

- `last_power`: on/off;
- `last_source`: music/cdin;
- `last_arg`: native rate or resampling request and design;
- actual BruteFIR config/process, virtual_oss process/rate/nodes, and DAC role.

A matching chain is a no-op. A missing DAC causes a transient teardown while preserving intent. Desired off remains off. A partial or wrong-rate chain is rebuilt once. The old active marker is not consulted.

The `off` verb writes `last_power=off` before touching BruteFIR, MPD, CD input, or CUSE. This ordering matters under `set -e`: a teardown failure must not leave the previous on intent, which would bring DRC back at reboot. `stop` is different: shutdown and detach use it as a transient teardown and do not change desired power.

`restore` reads state only after acquiring the same DRC lock, eliminating the old time-of-check/time-of-use window. The explicit UI/CD verb is:

```
drc.sh cdin
```

It records `last_source=cdin` before requesting the 44.1 kHz chain. Reboot therefore restores CD mode at 44.1 kHz. An ordinary rate action is the explicit return-to-music action and writes `last_source=music` before DAC presence and configuration validation or any teardown/startup work. This is write-ahead intent, not a success marker: if the requested music transition fails, the next `restore` or hotplug `reconcile` retries music instead of resurrecting the stale CD input at 44.1 kHz. `off` and transient `stop` do not change the source. A geometry change in CD mode is refused if the target geometry has no 44100 Hz configuration. The status/UI distinguishes CD input from ordinary 44.1 kHz music.

3.5.5 The verified boot-recursion failure and its removal

The old `omdrc_sndlink` script re-entered itself under `lockf` using `$0`. This was safe under `service(8)`, where `$0` was the absolute script, but false during normal boot: FreeBSD's `/etc/rc` sources each rc.d script, so `$0` remained `/etc/rc`. The lock command therefore launched:

```
/bin/sh /etc/rc oneupdate
```

That caused a second rc pass: duplicated host UUID/network/route and service startup, devd already-busy errors, and long wireless timeouts. It was bounded to one extra pass only because the inherited lock marker suppressed another re-entry, but one nested boot was enough to destabilize the machine.

The emergency old-script correction used `rc_service`, because rc.subr sets it to the absolute rc.d path before sourcing the script. The migration removes that script entirely. `omdrc_audio` retains the safe form for its short locked `roles` re-entry:

```
/bin/sh "$rc_service" oneroles
```

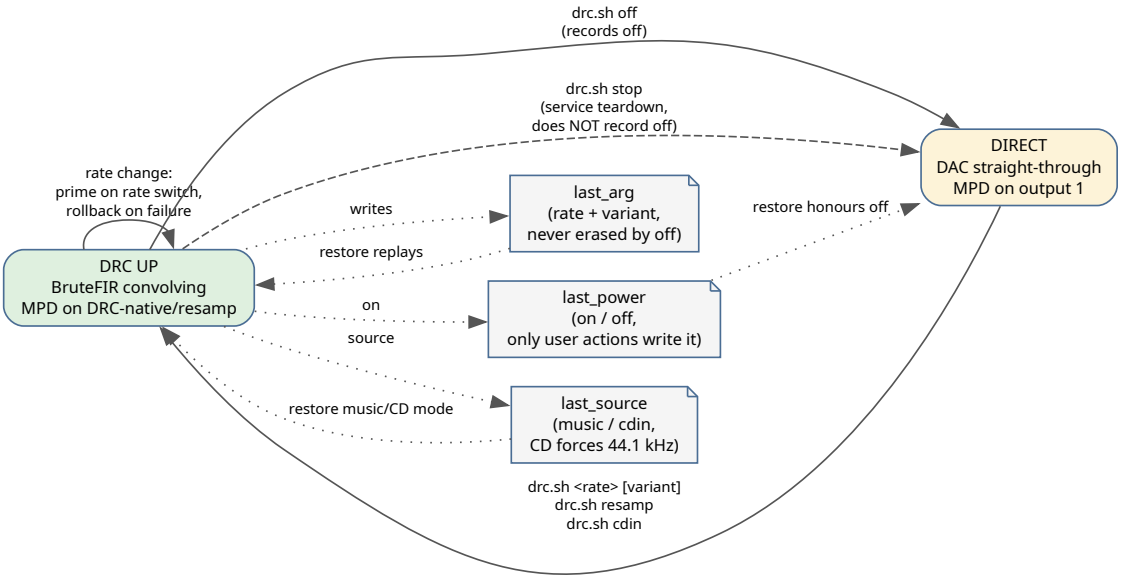
The implementation falls back to the executing script path only outside rc. There is no call to `/etc/rc`, no dirty/debounce recursion, and no helper that can replay boot.

4 Usage: drc.sh, filters, and configuration

4.1 drc.sh — the single control point

```
drc.sh <rate>|resamp|cdin|reconcile|restore|off|stop|status|session [variant]
```

Verb	Effect
`<rate>`	Start BruteFIR at 44100 / 48000 / 88200 / 96000 / 192000 Hz; restart the loopback at the same rate; switch MPD to `DRC-native`
`resamp`	Everything at 192 kHz; switch MPD to `DRC-resamp` (MPD resamples with soxr)
`cdin`	Persist CD/S-PDIF source mode and request its required 44100 Hz chain
`reconcile`	Compare saved power/source/rate with actual processes, config, rate, nodes and DAC role; repair only a mismatch
`restore`	Re-apply state at shutdown: honour `last_power`, `last_source`, and `last_arg`
`off`	Stop BruteFIR + loopback; MPD back to direct output; **records the off state**. The user-facing disable
`stop`	Same teardown as `off` but does **not** record off. Used by service stop paths so a reboot of a *running* system is restored
`status`, `session`	Report actual chain state or the exact persistent restore tuple
`geometry`, `design`	Show/list/switch the persistent room geometry or audited filter design
`variant`	Optional second argument (a config-filename suffix): selects an alternate filter set; superseded by `design`



drc.sh verbs and the persistent lifecycle state.

State lives in the resolved persistent state directory (git-ignored checkout state in run-from-repo mode, or the configured/installed user state path):

- **`last_arg`** — the last *active* rate and optional variant (`192000`, `resamp`, `192000 <variant>`). Written on each successful run, **never erased by `off`** — turning DRC back on restores the last rate. It records the *desired* state: a failed start never rewrites it, so the next trigger retries the same configuration.
- **`last_power`** — `on` or `off`. Only real user actions write it (a rate run writes `on`, `off` writes `off`; `stop` leaves it alone).
- **`last_source`** — `music` or `cdin`. CD mode survives reboot and forces 44100 Hz; an ordinary rate selection records the return-to-music intent before fallible validation/transition work, so failure cannot restore stale CD mode on the next boot.
- **`cdin-mpd-output`** — the audible MPD output remembered when CD input is selected (`OKTO-DAC`, `DRC-native`, or `DRC-resamp`). Stopping the bridge restores this exact output; the Spectrum FIFO is never recorded or gated.
- **`last_geometry`** — the current geometry override. Design remains part of `last_arg`, so the full tuple can be restored without guessing.

The configuration's `GEOMETRY` is the default; `drc.sh geometry <name>` records a runtime override in `last_geometry`.

What a required rebuild does, in order: stop any running BruteFIR and wait for the DAC to be released; make bounded MPD release requests; stop a running CD bridge and verify process exit as its output-release acknowledgement; (FreeBSD) restart `virtual_oss` at the target rate and wait for `/dev/dsp.loop`; prime the DAC if the rate changed; start BruteFIR and **verify it stays up**; make a bounded attempt to enable the matching MPD output; record the state. If BruteFIR cannot come up, `drc.sh` **rolls back** — stops the loopback and re-enables the direct output — leaving a clean, audible system equivalent to `off`.

The priming quirk: the OKTO DAC routes silence on the *first* stream opened at a new sample rate; a second open fixes it. On a detected rate change `drc.sh` opens BruteFIR once, tears it down, then starts it for real. (The kernel-level fix for this is the clock-before-alt patch, Appendix ; with it installed,

DAC_PRIME_CYCLES defaults to 0.)

`drc.sh status` (and `drc-status.sh`) reports the *actual, observed* state — config, loopback/ALSA rate, BruteFIR, MPD output and rate — derived from what is running, not from `last_arg`.

4.2 MPD outputs

Output	Device	format	Meaning
`OKTO-DAC`	direct DAC	—	bit-perfect direct path, no DRC
`DRC-native`	loopback	`*:*:*`	native-rate DRC; MPD does **not** convert; you pick the `drc.sh` rate matching the track
`DRC-resamp`	loopback	`192000:24:2`	MPD resamples everything to 192 kHz (soxr); for mixed-rate playlists

`*:*:*` (rate:bits:channels, asterisk = unenforced) is deliberate: native DRC mode must not force MPD conversion. `drc.sh 192000` and `drc.sh resamp` both use the 192 kHz BruteFIR config but are distinct modes: the web UI shows them as *Flat 192 kHz* vs *Flat auto-resample*. In native mode the sample rate is the routing selector, never the bit depth — BruteFIR processes in floating point.

4.3 The filters/ and configs/ trees

```
filters/<geometry>/<rate>/{L,R}.raw          raw FLOAT64_LE FIR coefficients
filters/<geometry>/<rate>/@<design>/{L,R}.raw   an immutable A/B design
filters/<geometry>/provenance/<design>.json  hash-bound manifest
filters/<geometry>/analysis/<design>.json    precomputed response traces
filters/<geometry>/rew/                      REW-exported source WAVs
configs/<geometry>/brutefir-<rate>[@<design>].conf.in
```

These two trees are the *site data*. They need not live in this checkout: CMake resolves them along `OMDRC_SITE_DATA_DIRS` and the design scripts along `OMDRC_SITE_ROOT`, so one room's measurements can be a separate repository while the engine ships only the generic `flat` set. See Filter provenance and verification.

`drc.sh` builds the config path as `configs/<geometry>/brutefir-<actual_rate><variant>.conf` (for `resamp`, `actual_rate` is 192000) and BruteFIR reads the filter paths from that config — the config is the authoritative link. A variant works only if the matching config file exists; nothing is auto-discovered.

For a new rate or variant, create all the pieces: the `L.raw`/`R.raw` pair, the config pointing at them, and verify the `attenuation:` (below).

4.4 Filter generation workflow

This is the low-level route: it produces coefficients but no provenance bundle, so the web UI cannot verify the result and will not show stored measurements beside it. For deployable work use the audited workflow in Filter provenance and verification; what follows is still useful for experiments and is what the audited path drives underneath.

1. Export the corrected impulse responses from REW as WAV (typically 48 kHz).
2. Convert for every rate directory:

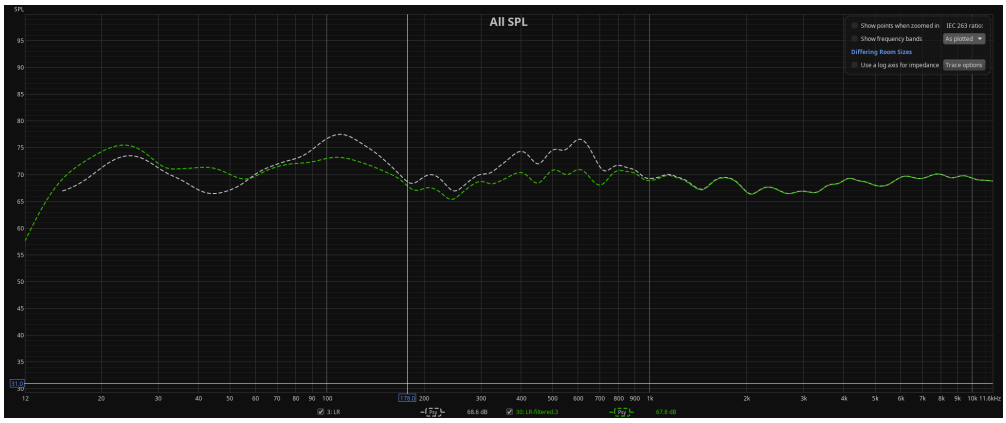
   ```
   scripts/REW2raw-all-rates.sh \
     -L filters/120.blue/rew/FLX-trimmed-48k.wav \
     -R filters/120.blue/rew/FRX-trimmed-48k.wav \
     -o filters/120.blue
   ```

   `REW2raw.sh` (called per rate) resamples with SoX at very high quality (`-v -L -s`, 64-bit float intermediate) and applies **one deterministic FIR coefficient scale** — `scale = Fs_source / Fs_target` (Julius O. Smith, *Physical Audio Signal Processing*) — never peak normalisation, which would alter the intended filter gain. A `sox.txt` log records every command and measured stat.
3. Compute the clipping headroom and set `attenuation:` in each config:

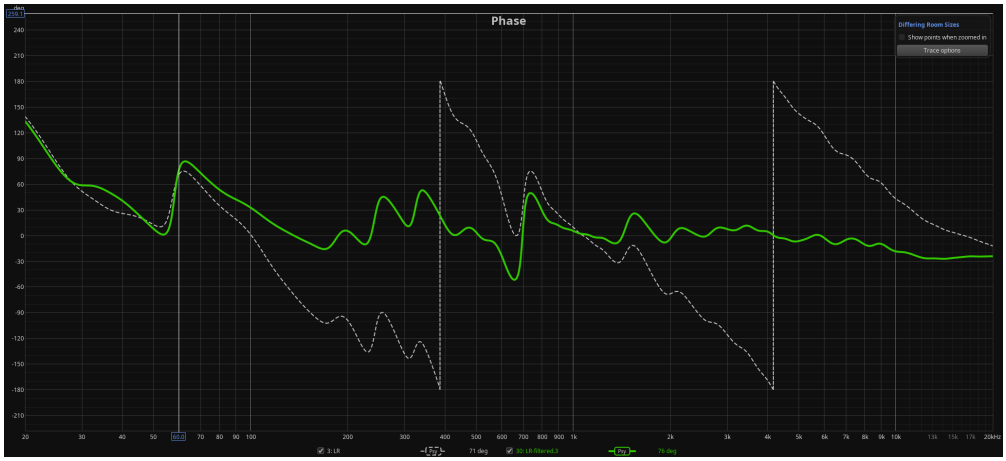
   ```
   python3 scripts/headroom_calc.py
   ```

   BruteFIR works in float64, so clipping can only happen at the output boundary where the filter has gain > 0 dB. The script FFTs each raw filter, takes the worst-case gain across frequency, adds a safety margin (default 1 dB), and reports the suggested per-pair attenuation (BruteFIR applies one `attenuation:` per coeff block to both channels, so the louder channel limits). Attenuation in float64 is lossless — the only goal is avoiding clipping.

The measured result of the current filter set (`120.blue`, v1.5.0):



Amplitude response: corrected vs uncorrected.



Phase response: corrected vs uncorrected.

4.5 Browser audio without DRC

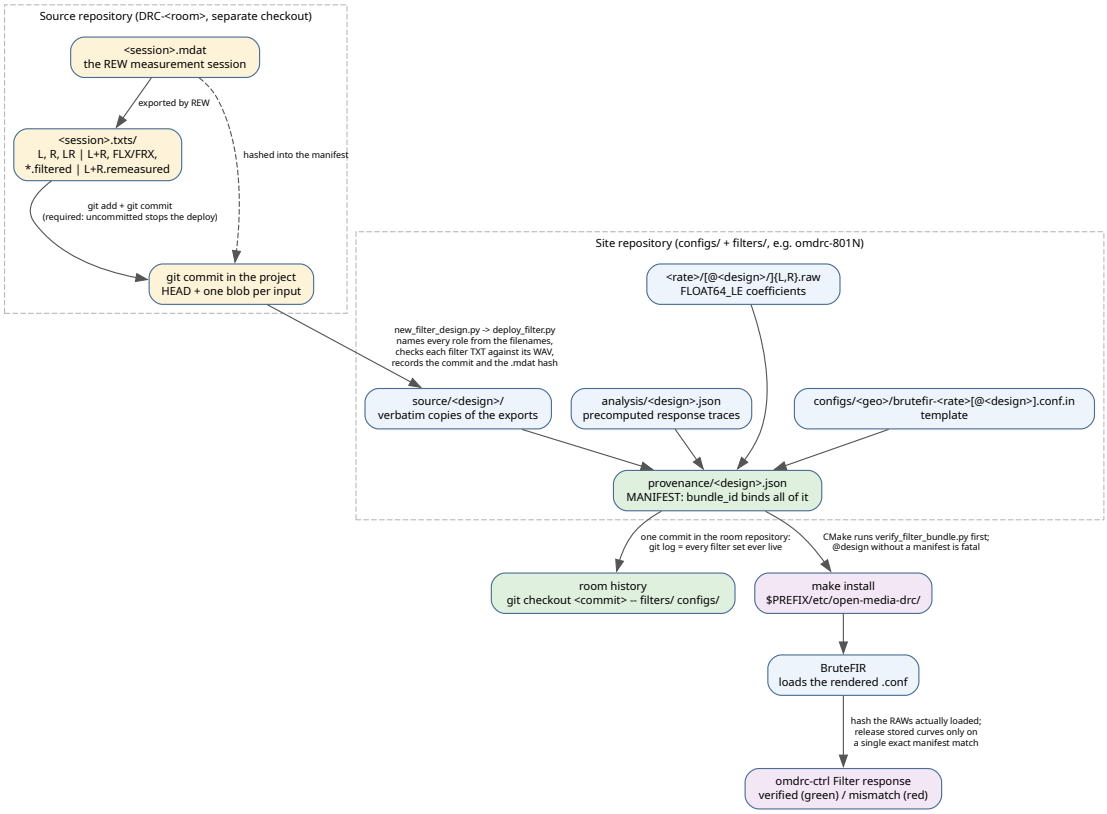
While DRC runs, BruteFIR holds the DAC single-open and browsers get silence. Each `browser-nodrc/` launcher: (1) snapshots `last_power` + `last_arg`; (2) runs `drc.sh off`; (3) runs the browser in the foreground; (4) restores the exact pre-launch state from an `EXIT/INT/TERM` trap. It deliberately does *not* use `drc.sh restore` (which would honour the `off` just recorded and leave DRC down). Firefox runs with `--no-remote`; Chrome/Chromium detect an already-running instance with `pgrep` and then just hand over the URL without touching DRC. The rendered `.desktop` launchers are symlinked into `~/.local/share/applications/`.

5 Filter provenance and verification

A room-correction filter is an empirical artifact: it is only as good as the measurement session it came from, and it is indistinguishable, once converted to a `.raw` blob of float64 coefficients, from any other blob of the same length. The response page in the web UI shows *stored* room measurements — curves that were computed offline, months earlier, from the original REW exports. Showing them beside a filter that is not the one they describe would be worse than showing nothing, because it looks authoritative.

This chapter describes the machinery that prevents that: a chain of content hashes running from the REW exports in the source repository to the coefficient bytes BruteFIR has actually loaded, with a refusal at every link that cannot be checked.

What follows is the synthesis: what the chain is for, and how it hangs together. The *normative* detail lives beside it in `doc/FILTER_PROVENANCE_AND_RESPONSE.md` (built to a printable `doc/FILTER_PROVENANCE_AND_RESPONSE.pdf` by the same script as this manual) — what every export must be, what may never be shown as verified, the exact deployment and removal transactions, and the audit record of the bundles actually deployed in this room. Read this chapter to understand the design; read that document when you need the rule.



Provenance chain: each labelled arrow is a content check that must pass before a design is deployable, installable, or shown as verified.

5.1 The three repositories

The workflow spans three trees, deliberately kept apart:

Tree	Holds	Example
Source repository	one geometry's REW projects (`.mdat`) and the sessions exported from them (`<session>.txts/`)	`../DRC/DRC-120.blue`
Site repository	`configs/<geometry>/`, `filters/<geometry>/` — one physical room	`omdrc-801N`
Engine repository	`drc.sh`, the scripts, CMake, and the generic `flat` set	`open-media-drc`

The site data used to live inside the engine checkout. It no longer has to: `configs/<geo>` and `filters/<geo>` are resolved through a *site root*, so personal room measurements can be versioned and deployed independently of the software. Two settings control it, and they are not the same thing as the runtime `OMDRC_SITE_DIR`:

Setting	Read by	Meaning
`OMDRC_SITE_DATA_DIRS`	CMake	semicolon-separated *search path* for `configs/<geo>` + `filters/<geo>`; first match wins
`OMDRC_SITE_ROOT`	the design scripts	the one checkout they read and write room data in (also `--site-root`)
`OMDRC_SITE_DIR`	`drc.sh` at runtime	the *installed* `$PREFIX/etc/open-media-drc`

Both default to the engine checkout, which is exactly the historical single-repository layout. Set the search path in `host.cmake`:

```
set(OMDRC_SITE_DATA_DIRS "${CMAKE_SOURCE_DIR};$ENV{HOME}/devel/omdrc-801N"
    CACHE STRING "Search path for configs/<geo> + filters/<geo>")
```

At configure time CMake prints which directory supplied each set, so the substitution is never silent:

```
-- core-drc: filter set search path
--     /home/giacomo/devel/open-media-drc (this checkout)
--     /home/giacomo/devel/omdrc-801N
--   120.blue (default) <- /home/giacomo/devel/omdrc-801N
--   flat <- /home/giacomo/devel/open-media-drc
```

A missing *extra* set is a warning and is skipped; a missing default `GEOMETRY` is a fatal error, because installing it would record a geometry in `omdrc.conf` with no configs behind it.

5.2 Geometry, design, variant

Three words that are easy to confuse:

- a **geometry** is a physical setup — speaker and listening position, e.g. `120.blue`. It is a directory under `configs/` and `filters/`.

- a **design** is one immutable filter revision inside a geometry, addressed as `@design-id` (e.g. `@rscreen-20260812`) and carrying a provenance manifest. The historical revision has the reserved id `default` and keeps the un-suffixed paths.
- a **variant** is the older mechanism: an arbitrary suffix appended to the config filename. It survives for compatibility, has no manifest, and is therefore always displayed as unverified. New work should use designs.

5.3 The bundle

Everything belonging to one design lives in the site repository under the geometry:

```
filters/<geometry>/
  provenance/<design>.json          the manifest (the commit marker)
  provenance/<design>.source.json   the build recipe (development input)
  analysis/<design>.json            precomputed response traces
  source/<design>/                  verbatim copies of the ten inputs
    L.txt  R.txt  LR.txt            measured, before correction
    FLX-trimmed.txt  FRX-trimmed.txt          exported filter responses
    FLX-trimmed-48k.wav  FRX-trimmed-48k.wav  deployable impulses
    L.filtered.txt  R.filtered.txt  LR.filtered.txt   after correction
  <rate>/{L,R}.raw                  runtime coefficients (design `default`)
  <rate>/@<design>/{L,R}.raw        runtime coefficients (immutable design)
configs/<geometry>/
  brutefir-<rate>.conf.in           template; @REPO_DIR@ -> $SITE_DIR at install
  brutefir-<rate>@<design>.conf.in
```

Sources keep the names they had in the export directory, because those names *are* the role assignment. The copy is deliberate: a deployment must never depend on a mutable sibling checkout or on an absolute path that will not exist on the playback machine.

5.4 What is hashed

The manifest records, for every source export, RAW file, config template and the analysis file: the logical role, relative path, byte size, format, sample rate/count where applicable, and SHA-256. Around that it records where the design came from — the source project's repository, remote, branch, HEAD commit and subject, the repository-relative path of the export directory, a Git blob id per input, and whether everything was committed — together with the REW `.mdat` those exports were taken from, by name, size, SHA-256 and blob id. It then records the aggregate convention taken from the filenames, parsed REW header metadata (measurement name, date, notes, smoothing, frequency step, timing reference, REW version), the TXT-versus-WAV validation results, per-channel headroom with safety margin and required attenuation, and the exact rate-to-config mapping with the expected BruteFIR format and attenuation.

The `bundle_id` on top is the SHA-256 of a canonical identity object containing a hash of the complete source block — export directory, project, session and every artifact record — plus every source artifact hash, the runtime config and RAW hashes and settings, and the analysis hash. Editing any provenance value the UI displays therefore invalidates the bundle rather than quietly changing a label; that includes the project commit and the `.mdat` hash.

A hash proves bytes; it cannot say where they came from or bring them back. The commit id does that. It is the only part of the chain that is not self-verifying, which is why the deployment refuses to run until the exports and the `.mdat` are committed.

5.5 The scripts

All of them are offline. None starts REW, and none opens a `.mdat` project except the optional auditor below.

Script	Role
`new_filter_design.py`	the one deployment command: reads one export directory, resolves every role from the file names, checks each filter TXT against its impulse WAV, hashes the `.mdat`, requires the project commit, reports, asks, publishes, and commits the room repository
`remove_filter_design.py`	the exact inverse: removes one design completely — manifest, analysis, source copies, every rate's coefficient pair and every config template — manifest first, then records the removal in the room repository. Refuses the reserved `default` set
`deploy_filter.py`	the engine underneath: regenerates every requested rate in a temporary directory, validates TXT against WAV, computes headroom, bakes and reads back each config, carries the exports into the analysis file unchanged, and writes the bundle
`verify_filter_bundle.py`	read-only re-verification of committed bundles: bundle id, source copies, analysis dependencies, configs, exact RAW hashes and headroom. `--no-next` for CMake and CI
`console_ui.py`	shared terminal contract for publication and removal: stages, colours, confirmation, warnings, and the uniform failure line
`filter_workflow_next.py`	shared operator handoff — prints the exact commit/install/select/verify commands, and names a working directory per step when the site data lives in its own repository
`headroom_calc.py`	minimum `attenuation:` per pair from the worst-case FFT gain plus a safety margin; also reports what each config currently specifies
`rew_mdat_audit.py`	optional archival evidence: audits selected REW project traces via the REW API, comparing TXT responses numerically and final WAV impulses sample-by-sample against the project. Not a deployment dependency
`REW2raw.sh`, `REW2raw-all-rates.sh`	the low-level SoX conversion underneath, usable directly for experiments; they produce no provenance bundle

5.6 The workflow

```
# 1. In the source repository: give the exports their imposed names, commit them
#    together with the .mdat they came from.
git -C ../DRC/DRC-120.blue add -- 120.blue.Rscreen.txts 120.blue.Rscreen.mdat
git -C ../DRC/DRC-120.blue commit -m 'Rscreen measurement session'

# 2. In the engine repository: one command, one directory.
export OMDRC_SITE_ROOT=~/devel/omdrc-801N
python3 scripts/new_filter_design.py ../DRC/DRC-120.blue/120.blue.Rscreen.txts

# 3. Re-verify independently, then push the room's history.
python3 scripts/verify_filter_bundle.py --all --require-sources
git -C ~/devel/omdrc-801N push
```

Nothing on the command line says what a file is: `L.txt`, `R.txt`, `LR.txt` (or `L+R.txt`), `FLX-trimmed.txt`, `FRX-trimmed.txt`, the two impulse WAVs, and `L.filtered.txt`, `R.filtered.txt`, `LR.filtered.txt` (or `L+R.filtered.txt`, or `L+R.remeasured.txt` when the room was measured again with the DRC running). A missing name, a duplicate spelling or a mismatched aggregate style stops the run in colour before anything is written.

Three checks run before anything is hashed or written, and none of them has an override. Every text export must be **unsmoothed** (`* Smoothing: None`; one that states no smoothing is refused too, since it cannot be shown to be unsmoothed) — REW's smoothing is baked into the numbers and cannot be undone downstream, while the browser's Smoothing selector is a separate, reversible view. Every text export must come from a **measurement at 48 kHz or below**, checked as *it must not reach past 24 kHz*, because the runtime coefficients are all resampled from one 48 kHz impulse. And each **filter TXT must be the exported response of its WAV**: one integer causal delay and one constant export gain are detected, then the residual magnitude and phase errors must stay inside the declared limits — the only DSP in the pipeline, and what makes the plotted FLX/FRX curve a statement about the bytes BruteFIR will load.

The command reports the eight curves the web remote will plot, the project and session behind them, the two filters with their TXT/WAV residuals and every path it will write, and then asks. `--dry-run` runs every check, including the SoX conversions, and stops without asking.

The safety controls are intentionally narrow. `--yes` supplies confirmation but skips no check. `--replace-design` permits differing bytes under an existing design id but does not permit partial publication. `--allow-uncommitted` records the source-recoverability gap as `clean: false`, which remains visible to the verifier and UI. `--no-commit` gives up the automatic room-history commit but does not alter bundle verification. `--site-root` selects the site checkout for the scripts; it is separate from CMake's `OMDRC_SITE_DATA_DIRS` search path. The web frontend requires a clean site work tree and a successful deployment commit when its configured design root is Git-managed; only a plain directory uses the explicit uncommitted/no-history mode.

Publication is a transaction. Without confirmation every check runs in temporary storage and nothing is touched; the dry run also reports which runtime files *would* change. On confirmation the order is fixed: source copies first, then the analysis, then the runtime RAW pairs, and the manifest **last**. The manifest is the commit marker — readers ignore an incomplete deployment until it exists and verifies every preceding hash. Replacing bytes that already exist additionally requires `--replace-design`, so an accidental overwrite of a deployed design cannot happen silently.

A design owns `filters/<geo>/source/<design>/` and every `filters/<geo>/<rate>/@<design>/`, so after publication those directories hold exactly what the new manifest names — a redeployment that renames its inputs prunes what the previous one left there. Pruning runs after the manifest, so a failure leaves harmless leftovers rather than a manifest naming a deleted file. A bare `filters/<geo>/<rate>/` is shared with the `default` set and is never touched.

Removal inverts all of this:

```
python3 scripts/remove_filter_design.py --list
python3 scripts/remove_filter_design.py 120.blue@rscreen-20260812
```

It deletes the manifest, the recipe, the analysis file, the source copies, every `@design` coefficient directory and every `brutefir-<rate>@<design>.conf.in`, and nothing else. The manifest goes **first**: while it exists it is the claim that the rest is present, so removing it is what makes the design cease to exist for every reader. The reserved `default` set is refused, because a geometry falls back to it. The removal becomes one commit in the room repository, and the design stays recoverable from the deployment commit that introduced it.

Step 2 ends with one commit in the room repository, staging `filters/<geometry>` and `configs/<geometry>` and naming the geometry, design, bundle id, project commit, session hash and rates. That commit is the deployment history: `git log` lists every filter set that was ever live, and `git checkout <commit> -- filters/<geo> configs/<geo>` brings any of them back byte for byte. `--no-commit` skips it. Verification never depends on it.

5.7 Deploying the verified identity

Publication changes the site repository, not the installed playback tree. The handoff has four boundaries: the source-project commit makes every export and the `.mdat` retrievable; the site-repository commit records the published bundle; CMake verifies and copies that bundle; and the running BruteFIR config plus exact L/R RAW hashes determine whether the web UI can go green.

On the design machine, `new_filter_design.py` makes the site commit by default. Re-verify those bytes and push the site repository:

```
export OMDRC_SITE_ROOT=~/devel/omdrc-801N
python3 scripts/verify_filter_bundle.py --all --require-sources
git -C "$OMDRC_SITE_ROOT" status --short
git -C "$OMDRC_SITE_ROOT" push
```

On the playback machine, pull the site repository. `host.cmake` must include it in `OMDRC_SITE_DATA_DIRS`. Initialise a first build from that file; later deployments can reuse the correctly initialised cache:

```
git -C ~/devel/omdrc-801N pull

# First build for this host
cmake -C host.cmake -S . -B build

# Later deployments may reconfigure the existing cache
cmake -S . -B build

cmake --build build
sudo cmake --install build
```

Configure prints the checkout that supplied each geometry and runs the read-only verifier before anything can be installed. The install contains the RAWs, rendered configs, manifests and analysis JSON; source exports and recipes remain development-only in the site repository.

Restart the panel, select the installed geometry and immutable design, and then verify what is actually running:

```
# FreeBSD; use systemctl restart omdrcctrl on Linux
sudo service omdrcctrl restart
/usr/local/bin/omdrc geometry 120.blue
/usr/local/bin/omdrc design --list
/usr/local/bin/omdrc design @rscreen-20260812
```

Open `http://<box>:9090`, enter **Filter response**, and require the green identity to show the selected geometry/design and the complete `bundle_id` printed during publication and verification. A missing or different identity means the curves must not be trusted. The scripts' **NEXT** block is the host-specific version of this sequence and names the correct working directory for each step in a split engine/site layout.

5.8 Live browser-driven installs

The sequence above is the offline/Git path. `omdrc-ctrl`'s **`/configuration`** page (section) drives the same audit/build engine from a browser on the trusted LAN, with no git checkout or shell access to the box, and is the normal way an operator installs or removes a room-correction design. The command-line procedure remains for offline publication and machine provisioning; it is not a follow-up step after a web install, because the web page already invokes the identical pipeline.

The browser uploads one REW-exported `.txts` directory plus its matching `.mdat`. `omdrc-ctrl/src/configuration.py` stages the upload under a private per-job directory, rejecting anything but an exact-basename pair, a mixed export folder, or a mismatched `.mdat` — selecting the common parent of the pair fills the session field automatically, but a browser cannot inspect `../` after access was granted only to the `.txts` directory, so selecting the export directory itself leaves one explicit `.mdat` choice, and both browser and server require its basename to match.

`new_filter_design.py` gained the flags this needs: `--live` publishes runnable `.conf` files (instead of `.conf.in` templates) into `--site-root` rather than the repo checkout; `--coefficient-root` bakes an absolute live path into those configs when staging happens outside the real site; `--archive-mdat` copies the full `.mdat` into the deployed source bundle instead of only hashing it; `--upload-provenance` records a browser upload's own name as its provenance instead of a temporary staging path.

Every web publication first enters the persistent `[configuration] design_root`. The Configuration page displays this server-configured path. If that directory is already a Git work tree, it must be clean before publication and the deployment commit is required before runtime installation, exactly as in the manual sequence above; the frontend does not pass `--allow-uncommitted` in this case. Otherwise it is an ordinary managed folder and the explicit uncommitted/no-history mode is used. When the live site is not writable by the unprivileged web process, the design is staged and handed to `scripts/omdrc-config-helper.py`, the same privileged helper that pins audio hardware roles (section): it re-verifies every claimed hash before copying anything and writes the manifest **last**, so a reader can never observe a claim before the bytes behind it exist — `site_root` is derived runtime state, not a second authority. A repeated install of an already-live, byte-identical bundle is a verified no-op (`installed_bundle_matches()` in `deploy_filter.py`) rather than a re-write. Progress streams back to the browser over Server-Sent Events. Installation never activates a design, and a running or saved design must be switched away before removal — both enforced the same way whether triggered from the page or the command line.

5.9 Verification at install time

`cmake/core-drc.cmake` runs `verify_filter_bundle.py --require-sources` over every manifest of a geometry *before* installing it, and fails the configure step if a bundle does not verify. It also rejects any `@design` config that has no same-named manifest. A broken bundle therefore stops the build rather than producing an installed system that looks authoritative.

The install copies the manifests and analysis JSON beside the RAWs, and deliberately excludes `rew/`, `source/` and the `.source.json` recipes: their hashes and parsed metadata are already embedded in the installed manifest, so the playback machine needs none of the development material.

5.10 Verification at runtime

The response page never trusts the geometry name or the rate. For every request `omdrc-ctrl`:

1. finds the `.conf` of the **running** BruteFIR process from its argv;
2. parses the coefficient blocks and hashes the exact `.raw` files named there;
3. requires exactly one manifest matching that `(relative path, SHA-256, format, attenuation, rate)` tuple;
4. verifies the analysis file's own hash and that its recorded input hashes equal the manifest's source artifact hashes;
5. releases the stored exports **unmodified** — no scaling, offset, attenuation subtraction or resampling stands between the analysis file and the browser.

Only then are the stored room measurements released, with the green banner carrying the bundle id; the details panel adds the export directory, the source project and commit, the `.mdat` behind the measurements, every plotted export with its hash and the active L/R RAW hashes.

Anything else is **mismatch** (red): no manifest matches the active bytes, a hash differs, the config attenuation or format differs, or an analysis dependency fails. **No graph is drawn** — not even a diagnostic FFT of the live coefficients, because a calculated curve on a page whose whole promise is *these are REW's numbers* is worse than no curve. A legacy variant, having no manifest, always lands here.

A/B switching obeys the same rule. After `drc.sh` returns, the server re-reads the running process, parses the config actually in use, hashes its RAWs, and reports the new selector as verified only if that runtime identity matches one manifest. A design that starts but fails this check is an assurance failure, not a successful switch.

5.11 What deliberately cannot go green

- A pair whose bytes do not reproduce from the recorded source exports. It must be re-exported and redeployed as a new bundle.
- Any selector without a manifest, whatever its audio quality.
- A design whose configured attenuation is below the computed requirement: the base `120.blue` pairs peak at about +1.28 dB and need 2.3 dB including the 1 dB margin, and are configured at 3.0 dB. A design whose filters never exceed unity gain legitimately requires 0.0 dB.

One gap is worth naming explicitly: the manifest pins the config *template*, not the rendered `.conf` that BruteFIR loads, because `@REPO_DIR@` is only substituted at install time. Runtime verification closes this for everything that matters — coefficients, format, attenuation and rate are all re-checked against the bytes in use — but a post-install hand-edit to a rendered config's routing or device settings is outside the chain.

6 Tools

6.1 omdrc-ctrl — the web control panel

A Flask app serving a dark, touch-friendly control panel to any browser on the LAN (default `0.0.0.0:9090`). Originally a replacement for KDE Connect's feedback-less "Run command" plugin. Everything is driven by a plain INI file, `commands.conf`; no command is hard-coded.

Widget types — each `[section]` of `commands.conf` is one command:

- **READ** — runs a shell command, shows its output next to a label, optionally auto-refreshing (`refresh = N` seconds). A READ widget with `details_root` gains a dynamic **Details** button whenever `{details_root}/{output}/README.md` exists — so each filter configuration can carry its own documentation page with images.
- **WRITE** — a labelled button firing a command; green on success, red on failure, optional confirmation dialog (`confirm = yes`).
- **LINK** — opens a URL in a new tab.

Built-in monitoring panels (always present, below the command cards):

- **MPD panel** — the audio-health centrepiece for a headless server: daemon state, playback state and song, the stream MPD reports (rate/bits/ channels), the **DAC feed** (ALSA `hw_params` read from `/proc/asound` on Linux; the `virtual_oss` rate on FreeBSD), the BruteFIR rate, a green/red **SAMPLE RATE MATCH / RESAMPLING** comparison, and a plain-language **bit-perfect verdict**: *Bit-perfect passthrough* (DRC off, all rates equal), *Full-resolution DRC, no resampling* (BruteFIR at native rate), or *Resampling active*.
- **Qobuz Connect panel** — current track via qobuzconnect2mpd, with restart button and colour-coded log viewer; plus a renderer switch (qobuzconnect2mpd vs upmpdcli — never both) driven per-OS (`systemctl --user` on Linux, `sudo service ... onestart/onestop` on FreeBSD).
- **CD input panel** — what the FreeBSD `omdrc-cdin` daemon or Linux `alsaloop` supervisor is doing, read entirely from its log: an availability LED, the state in one line, the newest failure in another, the `[stats]` line as chips, and a Start/Stop source hand-off. Start remembers and disables MPD's audible output; Stop restores it. The Spectrum FIFO is deliberately outside this gate. The card is the size of its news — one line while no disc is playing, full size while one is (chapter).
- **BruteFIR CPU** — per-process CPU for every brutefir instance (matched by `argv[0]`, because on Linux brutefir renames its `comm`).
- **Audio Devices** — `/dev/sndstat` on FreeBSD with `fmt 0x...` bitfields decoded to `AFMT_*`/`PCM_CAP_*` labels; collapsible.
- **Advanced** (FreeBSD only) — `sysctl dev.pcm.<DAC unit>` (resolved from `/dev/dsp.dac`) and `sysctl hw.usb.uaudio` diagnostics.
- **Top CPU** — processes above a configurable threshold.
- **Debug card** — the glitch-detection switch (section).

DRC filter response page — charts the *live* filters loaded by the running BruteFIR: magnitude (dB), delay-compensated wrapped phase, and residual group delay, computed on demand by FFT (NumPy) from the active config's `L.raw`/`R.raw`. Chart.js is vendored, so the page works offline. The filter files are only ever read, never modified.

Live spectrum analyzer — an optional card fed from a FIFO of raw S32_LE stereo. It FFTs whatever arrives and knows nothing about who wrote it, which is what lets more than one part of the chain feed it. `source` picks the producer: `mpd` (a secondary `OMDRC Spectrum` fifo output), `cdin` (the CD / S-PDIF bridge), or `auto` — the default — which takes `cdin` while a disc is actually playing through the bridge and `mpd` otherwise. CD audio never passes through MPD, so without the `cdin` source the analyzer is simply blank for the whole of a disc. The rate travels *with* the source, because CD is 44.1 kHz where the MPD FIFO is 48 kHz and analysing at the wrong rate mislabels every bin while reporting no error.

How the CD PCM reaches the FIFO differs by operating system. On FreeBSD `omdrc-cdin` tees it itself from the period it is about to write, opened non-blocking and dropping rather than ever delaying a write to the DAC; the reader's presence is the entire protocol, so nothing happens until the panel opens the FIFO. On Linux `alsaloop(1)` is opaque and the supervisor never sees a sample, so omdrcctrl reads the *same capture device* a second time through an ALSA `dsnoop` and writes the FIFO itself — deliberately an independent reader, so that a stalled analyzer misses samples instead of stalling the CD.

- Started/stopped from the page; the source is enabled only while at least one browser is streaming (Server-Sent Events; multiple clients share one capture thread), force-disabled at startup for crash recovery.
- 24 logarithmic bands from 31.5 Hz, 25 Hz refresh, Music (16384-point) vs Precision (65536-point) FFT windows, VU bars or needles measured over a ~50 ms trailing window, one Floor slider driving graphs and meters.
- Each band peak-holds across the complete publication interval, so short transients cannot fall between FFT windows. Rises are immediate; the browser animates only the release at `fall_db_per_s` (30 dB/s by default), including after the server stops publishing unchanged frames.
- **DRC sync**: the tap is at the top of the DRC path, so the display is held back by a clock-anchored estimate of the whole running chain: `virtual_oss` blocks, filter group delay, the convolver partition, BruteFIR I/O partitions, and DAC/USB output delay. The card shows the term-by-term breakdown and base/delta/total values. **Auto sync delay** periodically follows rate, process and filter changes; with it off, `drc_delay_trim_ms` replaces the modelled buffering terms. The Sync slider is persistent and takes effect on the next frame because the buffer reserves its full travel.
- With `source = auto`, an open card follows MPD-to-CD and CD-to-MPD hand-offs without closing the browser stream. MPD owns and creates its FIFO; the analyzer never replaces that inode. Writer loss and FIFO replacement are detected, stale pre-pause history is dropped on resume, and a disconnected browser stream reconnects automatically.

The delay model exposes these configuration terms:

Stage	Derived value
virtual_oss	`drc_voss_blocks` times the running process's `-s` duration
filter	peak index of the active impulse response divided by sample rate
convolver	one BruteFIR `filter_length` partition
BruteFIR I/O	`drc_brutefir_io_partitions` additional partitions

Stage	Derived value
physical output	`drc_output_delay_ms` for the OSS/DAC buffer and USB path

The defaults are three virtual_oss blocks, two additional BruteFIR I/O partitions, and 150 ms of output delay. The built-in `dirac pulse` coefficient has zero group delay, but still pays the convolver partition. Because virtual_oss's `-s 200ms` is a duration, that term stays constant across sample rates; the partition terms shrink as the rate rises. When the DRC chain is down, every term is zero. This is a configuration-derived estimate, not an acoustic measurement; `omdrc-ctrl/tools/measure-drc-delay.sh` remains the disruptive end-to-end calibration path.

Hold-back is anchored to elapsed time rather than a fixed offset from the last byte received. The read point therefore drains the buffered tail when a writer stops, and non-blocking CD-FIFO drops cannot permanently walk the display out of sync. After a silence gap, retained PCM is discarded before the source resumes so the analyzer cannot show the previous track while the new one is still travelling through the chain.

The analyzer enables MPD's Spectrum output first and then opens the FIFO inode MPD created. It never creates or replaces that path: MPD keeps its existing descriptor open and would otherwise silently write an unlinked inode until it is restarted. The CD FIFO has the opposite ownership — omdrcctrl creates it, and the appearance of a reader tells the bridge to tee samples. Identical frames are suppressed, so paused playback produces no SSE traffic.

Install: omdrcctrl has no standalone deployment. Configure and install the top-level open-media-drc project so the panel, core wrappers, site data, and state share one host configuration (systemd system unit on Linux, rc.d script on FreeBSD). On FreeBSD: `sysrc omdrcctrl_enable=YES && service omdrcctrl start`; the rc.d script uses `daemon(8)`, drops privileges via the standard rc.subr `${name}_user`, and keeps its pidfile in a subdirectory of `/var/run` created by `start_precmd` (plain `/var/run/*.pid` would be root-only).

Security: the server executes arbitrary shell commands from `commands.conf` as the service user — trusted LAN only, never a public interface.

6.1.1 Configuration page — filter installs and audio hardware roles

`/configuration` lets an operator on the trusted LAN install or remove room-correction designs and pin physical audio cards by USB identity, with no git checkout or shell access to the box. `omdrc-ctrl/src/configuration.py` drives two independent workflows, both through the same privileged helper, `scripts/omdrc-config-helper.py`, installed with the panel:

```
<AUDIO_USER> ALL=(root) NOPASSWD: /usr/local/libexec/omdrc/omdrc-config-helper *
```

- **Filter installs** — upload a REW `.txts/.mdat` pair and publish it as a live, verified bundle. This is the normal live-install path described in full in section ; the page invokes the same audit/build engine the command-line procedure does, streaming progress back over Server-Sent Events.
- **Audio hardware roles** — pick the DAC and (where a bridge exists) the capture interface from the cards actually attached, by USB identity (`vid:pid[:serial]`), rather than editing `rc.conf`/`sysrc` or a systemd unit by hand. `apply_audio()` refuses a DAC that is not attached or cannot play, and refuses either role when two identical cards have no serial number to

disambiguate — unplug one before applying. On FreeBSD, Apply updates the two `omdrc_audio_*` role keys, reconciles the `omdrc_audio` rc.d service, and verifies the resulting `/dev/dspX` nodes exist. On Linux it writes `<prefix>/etc/open-media-drc/audio-roles.conf` (`OMDRC_AUDIO_DAC` / `OMDRC_AUDIO_CAPTURE`, the USB identities, surviving a reboot) and resolves the saved identity to the current ALSA card number into `/run/omdrc/audio.roles` on every hotplug restore, so nothing in the CD bridge or the panel's chain diagram has to hard-code a card number that USB attach order can change. A configured capture card that is not plugged in this boot is dropped with a notice rather than failing the whole reconcile.

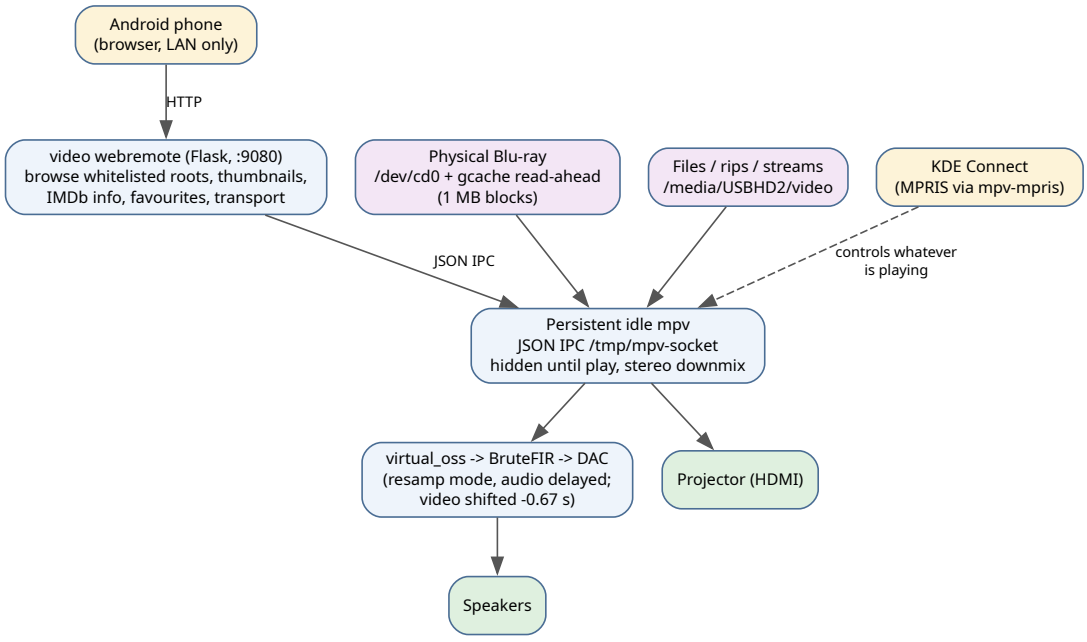
The helper validates USB identities, selectors, canonical bundle IDs, hashes, config derivation, and destination roots before touching anything root-owned; `configuration.py` itself runs as the unprivileged web user throughout.

A finished job keeps only its log. Both large working trees — the browser's upload and the staged site the helper publishes from — are discarded when the job ends, and payloads orphaned by an earlier run of the panel are swept at startup. Before that, every install leaked its payload permanently: five of them had accumulated 901 MB under `~/.local/state/omdrc/configuration` and filled the root filesystem.

6.1.2 Bit-perfect check page

`/bitperfect` runs the USB wire tap from the browser through any of five playback paths — including the real **upmpdcli** and **qobuzconnect2mpd** routes — and shows the reference bytes beside the bytes the DAC actually received, as a whole-stream colour map and a synchronised hex view. It shares this page's operation lock, CSRF token and Origin rule. Fully described in The `/bitperfect` page and its implementation.

6.2 Video: mpv playback + phone web remote



Video playback and control paths.

6.2.1 Playback launchers

- **`play-bluray.sh`** — physical Blu-ray discs. Kodi cannot read a physical BD on FreeBSD (raw `/dev/cd0` wants 2048-byte-aligned reads; the kernel cannot mount UDF 2.50), so mpv + libbluray read the raw device. Because the USB drive only sustains full speed in ~1 MB chunks and raw `cd0` has no kernel read-ahead, the script fronts the drive with a **GEOM cache** (`gcache create -b 1048576 -s 268435456 bd cd0` -> `/dev/cache/bd`), and probes `bd_list_titles` to play the *genuinely longest* title (mpv has no BD menu support; e/E cycle titles at runtime).
- **`play-media.sh`** — local files, playlists, and network/stream URLs (m3u8, yt-dlp sites); same DRC audio routing, no gcache.

Both source `lib/drc-audio.sh`, which ensures the chain is in **resamp mode** before playing — necessary because the direct DAC is bit-perfect (`bitperfect=1` on the DAC's unit): a 48 kHz movie on a higher-clocked DAC would play ~2x fast. With DRC up, mpv plays to `oss//dev/dsp.play` and delays the **video** by `DRC_VIDEO_DELAY` (default **0.67 s**) to match the audio-path latency; subtitles ride with the picture automatically.

The 0.67 s is derived, not guessed (full derivation in `video/AV-SYNC-DELAY.md`): the FIR filter's impulse peak sits at sample 96000 of 524288 taps at 192 kHz = **0.500 s group delay** (the coefficient list *is* the impulse response; the peak is when a transient emerges), plus one BruteFIR partition (32768 samples at 192 kHz = **0.171 s**), plus a little `virtual_oss` buffering.

DVDs are simpler: they mount fine (UDF 1.x) and are low-bitrate, so `mpv dvd:// --dvd-device=/dev/cd0`; commercial discs need `libdvdcss`.

Remote control: the `mpv-mpris` package auto-loads into every mpv, so KDE Connect's Android *Media control* (and `playerctl`, Plasma widgets) drive whatever is playing — play/pause/seek/volume/metadata.

6.2.2 The web remote (`video/webremote/`)

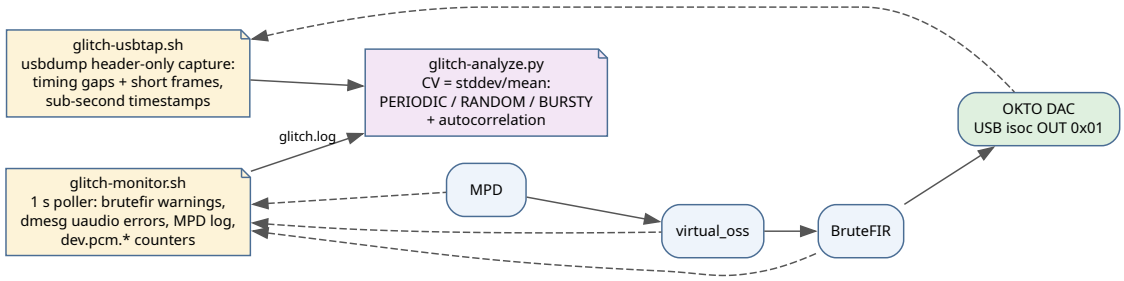
KDE Connect controls what is *already playing*; the web remote is what *starts* a title. A separate Flask app (port 9080, LAN-only, rc.d service `omdrcvideo`) serving a phone UI to:

- **Browse** the whitelisted media roots (realpath containment — no `..` or symlink escape), with entries classified server-side: folder, Blu-ray rip (`BDMV/index.bdmv`), DVD rip (`VIDEO_TS`), or playable file. Grid (poster thumbnails) or compact list.
- **Thumbnails** via ffmpeg frame grabs, disk-cached by path+mtime, with a background prewarm thread and bounded concurrency.
- **IMDb info** — title deduced from the name and, with an OMDb API key, verified and enriched (year, director, cast, plot, rating).
- **Play** on a **persistent idle mpv** over its JSON IPC socket (`/tmp/mpv-socket`) — hidden until something plays, DRC audio configured once at startup, `audio-channels=stereo` so 5.1/7.1 sources downmix. Blu-ray rips get the longest-title probe; a **Play Blu-ray disc** button reuses the gcache lifecycle for physical discs, loading into the same mpv.
- **Transport** — seek, +-10/30 s, play/pause, mute, stop, audio and subtitle track menus; **favourites** pinned to the main page.

The idle mpv is autostarted by the KDE/Plasma session, from `~/.config/autostart/mpv-idle.desktop` (linked to the installed entry by `make user-install`); a `git pull` + `service omdrcvideo restart` is the whole update path.

6.3 Glitch detection

Implemented for FreeBSD (on Linux the same scripts run but the FreeBSD-specific sources are simply quiet). One global switch — `glitch-debug.sh on|off|status|analyze|usbtap|tail|clear` — also exposed as the Debug card in omdrc-ctrl.



The glitch-detection layers and where each taps the chain.

- **`glitch-monitor.sh`** (always-on, lightweight): polls every second and logs new anomalies from four sources — BruteFIR warnings (missed real-time deadlines), kernel `uaudio`/USB errors, the MPD log, and any increasing `dev.pcm.*` under/over/err/xrun counter — into a unified `glitch.log`.
- **`glitch-usbtap.sh`** (definitive, heavier, CLI-only): taps the OKTO's isochronous OUT endpoint 0x01 with `usbdump`, downstream of every software stage. Header-only analysis scales to multi-minute captures; it flags **timing gaps** (> 2.5x the nominal ~4 ms interval) and **short frames** (SLEN < 0.5x nominal), while the constant +-few-samples feedback wobble of asynchronous USB is counted separately and never flagged. Blind spot: a full-length block of zeros (silence insertion) needs payload inspection — use `verify-bitperfect.sh` for that.
- **`glitch-analyze.py`**: classifies inter-event intervals per stage by the coefficient of variation — CV ~ 0 **PERIODIC** (a buffer/clock cycle), CV ~ 1 **RANDOM/Poisson** (CPU/scheduling), CV > 1.5 **BURSTY** (something waking up) — plus autocorrelation and correlation against DRC rate switches in `drc.log`.

6.4 Bit-perfect verification

`scripts/verify-bitperfect.sh` *proves* the DAC receives bytes unchanged. Two levels:

1. **Structural** (kernel-certified): with `hw.snd.verbose=2`, `/dev/sndstat` must show the play channel BITPERFECT with the feeder graph exactly `{userland} -> feeder_root -> {hardware}` — any `feeder_rate`/`feeder_volume`/format node means the kernel is altering bytes. Preconditions: `bitperfect=1` and `play.vchans=0` on the DAC's unit — which is what `omdrc_audio_dac_sysctls` asserts on every attach.
2. **Empirical wire tap** (gold standard): play a deterministic test signal (near-silent ~-90 dBFS per-sample counter in the low 16 bits, distinct L/R — maximally sensitive to truncation, dither, volume, resampling, channel swap) while capturing the USB isochronous OUT endpoint 0x01 with `usbdump`; decode, align, and byte-compare. The embedded OSS writer aborts loudly if the

kernel coerces format/channels/rate. Verified on this host at 44.1/48/88.2 kHz: hundreds of kB contiguous identical bytes, and on the live MPD both direct (**BIT-PERFECT**) and through `virtual_oss` (**VALUE-EXACT**, 0 slips).

The subtle part is clock domains: a producer must be **flow-controlled by the sink's clock** (blocked writes). MPD is; a free-running test writer is not, and drifts. `virtual_oss` itself is bit-transparent with a flow-controlled producer. The one caveat: BruteFIR bridges the loopback's software clock to the DAC crystal without resampling, so an inaudible one-sample slip occurs every several minutes on the DRC path — values are never altered. The DRC path is *intentionally* not byte-equal (that is the correction); to test its plumbing, use a unit-impulse filter with attenuation 0.

The test asset (`tests/`) is a 44.1 kHz S32 WAV whose PCM payload is byte-identical to the reference raw — MPD cannot play headerless raw.

6.5 The `/bitperfect` page

`verify-bitperfect.sh` above proves one segment: *host to USB wire*, with the tool itself as the player. That is the right control experiment, and it is not how the box plays music. Music arrives through **upmpdcli** or **qobuzconnect2mpd**, both of which drive **MPD**, and every one of those layers can break bit-perfection in a way the direct test would never see — MPD resampling, a decoder promoting samples differently, or a renderer quietly setting MPD's volume or replaygain.

The control panel's **Bit-perfect check** page (`http://<box>:9090/bitperfect`) runs the same tap and the same verdict engine, varies *who plays*, and — the part no command line offers — shows the compared bytes.

6.5.1 The five paths, and which question each answers

Source	Who starts playback	What it proves
`aplay`	the panel, via the per-OS tap script, delegated to unchanged	host to USB, no renderer. The control.
`mpd`	the panel: MPD plays a local file staged into `music_directory`	MPD to DAC: the segment both renderers share
`mpd-http`	the panel: MPD fetches an HTTP URL it serves	MPD's curl input plugin and streaming decoder — structurally the path a Qobuz stream takes, but with a *known* file, so the verdict is real byte equality
`upnp`	the panel: upmpdcli is found by SSDP and told to play it itself	the whole **upmpdcli** to MPD to DAC path (needs upmpdcli to be discoverable — see §)
`live`	**you, in the Qobuz app** — the panel only arms the tap and waits	the whole **qobuzconnect2mpd** path, against the renderer's own buffer

`live` is the one row where the panel is not the playback initiator, and that is the point rather than a limitation. Every other source hands material to the chain; `live` deliberately touches nothing, arms the USB tap, and prompts you to start a track in Qobuz — so the bytes it captures were produced by

the real service path, with nothing synthesised on their behalf. Read “the page plays nothing” as *the page starts no playback of its own*, not as *no audio flows*: the tap has to see a live stream or there is no capture to compare.

Running `mpd` and `upnp` on the same material is what *localises* a fault: if `mpd` is bit-perfect and `upnp` is not, the renderer is the cause. Both renderer-driven modes snapshot MPD’s volume and replaygain before and after and report any change, because a renderer that silently enables replaygain destroys bit-perfection without altering a byte of the file it was handed.

6.5.2 The live Qobuz reference: the renderer’s own buffer

`live` needs a reference for a stream nobody has a copy of. It does not ask for one. **The renderer already wrote the exact bytes it fed MPD to local storage**, so that file *is* the reference — genuine byte equality against the real service, not a comparison against a “same” track that might be a different master.

On this box `qobuzconnect2mpd` stages a **complete FLAC per track**:

```
/tmp/qobuzconnect2mpd-<uid>/cache/track_<id>_<fmt>_<pid>_<n>.flac
```

Resolution happens at run time and never trusts a hard-coded path:

1. `mpc current` — if MPD reports a local file, that is the reference;
2. otherwise the open file descriptors of the MPD and renderer processes (`/proc/PID/fd` on Linux, `fstat(1)` on FreeBSD) — renderer-agnostic;
3. otherwise the known buffer locations, **filtered to files that changed during the tap window**. That filter is not a nicety: the cache keeps completed tracks for hours, and comparing this capture against a stale one would report a fault that never happened.

A directory of numbered *segments* is concatenated in **numeric** order (lexical order would put `part10` before `part2` and manufacture corruption); a directory of whole tracks is never concatenated. If the buffer turns out to be a **lossy** container the run is reported as *decode-path transparent*, not bit-perfect: the DAC legitimately receives the decoder’s output rather than the file’s bytes.

6.5.3 Checking any track, not only the generated asset

The page loads a **WAV or FLAC** (anything `ffmpeg` decodes). The file is decoded once to build the reference while the **original** is played, since the decoder is part of what is under test. Rate and depth come from the file, so a 96/24 FLAC tests 96/24 with nothing to configure, and the existing `prep` promotion widens it to the S32 wire container exactly as before.

One hazard belongs to real music alone. Alignment anchors on a 4 KiB window of the reference; the generated counter can never repeat one, but music can — digital silence, a looped intro, a repeated bar. Aligning on the wrong occurrence would be a *silent* error, so `find_probe_offset` takes `unique_in=` and skips any window occurring more than once, and the loader reports the anchor it chose *before* a run rather than after it.

6.5.4 Using the page

1. **Chain readiness** names the DAC, who holds it, whether BruteFIR is running, whether `sudo -n` will start the tap, and MPD's volume, replaygain and enabled outputs. Blocking conditions are listed explicitly; `drc.sh off` clears the usual one.
2. **Test material** — either generate the per-rate counter asset (30 s for rates up to 96 kHz, 10 s at 192 kHz) or load a file. Each row shows size and sha256 so a run is identifiable from its report alone.
3. **Run a check** — pick the path, the material and (for `live`) the tap window, then start. Progress streams as a phase strip (`prep`, `tap`, `play`, `drain`, `align`, `verdict`) with live tap counters: URBs, payload bytes, kernel drops. The runner also reports finer stages that are not chips of their own — decoding the capture, resolving what the renderer streamed, restoring MPD — and those are shown as a caption beside the strip: the strip only ever advances, so a stage it does not recognise holds position rather than clearing it.
4. **Result** — the verdict, then every stage named and hashed (input file, reference bytes, untrimmed wire, tapped payload), then the byte view.

The identical run from a terminal:

```
scripts/bitperfect_runner.py --source mpd  --input track.flac --out bp-results/run
scripts/bitperfect_runner.py --source upnp --input tests/bitperfect-test-44100-s32-stereo-
        30s.wav \
                             --out bp-results/upnp
scripts/bitperfect_runner.py --source live --duration 60 --out bp-results/qobuz
scripts/bitperfect_material.py load album.flac --out-dir /tmp/mat
scripts/bitperfect-lib.py window PREFIX.ref.raw PREFIX.wav 80000 8 2
scripts/bitperfect-lib.py scan   PREFIX.ref.raw PREFIX.wav 200 2
scripts/bitperfect-lib.py leadin PREFIX.wire.raw 5648 2 0 32
```

6.5.5 Seeing the bytes

A verdict says *whether*. The byte view says *where* and *what*, at two zoom levels served by two subcommands added to `bitperfect-lib.py`:

- **The colour map** paints the whole stream, one cell per slice: green where the wire carried exactly the reference bytes, red where it did not, grey where the capture did not reach. Every byte of the overlap is really compared — only the level row underneath is estimated. It makes the *shape* of a fault legible: an isolated red cell is a bit flip, a red tail is an underrun, dense speckle is a converting feeder in the path. Clicking jumps the hex view there.
- **The side-by-side hex view** shows the reference beside the tapped wire, one row per frame — byte offset, hex, and the decoded per-channel integers for both — with the differing byte positions highlighted in *both* columns at once. A single slider drives the offset across the whole file with the two views locked, so dragging it is a continuous consistency check; `first mismatch` and `random spot check` jump to the interesting places.

Both read only the artifacts `finalize` already wrote, so they cannot disagree with the verdict — a property `tests/test_bitperfect_window.py` pins down on captures whose faults are known to the byte.

A third subcommand, `leadin`, opens the box at its left edge. Alignment discards everything the tap saw before the first reference byte — attach noise, then whatever the audio stack emitted ahead of the music — and the verdict says nothing about those bytes. **Before the stream**, a collapsible panel under the hex view, reads them back out of `PREFIX.wire.raw`, which every path already writes. There is no reference column: before the stream starts there is nothing to compare against. Rows ahead of the anchor are marked *trimmed*, so a view spanning the boundary shows exactly where the compared stream began, and the summary separates the two things living in that head — a run of zeros immediately before the anchor is the ring's priming silence, while a non-zero byte anywhere in it came from somewhere else and is worth looking at. Unlike the other two, it does not need the aligned pair: the untrimmed wire is precisely what is still worth reading after an `ALIGNMENT FAILED` verdict, when no aligned pair exists.

6.6 Bit-perfect verification: implementation

6.6.1 The pipeline

```
material (WAV/FLAC)             the DAC
   |                               ^
   | prep + lossless promotion     | isochronous OUT, endpoint 0x01
   v                               |
ref.raw  (S32_LE wire container)   +--- usbmon (Linux) / usbdump (FreeBSD)
   |                                           |
   |            play (one of five sources)     v
   +------------------------------------>  cap.raw
                                               |
                       finalize: align, compare, classify
                                               v
                    PREFIX.{txt,json,wav,ref.raw,wire.raw}
```

The tap sees the stream at the **last host-controlled point** — the URBs handed to the USB host controller. Only the controller's DMA engine and the DAC's own receiver lie beyond it; neither can be tapped in software on any OS, and neither has any mechanism to alter a PCM payload.

`scripts/bitperfect_runner.py` orchestrates; it does **not** reimplement the verdict. `--source aplay` is delegated to `bitperfect-tap-{linux,freebsd}.sh` unchanged, so the control experiment stays byte-identical to every result recorded in `doc/BIT-PERFECT-VERIFICATION.md`.

6.6.2 Artifacts

File	Content	Reproducible
`PREFIX.txt`	the human report: every stage named and hashed, then the verdict	yes (this is the committed cross-OS key)
`PREFIX.json`	the same as structured data, plus `first_mismatch`, `start`, `probe_offset`, `matched`, `slips`, the MPD before/after state and the source	yes
`PREFIX.wav`	the aligned capture, trimmed to reference length	yes

File	Content	Reproducible
`PREFIX.ref.raw`	the promoted reference — kept so the byte view has both sides	yes
`PREFIX.wire.raw`	the untrimmed capture: priming zeros, pad, trailing packets	**no** — provenance only, never a comparison key

6.6.3 Alignment: the anchor, not the silence

The wire stream never starts at the first source byte: the capture begins before playback, and the audio stack emits priming zeros before the first written sample (measured: exactly 16 ms at any rate — a fixed-duration buffer prime, hence reproducible). `finalize` therefore takes a 4 KiB *anchor* from the reference at a known offset `po`, finds it in the capture at `pos`, and derives `start = pos - po`.

It anchors on content it already knows rather than "skipping the leading zeros" because **zeros can be data**: a track may legitimately open with silence, and on the wire those bytes are indistinguishable from the priming zeros in front of them. A "skip the zeros" rule would eat a quiet intro.

6.6.4 The silence pad

What is *played* is not what is *compared*: playback gets a few seconds of digital silence appended, the reference does not, and `finalize` cuts by arithmetic (`cap[start : start + len(ref)]`) rather than by detecting silence.

Two independent reasons, both measured:

- `usbdump` buffers its pcap and loses whatever is unflushed when it is terminated (hence also `SIGINT`, not `SIGTERM`);
- **MPD does not drain its output buffer on close.** A 10 s WAV reached the wire 129744 bytes (0.74 s) short — identically across runs, and regardless of how long the tap kept recording afterwards, which is what proves it is the player discarding its tail rather than the tap stopping early. The direct OSS writer has no such gap because it `SNDCTL_DSP_SYNC`s first.

The pad cannot mask a defect: the comparison window is still exactly `len(ref)` bytes, so every reference byte is still checked, and a bit flipped mid-stream is still reported at its exact offset. What the pad removes is only the ability of a late capture cut to masquerade as a playback fault.

6.6.5 Renderer arbitration

`qobuzconnect2mpd` and `upmpdcli` are mutually exclusive front-ends, and an active one does not sit still: it watches MPD and re-queues its own track. That is not hypothetical — it truncated a 10 s run to 2.2 s, with MPD's log showing the test WAV replaced by a Qobuz stream mid-playback. So:

Source	Requirement
`mpd`, `mpd-http`	nothing else may drive MPD — the renderer is stopped and restarted around the run
`upnp`	`upmpdcli` must be the one running
`live`	`qobuzconnect2mpd` must be running and playing; nothing is touched

`omdrc-renderer stop` deliberately leaves the *remembered* choice alone, so the restart afterwards is exact.

6.6.6 Two MPD facts the runner has to work around

- **MPD refuses `file://` from a TCP client** ("Access to local files via TCP is not allowed") and this MPD has no Unix socket. The only way to exercise the local-file input plugin is to place the material inside `music_directory` and add it by relative path. When that directory is not writable the run falls back to an HTTP URL — and *says so*, because the path under test changes from MPD's file input plugin to its curl one.
- **Stored playlists are disabled**, so an `mpc save` backup silently does nothing and the restore then fails with "No such playlist". The queue's URIs are held in memory instead, which works on any MPD.

6.6.7 Privilege, and what the page refuses

The panel runs unprivileged. The tap needs root — `usbmon` on Linux, `usbdump` on FreeBSD — and escalates with `sudo -n`, probed up front with `true` so a refusal is reported before a capture is wasted rather than discovered halfway through it.

Runs are **refused while BruteFIR is convolving**. The DRC path applies the FIR filter, so its output is *supposed* to differ; a verdict there would look like a failure and mean nothing. `--allow-drc` overrides it for an operator who knows what they are asking for.

The page shares the configuration page's operation lock (one box, one DAC: a filter publication and a tap run must never overlap), its CSRF token and its Origin rule. One route is necessarily token-free — the asset MPD's curl plugin and upmpdcli fetch by URL, which can present no header — and it resolves basenames inside the asset cache and nothing else.

6.6.8 Verdicts and exit codes

Exit	Meaning	Verdicts
0	judged, and the chain is transparent	`BIT-PERFECT`
1	judged, and something is wrong	`HEAD LOST`, `INCOMPLETE`, `VALUE CORRUPTION`, `TIMING SLIP(S)`, `UNDERRUN TAIL`
2	**could not judge** — capture unusable	`NO CAPTURE`, `ALIGNMENT FAILED`

Exit 2 does not by itself indict the capture setup. Aligning needs 4096 *consecutive intact* bytes, so a defect altering *every* sample (a volume feeder, say) leaves no such run anywhere and the search fails before any comparison. When exit 2 appears, open `PREFIX.wire.raw`: junk or zeros point at the tap, plausible-looking audio points at a converting feeder.

6.6.9 What running it exposed

Two pre-existing defects surfaced the first time the page was driven against real hardware, both worth knowing independently of the page:

- `bitperfect-tap-freebsd.sh` tapped `dev.uaudio.0`. On a box with a capture interface that is **not** the DAC: here `uaudio0` is the ESI U24 XL and the OKTO DAC8 is `uaudio1`, so the script recorded the wrong USB device and reported `NO CAPTURE` against a perfectly healthy chain. It now resolves the uaudio unit from the play device (`/dev/dsp.dac` to `pcm<N>.%parent`), as `glitch-usbtap.sh` always did. Same family as the `omdrc_sndlink` rename that once pointed the whole chain at the capture card.
- The `/configuration` page's job state root kept every filter install's upload *and* its staged site copy forever. Five installs had accumulated 901 MB in `~/.local/state/omdrc/configuration` and filled the root filesystem. Finished jobs now discard their payload and keep only `job.json`, and the manager sweeps what earlier runs orphaned at startup.

6.6.10 Measured results

On the OKTO DAC8 (FreeBSD 15.1-RELEASE-p2, `usbus0` devaddr 3):

Path	Material	Verdict
`aplay`	44100 / 32-bit	**BIT-PERFECT** — tap WAV file hash identical to the input
`mpd`	44100 / 32-bit	**BIT-PERFECT**
`mpd-http`	44100 / 32-bit	**BIT-PERFECT**
`mpd-http`	96000 / 24-bit FLAC	**BIT-PERFECT**, DAC clock followed (`feedback_rate` 96002)
`upnp`	44100 / 32-bit, driven through upmpdcli over OpenHome	**BIT-PERFECT**

The 96 kHz FLAC run carries the most information: it proves the FLAC decode is transparent *and* that the reference's 24-to-32 promotion matches MPD's own promotion exactly, which is the one place the two could have disagreed. The `upnp` run is the one that certifies the renderer itself: the whole upmpdcli-to-MPD-to-DAC path, with upmpdcli choosing what MPD plays.

`live` remains unverified against hardware — see Status and what is still owed.

6.6.11 Status and what is still owed

- **`live`** needs a human to start Qobuz playback, so it has not been run end to end. Its buffer resolver is unit-tested against the exact layout observed on this box, including the stale-buffer and never-concatenate-whole-tracks rules.
- `drc.sh cdin` re-execs without the design variant, so switching to CD input silently drops an active `@design`.

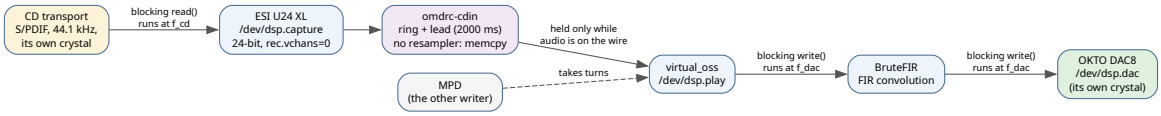
6.7 scripts/ — helper tools

Script	Purpose
`REW2raw.sh`	REW WAV -> brutefir raw FLOAT64_LE at a target rate, with the theoretically correct `Fs_source/Fs_target` coefficient scale (no peak normalisation)

Script	Purpose
`REW2raw-all-rates.sh`	Batch: `L.raw`/`R.raw`/`sox.txt` for every numeric rate directory under a filter root; prompts before overwriting unless `-y`
`headroom_calc.py`	Minimum `attenuation:` per config from worst-case FFT gain + safety margin
`new_filter_design.py`	Publishes a design from one directory of REW exports, and records it in the room's history
`remove_filter_design.py`	Removes one deployed design completely and records that in the room's history
`deploy_filter.py`	The offline audit/build engine underneath: rates, validation, analysis, manifest
`verify_filter_bundle.py`	Read-only re-verification of committed bundles (used by CMake and CI)
`console_ui.py`	Shared stages, colour, confirmation and failure formatting for design commands
`filter_workflow_next.py`	Shared operator handoff printed by the commands above
`rew_mdat_audit.py`	Optional archival audit of REW project traces against exports
`verify-bitperfect.sh`	The original bit-perfect proof tool; sources: built-in writer or `mpd:OUTPUT`; taps: `usb` or `loop:/dev/dsp.X`
`bitperfect-lib.py`	Shared engine for the tap scripts and the panel: WAV to S32 wire-container promotion, usbmon reader, usbdump decoder, alignment, verdict, report — plus `window`/`scan`, which read the two compared streams back for the byte view
`bitperfect-tap-linux.sh` / `-freebsd.sh`	Play a WAV to the DAC and record the exact bytes on the USB wire; same CLI and artifacts on both OSes, for cross-OS comparison
`bitperfect_runner.py`	Runs a tap through a chosen playback path (`aplay`, `mpd`, `mpd-http`, `upnp`, `live`); backs the `/bitperfect` page, emits @@PHASE/@@STAT/@@RESULT progress lines (§Implementation)
`bitperfect_material.py`	Decodes any WAV/FLAC into a run's reference, checks the alignment anchor is unambiguous, and resolves what a renderer is streaming for `--source live`
`bitperfect-compare.py`	Compares two tap artifacts from either OS (`.wav`, `.wire.raw`, or the tiny committable `.txt` report)
`systemd-user-install.sh`	Legacy: link + enable a `systemd --user` drc.service (Linux)

7 CD input: S/PDIF capture into the DRC chain

A CD transport is the second source the chain accepts, and the only one that is not a file: it arrives as a live S/PDIF signal, clocked by the disc, on a USB capture interface. On FreeBSD `omdrc-cdin` (`cdin/`) bridges it into the same `virtual_oss` entry point MPD uses; on Linux `alsaloop(1)` from `alsa-utils` does the equivalent job into `snd-aloop`. Either way a disc gets exactly the same room correction as everything else.



The CD path. Both ends of the bridge block on their own device, so the ring fill between them is the drift signal; there is no resampler anywhere in it.

It takes the seat `mpv` already takes for video (`video/lib/drc-audio.sh`): a non-MPD writer into the loopback that BruteFIR reads.

Most of this chapter — the FreeBSD `omdrc-cdin` daemon, the lead/drift arithmetic, the state machine, the transport simulator and the stats line — describes the daemon that had to be written because FreeBSD ships nothing that reconciles two free-running audio clocks. Linux does ship that reconciler, so section covers the Linux bridge on its own terms: what topology problem it has that FreeBSD does not. Both platforms now present the same user-visible rule: CD input is an exclusive source. The web panel's CD input card is unchanged either way — it parses the common log grammar, and the Linux supervisor emits the same grammar, so one card serves both operating systems.

7.1 Why a bridge is needed at all

Capture is slaved to the CD's crystal, the DAC runs on its own, and the two differ by a few ppm **forever**. Samples arrive at `f_cd` and must leave at `f_dac`. FreeBSD ships no `alsaloop` equivalent, which is what made this look blocked — but the missing piece was a tool, not a kernel facility: a blocking `read()` runs at the CD's clock, a blocking `write()` runs at the DAC's, so **the ring fill between them is the drift signal**. No OSS clock ioctl is needed to measure it.

There is no resampler. The data path is a `memcpy`, so what reaches BruteFIR is bit-identical to what the transport sent — and that is verifiable with `scripts/verify-bitperfect.sh`, which would be meaningless if a resampler sat in the path. Drift is absorbed by the *lead* instead.

7.1.1 The lead is the only number that matters

The lead — how much audio is buffered ahead of the output — is simultaneously three things:

- the **drift margin**: how long before the buffer runs out;
- the **startup delay**: you cannot pre-fill a lead you have not waited for;
- the **transport lag**: every Play/Stop/Skip is heard this much later.

They cannot be tuned separately. The upper bound is arithmetic: `time-to-splice = lead / drift`, so at a pessimistic **50 ppm** a **2000 ms** lead covers about **11 hours** of continuous gapless audio. A disc is at most 80 minutes, therefore **drift cannot cause a discontinuity inside a disc**.

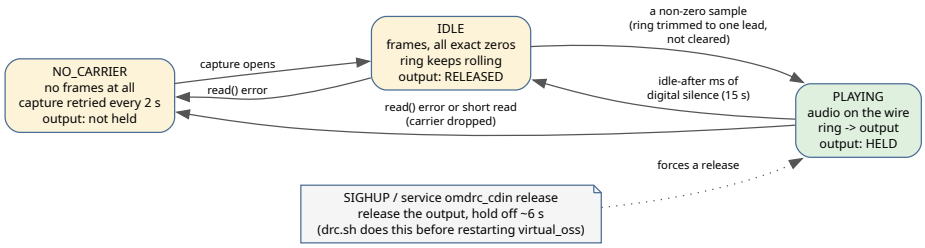
The *lower* bound is not set by drift at all. It is set by transport seeks and USB stalls: some players briefly **drop carrier** across a pregap or index boundary, which is a sub-second input stall that the lead has to absorb. That, not drift, is why the default is 2000 ms rather than the ~50 ms drift alone would need. Below ~250 ms the daemon warns that nothing is left to absorb a seek.

Calibrate it on the real transport with a full disc:

```
omdrc-cdin --in /dev/dsp.capture --out /dev/dsp.play --lead 2000 -d -s 10 \
    -l /tmp/cdin.log
```

`starves` must stay 0 — each one is an audible dropout. After about five minutes the `drift` field reports the measured ppm and the projected headroom, which replaces the 50 ppm assumption with the actual hardware. Step `--lead` down (1500, 1000, 750…) until the first value that produces **any** starve; that is below the floor the transport imposes, so go back up one step and keep a margin. Record the value: it becomes `omdrc_cdin_lead`.

7.2 The three states, and the two very different tenancies



The daemon's state machine. The output device is held only in PLAYING; the capture device is held for the whole session.

The daemon can remain up across CD player power and carrier changes, and that is safe at the OSS-device level because of what it does with the *output* device:

State	On the wire	The daemon	`/dev/dsp.play`
`NO_CARRIER`	no frames at all	retries the capture device	not held
`IDLE`	frames, all exact zeros	counts the silence	**released**
`PLAYING`	audio	ring -> output	held

The **capture** device is held for the life of a session: it is the interface's own node, nobody else wants it, and it is the only thing that can tell us whether a carrier exists. The **output** device is `virtual_oss`'s client node, and it is taken only for the duration of actual music.

That asymmetry is not politeness. `drc.sh` restarts `virtual_oss` on every rate change, and an open cuse client handle at that moment is what wedges the teardown *permanently*: `cuse_server_free()` spins uninterruptibly until every client handle is gone, SIGKILL does not touch it, and only a reboot recovers the machine (section). A daemon holding `/dev/dsp.play` around the clock would put a fresh instance of that hazard under every single rate change. The deployed source-selection policy is stricter than this internal tenancy: while the bridge service is selected, MPD's audible output is disabled even during silence. `virtual_oss` is a mixer, so relying on the device open alone would allow MPD and the disc to be audible together.

For the remaining CUSE exposure — a rate change while the bridge is running — `drc.sh` stops the service and treats process exit as the release acknowledgement:

```
service omdrc_cdin onestop
# wait until pgrep -x omdrc-cdin no longer finds the process
```

The old path sent `SIGHUP` and inferred release from new logfile lines. That failed when the log was rotated or unlinked. A successful transition restarts the bridge after rebuilding `virtual_oss`, including an instance originally started with `onestart` while its rcvar is disabled. If the bridge does not exit, teardown is refused; `drc.sh` attempts to restore MPD's direct output so the failed transition does not leave the machine needlessly silent.

Choosing `--idle-after` (default 15000 ms) has one failure mode at each end and a wide safe band between them: too short and Red Book's 2 s inter-track pause releases the device mid-disc, so every track change costs a lead to resume; too long and a stopped player keeps holding the chain. `--idle-after 0` disables the gate entirely, which is occasionally useful when measuring. Note this is silence *on the wire*: a player that drops carrier instead of sending zeros never reaches the gate at all — the read fails and it lands in `NO_CARRIER`, which is the state that reopens the device.

Resuming does not lose the first note. The ring keeps rolling through the silence, so an episode begins by *trimming* it to one lead rather than clearing it and waiting for a fresh pre-fill. The music still emerges one lead later, but the period that carried the first sample is still in the buffer.

7.3 What the transport does to the stream

A CD player's S/PDIF output is **always 44.1 kHz**. Transport actions change *what* is sent, never *how fast* — so there is no re-lock, ever:

Action	On the wire	Daemon behaviour
Pause	carrier alive, digital silence (most players)	plays the silence through; lead unchanged
Skip track	brief mute (0.1–1 s), then audio	a short silence, heard one lead later
Fast fwd / rewind	chopped scan snippets or mute, rate unchanged	ordinary audio or ordinary silence
Stop / tray / power	carrier drops	`read()` stalls or errors, session ends, device reopened

Only the carrier drop matters, and only because the wall clock keeps running while no frames arrive: the lead drains by exactly the dropout's length and never recovers.

7.3.1 Every row of that table is testable without a CD player

Point `--in` at a **directory** and its `*.wav` files become the tracks of a disc, played in name order; a single file is a one-track disc. Between tracks the rig emits `--gap` milliseconds of exact digital silence (default 2000, Red Book's inter-track pause) — which is what the silence gate looks for, so a `--gap` longer than `--idle-after` exercises the release/re-acquire cycle with no hardware at all. `--transport` then scripts the buttons as `AT:EVENT` pairs, `AT` being seconds into the stream:

```
omdrc-cdin -i DISC -o /dev/dsp.dac -d -s 5 \
    --transport "20:skip,35:pause=4,55:dropout=800,70:seek=+30,105:stop"
```

`dropout=N` is the important one — it drops the carrier for N milliseconds — and `--in-ppm` is its counterpart for the *clock*: it offsets the simulated source's pace, which makes the design's central claim testable in seconds instead of the full day real hardware would take. Both failure modes behave as the arithmetic predicts and are worth recognising in the stats:

- **lead exhausted** — `starves` increments, then `in` and `out` converge to the *same* wrong rate. That equality is the backpressure signature: with no buffer left, the DAC is paced by the source instead of by its own clock;
- **ring saturated** — `lead` pins at the ring capacity and `drops` climbs at the drift rate; audio is being discarded, one discontinuity per drop.

The rig emulates the CD player's clock, not the disk's seek time, so the disc is prefetched on its own thread (4 s deep) and the paced read is served from RAM — with the read inline, one 3 s stall on an external USB drive drained a 2 s lead and starved playback, which is a property of the rig and not of the design under test. If the medium genuinely cannot sustain realtime the daemon says so, and a schedule that slips more than 100 ms is shifted forward rather than firing every overdue deadline at once (hardware cannot replay frames it missed, and a burst would permanently inflate the lead). Both are counted and surface as `rig stalls N slips N`, shown only when non-zero.

7.4 Reading the stats line

```
[stats] lead 1635 ms (min 1625, max 1649)  drift +38.7 ppm (+/-387.0),
        ring fills in 46 h  in 44100.206 Hz  out 44088.817 Hz
        frames 4054016/3980288  drops 0 B  starves 0  silence 0%  up 90 s
```

- **`lead` is the ring only.** It settles *below* `--lead`, because the pre-fill hands the first few hundred ms straight to the output device's own buffer. End-to-end latency is this figure plus that buffer, plus `virtual_oss`'s 200 ms and BruteFIR's filter group delay; the startup delay actually waited is `--lead`.
- **`drift` is measured from the change in `lead`**, not from the frame counters: those carry each device's constant buffer offset, which at ppm scale would swamp the figure and which cancels in a difference. The `+/-` is the period quantisation over elapsed time — while it exceeds the estimate, the estimate means nothing. It needs minutes and tightens for hours.
- **`in` / `out` are measured from the instant playback began**, and the window restarts at every discontinuity, so a dropout does not leave the cumulative average reading low for the rest of the session.
- **`starves`** counts events, not periods: one continuous starvation is 1.
- **`drift ref dropped (lead jumped)`** means the estimate was thrown away and restarted because the lead moved for a reason that is not the clocks. A 3.3 s jump inside a 60 s window once read as `+54361 ppm`, which is a stall wearing a drift figure's clothes.

7.5 Running it as a service

```
# /etc/rc.conf
omdrc_cdin_enable="YES"
omdrc_audio_capture="ESI U24XL"   # names the ESI; that is what creates
                                  # /dev/dsp.capture, which cdin then uses
```

rc.conf variable	Default	Meaning
`omdrc_cdin_in`	`/dev/dsp.capture`	capture device (or a WAV file/directory, for the rig); the link comes from `omdrc_audio_capture`
`omdrc_cdin_out`	`/dev/dsp.play`	playback device; `/dev/dsp.dac` writes the DAC directly, bypassing BruteFIR
`omdrc_cdin_bits`	`24`	source width — the U24 XL's capture endpoint is 24-bit and nothing else (see below)
`omdrc_cdin_lead`	`2000`	lead in ms; drift margin, startup delay and transport lag at once
`omdrc_cdin_idle_after`	`15000`	digital silence before the output device is released; `0` disables the gate
`omdrc_cdin_logfile`	`/tmp/omdrc-cdin.log`	**must match** `log_file` in `commands.conf`'s `[cdin]` — this file *is* the web card
`omdrc_cdin_stats`	`10`	seconds between `[stats]` lines
`omdrc_cdin_user`	`AUDIO_USER`	run user; the same one that owns BruteFIR and MPD, because they take turns on the same devices
`omdrc_cdin_flags`		anything else: `--out-bits`, `--period`, `--retry`, `-v`

The rc script is installed by the CMake superproject (`cdin/CMakeLists.txt`, a subproject of the top-level build). It uses `daemon(8)`, drops privileges via the standard `rc.subr` `${name}_user`, and adds one non-standard verb, `release` (the diagnostic `SIGHUP` above). A `--` separates the daemon supervisor's arguments from the bridge's arguments; the public `omdrc_cdin_flags` value is moved out of rc.subr's reserved `${name}_flags` namespace before startup. Started unprivileged with `service omdrc_cdin onestart` it clears `${name}_user` — so `rc.subr` does not try to `su` to it — and uses a pidfile under `/tmp`.

The panel owns this service even when `omdrc_cdin_enable="NO"`. Selecting the CD source explicitly may start it with `onestart`; an incidental rate change never starts a bridge that was already stopped. When a running bridge must follow a rebuilt virtual output, `drc.sh` uses bounded `onestop`, waits for the process to disappear, then uses bounded `onestart`. This preserves a panel-started instance: `onerestart` would stop it and then let the disabled rcvar reject the start half. The timeout runs in foreground mode so its process-group cleanup cannot kill daemon(8)'s successfully detached supervisor.

Log lines are a **contract**, not just prose: the web panel parses them, so `state <name>: <why>` and `<device> <path>: <available|unavailable|acquired|released>` keep their shape. Availability and holding are separate axes: `unavailable` means nothing can play and is the red light, while `acquired`/`released` is the ordinary rhythm of a daemon doing its job and is never a fault.

7.6 The web panel card

`omdrc-ctrl` (section) shows a **CD input** card driven entirely by that log, under the Renderer card. It is configured by the `[cdin]` section of `commands.conf`, which configures nothing about the daemon itself — only where to read it from, plus whether the buttons exist:

```
[cdin]
enabled  = yes
log_file = /tmp/omdrc-cdin.log     # must match omdrc_cdin_logfile
process  = omdrc-cdin              # pgrep -x: tells "stopped" from "broken"
service  = omdrc_cdin              # what Start/Stop runs
control  = yes                     # offer the Start/Stop button at all
refresh  = 5
max_events = 20
```

The card is the size of its news, which for a bridge with no disc in the player is one line:

Daemon state	The card
`state playing`	full size, opened by itself — the only state with anything to watch
idle / no carrier	the status line and an expand chevron; still polling, so a disc reopens it
stopped	the status line and a **Start** button

Expanding or collapsing by hand sticks until the daemon's own state changes. Expanded, the card adds both device paths and their tenancy, the `[stats]` line as chips (`buffer 1962 ms (min 1955)`, `underruns 0`, `dropped 0 B`, `drift +1.2 ppm`, `fills in 46 h`, `silence`, `up`), a sentence for anything that has already cost something audible, the raw stats line, and a scrolling event list. Four rules decide what goes where:

- **the LED follows device availability only** — red when an end cannot be opened, because that is the question "can a disc play right now?". A released output device is the daemon working correctly and never colours anything;
- **failures are kept, health is replaced.** Every error stays in the list, in red, in chronological order, even after the condition clears, and the newest one gets its own red line above the fold. The healthy status line is replaced rather than accumulated. A live status line can never say "the output was missing for ten minutes this morning", and that is exactly the thing worth saying;
- **`starves` is the number to surface.** The lead is a margin; an underrun is that margin having run out — a dropout that already happened and that nothing else in the chain will ever mention again;
- **a daemon that is not running is idle, not broken**, however alarming the tail of its log is. The log outlives the process, and nothing is unavailable when nothing is trying to open it. With no daemon and no log at all, the card hides itself.

Three buttons: refresh, **Log** (opens the whole `omdrc-cdin` log in the Logs card), and **Stop**/**Start**, which performs a source hand-off around the rc service. Start records whichever of `OKTO-DAC`, `DRC-native`, or `DRC-resamp` is enabled, then disables all three before starting CD input; a failure to gate MPD aborts the start. Stop waits for the bridge to release its output and then restores exactly the recorded MPD output. The `OMDRC Spectrum` FIFO is not an audible output and is deliberately left alone. A failed or timed-out start also restores MPD.

The exit status of `service ... onestart` is not trusted — it forks a `daemon(8)` and returns before the daemon can die on a missing device — so the process is polled with `pgrep -x` until it agrees. On FreeBSD the button needs a `sudoers` grant; without one, set `control = no` rather than leaving a button that can only fail:

```
omdrcctrl ALL=(root) NOPASSWD: /usr/sbin/service omdrc_cdin onestart, \
    /usr/sbin/service omdrc_cdin onestop
```

7.7 The Linux bridge: alsaloop instead of a daemon

As of this writing, everything in this section is built and reasoned but has not run on the Linux box yet — see “What has not been measured” below before trusting it on real hardware.

```
CD player --S/PDIF 44.1k--> ESI U24 XL --USB--> hw:<cap>,0
                                                  |  alsaloop
                                                  v
                                           hw:Loopback,0,0   (snd-aloop)
                                                  |
                                                  v
                                           hw:Loopback,0,1
                                                  |  BruteFIR
                                                  v
                                                hw:0,0      (Okto DAC8)
```

On FreeBSD there is exactly **one** pair of independent clocks: the CD’s crystal and the DAC’s; `virtual_oss` sits between them and is itself clocked by the DAC it feeds, so it contributes no third clock. `snd-aloop` has no clock of its own and by default invents one, an hrtimer — which makes **two** independent pairs on Linux (CD <-> aloop-timer, aloop-timer <-> DAC), the second present even in plain MPD playback with no CD input at all, and normally invisible only because both BruteFIR stanzas in `brutefir_defaults.linux.conf` set `ignore_xrun: true`.

`etc/modprobe.d/omdrc-snd-aloop.conf` removes that second pair by pointing the module’s `timer_source` at the DAC card:

```
options snd-aloop index=1 id=Loopback pcm_substreams=2 timer_source="hw:0,0,0"
```

The loopback then advances at the DAC’s rate, the topology becomes the same shape as the FreeBSD one, and CD-to-DAC drift is the only drift left — a prerequisite for the correction below, not an optimisation: correcting one drift pair while a second runs free would only move the problem. `omdrc-config-helper` rewrites the marked line to whichever DAC is selected on `/configuration` (section); the module loads at boot, so a change takes effect on the next boot.

Drift correction uses alsaloop's `playshift` mode, which steers `snd-aloop`'s `PCM Rate Shift 100000` control so the loopback consumes at exactly the rate the CD delivers — **there is no resampler in the data path**, the same bit-perfect property the FreeBSD design protects. Other `--sync` modes exist for diagnosis (`OMDRC_CDIN_SYNC`, or `--sync`): `samplerate` (continuous libsamplerate resample, **not** bit-perfect, for when the shift control misbehaves), `simple` (insert/drop samples), `none` (measure the raw drift). `captshift` is not useful here: it needs a rate-shift control on the *capture* card, which a USB interface does not have. The supervisor reads the shift control back every stats interval and reports it as the `drift` field in ppm — a direct measurement of the correction being applied, rather than an inference from buffer level.

The CD input is an exclusive source on both platforms, for different low-level reasons. FreeBSD must enforce the rule explicitly because `virtual_oss` mixes clients: the card gates MPD's audible outputs around the bridge service. Linux's `hw:Loopback,0,0` is a **single substream**, so alsaloop and MPD's `DRC-native`/`DRC-resamp` outputs cannot both hold it; whichever opens second gets EBUSY. Consequently:

- `drc.sh cdin` disables every MPD output, brings the chain up at 44.1 kHz, and starts `omdrc-cdin.service`. MPD has no output while the CD input is selected — that is the correct state, not a limitation worked around, and the direct `OKTO-DAC` output is no help either because BruteFIR holds the DAC.
- Any rate action (`drc.sh 44100`, `192000`, `resamp`, …) records `music` as the source, stops the bridge, **waits for the `alsaloop` process itself to be gone** (not just for `systemctl stop` to return — that happens before the kernel has closed the substream, and reopening too early is an EBUSY that reaches the user as "MPD will not play"), then enables the MPD output.
- The unit is **not** enabled at boot; `drc.sh restore`/`reconcile` bring back whichever source was saved.
- `drc.sh off`/`stop` return the box to MPD direct and leave the bridge down. FreeBSD moves its bridge to the DAC instead on `off`; Linux cannot, because the off path hands that same DAC to MPD's direct output and the DAC is single-open. Re-select the CD input with `drc.sh cdin`.

The one thing that does carry over: when the DRC chain is down, the bridge writes straight to the DAC instead of the loopback, exactly as `cdin/src/outsel.h` describes for FreeBSD. The output device is settled once at startup — alsaloop negotiates a format against it and cannot re-take the decision while running — so the service is restarted whenever the chain moves.

Selecting the capture interface uses the same `/configuration` Audio hardware section as the DAC, on both roles now. Applying writes `<prefix>/etc/open-media-drc/audio-roles.conf` (survives a reboot) and `/run/omdrc/audio.roles` (resolved ALSA card numbers, does not); the ESI's input selector — the card has two inputs, only one live, chosen by a mixer setting that does **not** survive a reboot (`cdin/ESI-U24XL.md`) — has a Linux equivalent in an `amixer` capture-source switch, e.g. `amixer -c <card> cset name='PCM Capture Source' 1`, whose exact control name is card- and kernel-dependent and belongs in an `ExecStartPre=` systemd drop-in on `omdrc-cdin.service` so it survives a reboot the way the FreeBSD `omdrc_audio_capture_recsrc` rc.conf var does.

Requirements: `alsa-utils` (`alsaloop`, `amixer`); `snd-aloop` with `timer_source` support (mainline since 4.x); a capture interface that takes its clock from the incoming S/PDIF carrier the way the ESI U24 XL does automatically (ESI KB00307EN).

What has not been measured: whether `timer_source="hw:0,0,0"` is accepted in that spelling and actually pins the loopback's `hw_ptr` to the DAC's rate, and whether `--sync=playshift` finds the shift control on the real hardware (the supervisor logs a warning naming the control if not, and `--sync=samplerate` is the documented fallback). Full detail in `doc/CDIN-LINUX.md`.

7.8 The ESI U24 XL: what has to be configured, and three traps

The capture interface is an ESI U24 XL (USB Audio Class 1.0, USB 2.0 Full Speed, 32/44.1/48 kHz, 24-bit maximum). Two things the vendor documentation settles, so they are no longer assumptions:

- **it slaves to the incoming S/PDIF automatically** — ESI KB00307EN: when the source is clock master "the U24 XL will receive clock from the source and automatically will be slave", and there is no manual clock switch. The behaviour this design needs is the only behaviour it has;
- **the sample rate is not auto-detected.** Bit depth and rate must be set to match the incoming signal. Here that is `SNDCTL_DSP_SPEED` = 44100, which the daemon sets — but it means a non-44.1 source would be captured at the wrong rate rather than refused.

It is class compliant, so `uaudio(4)` drives it natively and there is no vendor driver and no Linux quirk to port (ALSA reaches its input selector through the same generic UAC1 parsing FreeBSD has). **Do not update the interface firmware:** a Linux report has S/PDIF capture working on the original firmware and becoming "completely distorted" after an upgrade.

Everything below was observed on FreeBSD 15.1-RELEASE-p2 and is documented at length in `cdin/ESI-U24XL.md`.

7.8.1 Unit numbers: nothing may depend on them

FreeBSD hands out `pcm` units in attach order, USB attach order is port order, and on this box the U24 XL sits on a lower-numbered root-hub port than the DAC, so it wins that race at every boot. Whichever card wins, the chain used to address the DAC by unit and nothing else — BruteFIR wrote `/dev/dsp0`, MPD's direct output was `/dev/dsp0`, `drc.sh` read the clock from `dev.pcm.0.feedback_rate` — so DRC would play into the S/PDIF interface and `cdin` would capture from the DAC.

There is no declarative way to pin the number, which is worth knowing before reaching for one. Unit wiring hints (`hint.pcm.1.at="uaudio0"`) rely on `BUS_HINT_DEVICE_UNIT`, which only `acpi(4)`, `pci(4)` and `isa(4)` implement — `uaudio(4)` does not, so the hint is silently ignored, and worse, `devclass_alloc_unit()` *skips* any unit carrying an `at` hint when numbering an unwired device, so hinting `pcm0` would take unit 0 away from the DAC as well. `hw.snd.default_unit` only selects among the units that already exist; it does not control enumeration. And `devd` is a userland consumer of events the kernel has already acted on: by the time the ATTACH notification arrives the unit is allocated and `/dev/dspN` exists, and it has no equivalent of udev's `NAME=/SYMLINK=`.

There is nevertheless one legitimate use of `hw.snd.default_unit`: pointing the unqualified `/dev/dsp` used by unrelated OSS applications at whichever card currently owns the DAC role. Because its value is still a volatile pcm number, `/etc/sysctl.conf` must not set it to a literal `0` or `1`. `omdrc_audio` writes it inside the locked role transaction after identity-based discovery and verifies the readback:

```
/sbin/sysctl "hw.snd.default_unit=${DAC_UNIT}"
got=$(/sbin/sysctl -n hw.snd.default_unit)
[ "$got" = "$DAC_UNIT" ] || warn "default unit mismatch"
```

This is ancillary to the stable links rather than a replacement for them. BruteFIR, MPD and cdin continue to use `/dev/dsp.dac` or `/dev/dsp.capture`; a later kernel or application change to the bare default cannot swap those project roles. Status prints both the discovered DAC unit and the default-unit readback so the distinction is auditable.

So stop trying to fix the number and stop using it. `devfs` accepts symlinks — that is exactly how `devfs.conf`'s `link` directive works, a plain `ln -fs` inside `/dev` run by `/etc/rc.d/devfs` — and the `omdrc_audio` service maintains two of them by **role**:

link	is	created
`/dev/dsp.dac`, `/dev/mixer.dac`	the DAC everything plays to	always
`/dev/dsp.capture`, `/dev/mixer.capture`	the CD/S-PDIF input	only when a capture card is named

Everything opens the DAC by name: BruteFIR's output device, MPD's `OKTO-DAC` output, `cdin --out`, the mpv launchers, `verify-bitperfect.sh`. A box with one sound card needs no configuration for this to be right, and a box with two stops caring which one enumerated first.

```
sysrc omdrc_audio_enable=YES
sysrc omdrc_audio_capture="ESI U24XL"   # only if you use the CD input
service omdrc_audio status              # prints the roles, exits 1 if unfilled
```

Roles are decided by identity, never by number. The DAC is not simply "the card that is not the capture card": a plain desktop has onboard HDA on `pcm0` and the USB DAC on `pcm1`, and capability cannot separate them either — an OKTO DAC8 reports `(play/rec)` exactly like a capture interface does (`dev.pcm.N.mode` is a bitmask, `PLAY 0x02 | REC 0x04`, and it reads 7). So an explicit match always wins, given either as a USB id or as a substring of the description `/dev/sndstat` prints:

```
sysrc omdrc_audio_dac="0x152a:0x88c5"          # vendor:product
sysrc omdrc_audio_dac="0x152a:0x88c5:000483"   # ...:serial, for two identical DACs
sysrc omdrc_audio_dac="OKTO RESEARCH"          # or just the name
```

With no match configured, playback-capable cards are ranked: a pure-playback USB DAC beats a USB play/record interface, which beats non-USB playback. If several candidates remain, the service reports that it guessed and prints the exact `omdrc_audio_dac` lines that can pin the intended card. The ids come from the card's USB parent, which the service reaches through `dev.pcm.N.%parent` -> `dev.uaudio.N.%pnpinfo`.

Two triggers, one code path. The rc.d service performs the cold-plug pass after devd is listening; a later attach/detach comes from `etc/devd/omdrc-audio.conf`, which fires on the **pcm** device rather than a USB interface. Both invoke the same complete reconcile operation:

```
attach 100 {
    device-name "pcm[0-9]+";
    action "/usr/sbin/daemon -f /usr/sbin/service omdrc_audio reconcile";
};
```

By the time the kernel announces `+pcm1 at ...` (`devaddq()`, `sys/kern/kern_devctl.c`) the unit is allocated and `/dev/dsp1` exists, so one pass reads the card's identity and links it with nothing to poll for. devd is not blocked because daemon(8) detaches the request. Matching the USB device instead — `system USB`, `subsystem DEVICE`, `type ATTACH` with `vendor/product` — fires *before* its `pcm` child exists and would need a retry loop to find it. `reconcile` is a full level rescan, so the same rule covers both roles, detach, a card that moved, and coalesced/reordered events.

The one thing a symlink cannot cover is a sysctl OID. `dev.pcm.<unit>.*` is keyed by the number we are trying to forget, and `drc.sh` reads the DAC's clock from it. That is not a blocker, it is two lines: the unit is read back off the link.

```
t=$(readlink /dev/dsp.dac)                    # -> "dsp0"
sysctl -n "dev.pcm.${t#dsp}.feedback_rate"
```

`drc.sh` has this as `dac_unit()`, the web panel as `_dac_unit()`, and `glitch-usbtap.sh` uses it to find the DAC's `uaudio` parent to tap. Everything that does *not* need the number uses `dac_dev()`, which is `/dev/dsp.dac` when the link exists and `/dev/dsp0` when it does not — so a single-DAC box that never enabled the service, and Linux, where the chain is ALSA, keep working unchanged.

The per-card `pcm` settings ride with the service, keyed by **role**:

```
omdrc_audio_dac_sysctls="bitperfect=1 play.vchans=0"
omdrc_audio_capture_sysctls="bitperfect=1 rec.vchans=0"
```

`/etc/sysctl.conf` is the wrong home for them for three independent reasons: `dev.pcm.<unit>.*` is keyed by the one thing that is not stable here; `/etc/rc.d/sysctl` runs at rc position 3, long before anything knows which card is which; and a re-attach re-creates the whole `dev.pcm.<unit>.*` tree from driver defaults, discarding anything applied earlier. That last point is why these are applied from the devd hook and not only at boot: unplug the DAC and plug it back in, and `bitperfect` is back to `1` before BruteFIR reopens it. Most global `hw.snd.*` and `hw.usb.uaudio.*` tunables are not unit-keyed and survive a re-attach, so they stay in `/etc/sysctl.conf`. `hw.snd.default_unit` is the deliberate exception: although the OID is global, its value names a role-dependent pcm unit and belongs to `omdrc_audio`.

Hotplug is now covered rather than refused, and nothing is ever replugged to achieve it — the previous design detached the capture card and USB-reset both so the kernel would renumber them in the wanted order, which cost the capture card its mixer state at every boot and could not be done at all while the chain was playing.

7.8.2 Trap 1: the S/PDIF input is called pcm2, and must be selected

The card has two inputs and only one is live at a time. Which one is a **mixer** setting, not a `cdin` setting, and on a fresh boot the card comes up on the *analog RCA* input — so `cdin` records silence from a perfectly healthy CD transport until the recording source is switched:

```
mixer -f /dev/mixer.capture pcm2.recsrc=set   # mixer.capture pairs with dsp.capture
mixer -f /dev/mixer.capture -s                # print just the active source
```

Only devices flagged `rec` can be a recording source; on this card those are `line` and `pcm2`, which are the two positions of the card's single USB selector unit (`sysctl dev.pcm.<unit>.mixer.selector_0`): `line` = position 1 = analog RCA, `pcm2` = position 2 = digital S/PDIF. They are mutually exclusive, so `set`, `add` and `toggle` all end with exactly one source. The name is not arbitrary: `uaudio` maps `UATE_SPDIF` to `SOUND_MIXER_ALTPCM` (`uaudio_tt_to_feature[]`), and index 10 of `SOUND_DEVICE_NAMES` is `pcm2`. The `dig1..3` names are only fallbacks for colliding selector pins, and the pre-14 syntax `mixer =rec dig1` no longer exists.

The setting does not survive a reboot, and a replug resets it too — `/etc/rc.d/mixer` only saves state for `mixer0` unless `mixer_enable="YES"` is set, and the U24 XL must be attached at boot for that to see it. So `omdrc_audio` asserts it, on every attach, right after it links the card:

```
omdrc_audio_capture_recsrc="auto"   # the default
```

`auto` prefers the digital input *by inspection* rather than by hardcoding this card: `mixer(8)` tags every device that can be a recording source `rec` and the active one `src` (`printdev()`, `usr.sbin/mixer/mixer.c`), so the service looks for `pcm2`, then `dig1..3`, among the sources the card actually offers, and leaves a card that has none of them alone. Name a device (`line`) to force the analog input instead, or `none` to keep hands off entirely. The command is idempotent and costs one USB control transfer.

7.8.3 Trap 2: `pcm2 = 0.00:0.00` is not a muted capture gain

`mixer -f /dev/mixer.capture` shows `pcm2` at `0.00` next to three devices at `0.75`, which reads as a zeroed record level. It is not: there is no gain to raise and raising it does nothing. Sweeping it moves no hardware node, while sweeping `line` moves them immediately, and there is no software fallback either (`sys/dev/sound/pcm/mixer.c` applies feeder volume only to `SOUND_MIXER_PCM`, which is the playback path). The `0.00` is a display artifact: there is no `[SOUND_MIXER_PCM2]` entry in `snd_mixerdefaults[]`, so it zero-initialises, exactly as `line` shows `0.75` only because it *is* in that table at 75. Nothing was measured from the card in either case.

The practical consequence is the correct one for this chain: the capture level on the S/PDIF input is whatever the transport sends, bit for bit, with no gain stage touching the samples.

7.8.4 Trap 3: a "44100 Hz" open that is neither 44.1 kHz nor bit-perfect

This is the expensive one, because every layer reports success. By default FreeBSD puts a **virtual channel** in front of the card. The hardware then runs at `dev.pcm.N.rec.vchanrate` — 48000, always — and a kernel `feeder_rate` resamples to whatever the application asked for. `SNDCTL_DSP_SPEED` returns 44100, `SETFMT` returns the requested width, every ioctl succeeds, and `cdin` is satisfied. What actually happens is that the card delivers 44100 frames per second into a stream the kernel believes is 48000:

> 44100 x 44100/48000 = **40517 Hz** arriving at the application.

cdin then reads 40.5k frames/s and writes 44.1k to the DAC. The lead drains at 3.6k frames/s, so a 2000 ms lead is gone in about 25 seconds, after which the DAC underruns continuously and the music is audibly destroyed. It looks exactly like catastrophic clock drift and it is not drift at all. The only place it is visible:

```
sysctl hw.snd.verbose=2 && cat /dev/sndstat

[dsp1.record.0]: spd 48000 ...                              <-- the hardware
dsp1.record.0[dsp1.virtual_record.0]: spd 44100/48000 ...
  ... -> feeder_rate(q:4  48000 -> 44100) -> {userland}    <-- the lie
```

The cure is the omdrc_audio_capture_sysctls above — rec.vchans=0 bitperfect=1 — after which the same command prints the whole capture path as {hardware} -> feeder_root(0x00210000) -> {userland}: one memcpy from the USB transfer to the read buffer, at the rate the transport is really running.

7.8.5 The consequence: capture is 24-bit

With the format feeder gone, the card's own width is the only one it accepts, and the U24 XL's capture endpoint offers exactly one — 24-bit S-LE, at either 44100 or 48000 Hz. So omdrc_cdin_bits is **24**, and the two ends of the bridge do not share a format: downstream runs S32_LE (virtual_oss -b 32, BruteFIR sample: "S32_LE"), and with bitperfect=1 the kernel's format feeder is gone by design, so the output device does not convert — it refuses the width. cdin therefore negotiates and converts itself (cdin/src/convert.c): the output is opened at the **source** width first, because no conversion is always truest; if the device refuses, a wider one is tried, **never** a narrower one. Widening is left-justification, which for little-endian PCM is pure byte placement (24 -> 32 moves bytes and adds a zero) — no arithmetic means no rounding, no dither, nothing to get wrong, so the bit-perfect claim survives it. Narrowing is never offered; it would need truncation or dither, which is what this daemon exists to avoid. --out-bits N forces the width instead of negotiating it.

24-bit capture also needed one fix inside cdin, and the arithmetic is worth recording because it looks like a bug in the device. SNDCTL_DSP_SETFRAGMENT encodes the fragment size as an exponent, so only powers of two can be *expressed*. A 24-bit stereo frame is 6 bytes, so 1024 frames is 6144 bytes = 2^11 x 3 — and no other period rescues it, since 6N = 2^k would need 3 to divide a power of two. **Every** period is inexpressible at 24-bit stereo. The fix was to stop treating the fragment as the transfer size: it is the device's *interrupt granularity*, i.e. how much the driver accumulates before waking a blocked read(), and the read itself asks for whatever it wants and blocks until that much has arrived. cdin now asks for the largest power of two that fits inside the period (4096 bytes for a 6144-byte period) and goes on reading its own 1024-frame periods. The device is woken slightly more often than strictly needed; not one byte of audio changes.

7.8.6 Checklist when the CD input captures silence

0. service omdrc_audio status — the U24 XL must hold the capture role with bitperfect=1 rec.vchans=0, and the DAC the dac role. It also prints the links and the active recording source. If the stream is not silent but *distorted*, and the lead drains to zero in about 25 s, it is Trap 3, not the selector.
1. ls -l /dev/dsp.capture — it must exist and point at the U24 XL's unit. If it does not, omdrc_audio_capture does not match the card's name.

2. `mixer -f /dev/mixer.capture -s` — must print `pcm2`. If it prints `line`, the card is on the analog input and `omdrc_audio_capture_recsrc` was set to `none` or to `line`.
3. Ignore `pcm2 = 0.00:0.00`; it is cosmetic.
4. Only then look at the transport, the cable and lock.

7.9 Status and what is still owed

The bridge, its state machine, the lazy output open, the width negotiation, the rc.d service and the web card are all in place, and the simulated transport has exercised a 12-track 16-bit disc end to end against a bit-perfect `/dev/dsp0` with `starves 0`, `drops 0` and the lead inside a one-period band throughout — including across track gaps, skips, seeks, a 4 s pause and an 800 ms scripted carrier dropout, which cost exactly 800 ms of lead and starved nothing. Three unit-test suites cover the places where a bug would be silent rather than loud: `test_ring` (wrap-around, the drop-oldest policy, every blocking path's wake-up, the trim-to-the-lead), `test_convert` (the widening, pinned by the *value* relation `src << (dst_bits - src_bits)` plus canary bytes, because a wrong byte index does not crash and does not warn) and `test_gate` (the silence threshold and the fact that one non-zero sample resets the whole run).

What is **not** yet proven is everything on the far side of `/dev/dspN` with a real transport attached:

- **carrier loss**: stop the CD, then unplug the coax. Does `read()` block, short-read, or error? The state machine assumes a short read or an error means `NO_CARRIER`, and that a player which merely *mutes* keeps delivering zeros and is caught by the silence gate. If a stopped player blocks the read forever instead, the daemon needs a read timeout to notice;
- **the real drift**: `omdrc-cdin --in /dev/dsp.capture --out none -d -s 30` reports the capture rate against the host clock; compare it with the OKTO's `feedback_rate`. Their difference is the drift the lead must cover;
- **loopback pacing**: `virtual_oss` runs with `-f /dev/null`, so it owns no hardware clock. Confirm that a writer to `/dev/dsp.play` is throttled at the DAC rate by BruteFIR draining `/dev/dsp.loop`, and check what happens when BruteFIR is *not* running — writes may block forever;
- **the shared-clock patch with two USB audio devices streaming** (section); ideally put the ESI on a different root hub.

Phase 2b — drift resync during the inter-track silence, by padding or trimming with a zero-crossing / 10 ms crossfade splice as the fallback — is not needed inside a disc, since drift cannot cause a discontinuity in 80 minutes, but it is needed for a session that never stops. The seams are marked `TODO(phase2b)` in the source.

8 FreeBSD peculiarities

A summary of everything FreeBSD-specific, in one place.

8.1 Naming and packages

- **MPD is `musicpd`**: package `audio/musicpd`, binary `musicpd`, service `musicpd`, client `musicpc` (vs `mpd`/`mpc` on Linux). omdrc-ctrl detects and uses the right client automatically.
- Services are rc.d scripts under `/usr/local/etc/rc.d/`, enabled with `sysrc <name>_enable=YES`, run manually with `service <name> onestart/onestop`. Hotplug is devd, not udev.
- Service names, rc.d filenames, rc.conf keys, and service-specific hook names use underscores (`omdrc_audio`, `omdrc_audio_enable`); standalone devd configuration filenames use hyphens (`omdrc-audio.conf`). The separators are not interchangeable in `service`, `rcorder`, or `PROVIDE`/`REQUIRE` tokens.

8.2 OSS instead of ALSA; virtual_oss and cuse

FreeBSD's native audio API is OSS. The loopback is `virtual_oss` (userland, base system) creating cuse character devices: `/dev/dsp.play` (MPD writes) and `/dev/dsp.loop` (BruteFIR reads, synchronized `-L` mode). The `cuse` kernel module must be loaded (`kld_list` or `etc/rc.d` glue). BruteFIR's OSS I/O is built in (no ALSA needed); the fork's OSS fixes matter here.

Key sysctls for the bit-perfect direct path:

```
bitperfect=1     # first opener's format becomes the hardware format
play.vchans=0    # no virtual-channel mixer/resampler
```

Applied to the DAC's unit by `omdrc_audio` (`omdrc_audio_dac_sysctls`), on every attach — a re-attach rebuilds `dev.pcm.<unit>.*` from driver defaults, so a replug would otherwise silently lose them.

These also mean the DAC is **single-open**: exactly one client at a time (BruteFIR when DRC is on; MPD's direct output or a browser otherwise).

8.3 Known FreeBSD issues and their status

- **A statically configured wired port that has no cable.** `em(4)` keeps `RUNNING` set without a carrier, so a static `ifconfig_em0="inet ..."` stays fully configured on a dead link. libupnpp then binds it and upmpdcli becomes undiscoverable while still running and still driving MPD; a static `defaultrouter` compounds it by overriding the route Wi-Fi learned. Both are `rc.conf` bugs, not upmpdcli bugs: use DHCP on the wired port and leave `defaultrouter` unset — see A disconnected wired port makes it invisible.
- **OKTO 44.1 kHz-family flicker** (bug #295933): the DAC continuously drops and re-acquires USB streaming lock on 44.1/88.2/176.4/352.8 kHz, while the 48 kHz family is stable and Linux plays everything fine. Root cause: the device has one UAC2 Clock Source **shared** between playback

and capture, and `uaudio(4)` lets the vestigial capture side reprogram it to its 48 kHz default under the active playback. Fixed by the shared-clock patch (Appendix).

- **Rate-change cold-open silence**: on the first open after *any* rate change the DAC shows the right rate and streams healthy USB but routes no audio; a second open fixes it. Stock `uaudio` programs the clock *after* selecting the streaming alt-setting (Linux does the opposite). Worked around by `drc.sh` priming; properly fixed by the clock-before-alt patch.
- **virtual_oss livelock** ("155% CPU", frozen chain): a `SNDCTL_DSP_SETTRIGGER` on a read-only fd could strand the synchronized engine in a wait that nothing wakes. Fixed by the settrigger patch (Appendix).
- **cuse teardown wedge** (bug #296291): stopping virtual_oss could leave it unkillable in D<E state, pinning `cuse.ko`, reboot required — a kernel refcount leak in `cuse_client_open()`'s `is_closing` error path (regression from commit `634e578ac7b0`, in 15.1). Kernel fix written (Appendix).
- **`pcm` unit numbers are attach order, and nothing declarative can change them**: no unit-wiring hint (`uaudio(4)` does not implement `BUS_HINT_DEVICE_UNIT`), and no devd rule can pre-empt it (the unit is allocated before the event is delivered). With a second USB audio device present, the DAC can lose `/dev/dsp0`. Handled by not depending on the number: `omdrc_audio` keeps `/dev/dsp.dac` on the right card at boot and on every hotplug, and the few sysctl readers resolve the unit from that link (chapter).
- **A capture open can succeed at the wrong rate**: with a record virtual channel in front of the card the hardware runs at `rec.vchanrate` (48000) and `feeder_rate` resamples, while every ioctl reports the rate that was asked for. A 44.1 kHz S/PDIF source then arrives at ~40517 Hz and looks like catastrophic clock drift. `rec.vchans=0` plus `bitperfect=1` is the cure (chapter).
- **musicpd 100% CPU on HTTP streams**: a libcurl leftover-fd spin — curl registers an event fd into MPD's I/O loop and never removes it, so the loop burns a core in `poll()`. **Audio is unaffected**; MPD upstream has triaged it third-party (issues #2244/#2229); the right venue is libcurl.
- **One wire format per attach**: `uaudio` fixes s32le at attach and pads 16-bit content to 32 bits on the wire, so the DAC panel shows 24 bits on 16-bit tracks. Bit-perfect (zero-padding is lossless), just unlike Linux's per-stream alt switching. Knobs: `hw.usb.uaudio.default_bits`/`default_rate`.

8.4 Video-related FreeBSD constraints

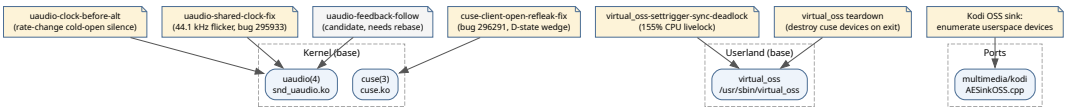
- Physical Blu-ray: no kernel UDF 2.50 mount; raw `/dev/cd0` needs sector-aligned reads and has no read-ahead — hence mpv + libbluray + gcache (section). Kodi's internal player cannot do it.
- Kodi's OSS sink does not enumerate cuse userspace devices at all — the in-tree Kodi patch fixes that (Appendix).

8.5 Toward a real FreeBSD port

The path from the run-from-repo model to an official `audio/open-media-drc` port is planned in four phases — upstreaming the patches, splitting engine from site data, the port itself, and submission. The full plan is Appendix .

9 Appendix A — FreeBSD patches in detail

Local fixes kept in-tree while the official FreeBSD fixes are pending. All `/usr/src` patches apply with `-p1` against `releng/15.1`.



The patched layers at a glance.

> **Upgrade caveat (applies to every kernel/userland patch below):** `freebsd-update` or `make installkernel` overwrites patched binaries with stock ones. After any OS/kernel update, re-apply the patches and rebuild. A `.ko` is ABI-specific to its kernel — never keep prebuilt binaries, always rebuild from the patches.

9.1 uaudio(4) patches (`freebsd-uaudio-patch/`)

Target device: OKTO RESEARCH DAC8 STEREO (USB `0x152a:0x88c5`, Thesycon UAC2 firmware, USB High-Speed). Two patches are applied in order on top of stock `releng/15.1`; a third, built but not yet installed, closes what those two left unfinished; a fourth is an unbuilt candidate.

9.1.1 1. `uaudio-clock-before-alt.c.patch` — rate-change cold-open silence

Problem. On the first open after *any* sample-rate change the DAC opens the stream, shows the correct rate, streams healthy USB (feedback present, no underruns) — and routes silence. A second open at the same rate fixes it, which is why `drc.sh` had to prime (and why users had to "run drc.sh several times").

Root cause. Stock `uaudio_configure_msg_sub()` starts a stream as: `SET_INTERFACE` to the streaming alt-setting (arming the device), *then* `SET_CUR` the sample rate on the UAC2 Clock Source (possibly a crystal switch, yanked under the armed interface), then start transfers. Linux does the opposite: park at alt 0, program the clock idle, then select the alt. The Thesycon firmware latches its stream configuration at `SET_INTERFACE` time, so FreeBSD's order arms the stream against the *old* clock. This also explains why the open/close prime works: every close parks at alt 0.

The fix. For UAC2 devices in the `CHAN_OP_START` path: park the streaming interface at alt 0; program the clock while idle (legal — the UAC2 clock lives on the AudioControl interface); on a genuine rate change sleep `hw.usb.uaudio.clock_settle_ms` (default 100 ms, runtime-tunable, clamped to 2000) for crystal relock; then `SET_INTERFACE` and start. UAC1 devices are untouched (their rate control lives on the streaming endpoint).

Status. Built + installed since 2026-07-06; listening reports say different-rate tracks now lock first try. Replaces the host-side `DAC_PRIME_CYCLES` prime for all clients (drc.sh, MPD-direct, browsers). If a crossing is ever silent, raise `sysctl hw.usb.uaudio.clock_settle_ms`.

9.1.2 2. `uaudio-shared-clock-fix.c.patch` — the 44.1 kHz flicker (bug #295933)

Problem. Continuous drop/re-acquire of USB streaming lock on the 44.1 kHz rate family; the 48 kHz family is stable; Linux plays everything.

Root cause. The DAC exposes **one UAC2 Clock Source shared between playback and capture**. On async playback, `uaudio` auto-starts the record channel purely as a jitter-information source — at its own nominal rate (48 kHz default), whose SET_CUR clobbers the shared clock out from under active 44.1 kHz-family playback.

The fix — three cooperating, device-agnostic changes:

a. **Rate-align the jitter record stream**: before the auto-started record channel starts, set its alt to the one matching the playback rate, so the rec-side SET_CUR becomes a same-value no-op *and* the rec channel's framing stays consistent (valid jitter feedback instead of catastrophically wrong).
b. **Shared-clock guard**: before any UAC2 SET_CUR to a clock shared by both directions, skip it if the other direction is running at a different rate — the first active stream owns the clock.
c. **Always submit the explicit-feedback SYNC transfer**, so `dev.pcm.<unit>.feedback_rate` stays live as a diagnostic (drc.sh's chain-sanity signal) even with capture present.

A guard-only version was audited and rejected: without (a), the rec channel expects 48 kHz framing while the device correctly delivers 44.1 kHz, the jitter estimate pins at its negative clamp, and the play callback strips samples continuously — worse than the flicker. This fix **replaces** the retired VID/PID-gated capture-disable workaround; `pcm0 (play/rec)` in sndstat is the *expected* state again.

Status. Applied to `/usr/src` 2026-07-07, builds `-Werror`-clean alone and on top of patch 1; the combined module was pending install + listening test at the time of writing. Upstream: bug #295933 / PR 2323, landed as `755685dd665e` (MFC `6886e8a9a0aa`) — superseded by patch 3 below.

9.1.3 3. `uaudio-clock-transaction.c.patch` — the residual 44.1 kHz silent open (2026-08-26 follow-up)

Problem. Patch 2 landed upstream as `755685dd665e`, but a follow-up audit of the OKTO still occasionally refusing 44.1 kHz found the shared-clock guard incomplete: it skips a SET_CUR only on a rate *mismatch*, and the same commit's own jitter-stream rate alignment guarantees a *match* — so the capture pass issues a second, same-value SET_CUR to the shared clock right after playback is armed and streaming. Confirmed on the wire with `hw.usb.uaudio.debug=6`: two SET_CUR to clock 41, one settle, on every open. Upstream is worse off than this host, having no clock-before-alt reorder (patch 1), so its second write lands under two armed interfaces rather than one. Separately measured: `uaudio(4)` auto-streams the OKTO's vestigial capture interface for the lifetime of every playback — 251.9 isochronous completions/s against 125.4 playback interrupts/s, half the device's isochronous bandwidth — to recompute a number the explicit feedback endpoint already reports.

The fix. (a) Never write a UAC2 clock another stream owns; (b) read it back with GET_CUR first and skip a redundant write, as Linux does; (c) stop borrowing the vestigial capture stream for jitter when the playback alt has an explicit feedback endpoint, gated by `hw.usb.uaudio.prefer_feedback`; (d) do not start the jitter capture stream at a stale rate.

Not claimed: that this causes the intermittent silent-open symptom that prompted the audit — it did not reproduce in 42 controlled open/play/close cycles (21 of them into 44.1 kHz). The defects stand on the code, the USB trace and the isochronous counters, independently of that symptom.

Status. Built `-Werror`-clean with and without `USB_DEBUG`, on top of patches 1 and 2. **Not installed, not listening-tested.** Split for upstream submission as `uaudio-upstream-0001-shared-clock-write-discipline.c.patch` and `uaudio-upstream-0002-prefer-explicit-feedback.c.patch` in `upstream-series/` — `git am`-clean on `main` after `755685dd665e`, with a PR description and Bugzilla comment ready to send (same route as before: a GitHub PR, committed by `christos@`; CC `hselasky@` on the isochronous-policy item). Full audit, device-generality analysis and A/B test plan in `uaudio-clock-transaction.md`; what remains declared-but-not-patched in `SUBMISSION-295933.md`. `bench/` holds a DAC lock bench (per-rate test tones, repeated open/play/close, verdict from the DAC's analog output or by ear) and `bench/uaudio-affects.py`, which decides from USB descriptors alone whether any device is affected.

9.1.4 4. `uaudio-feedback-follow.c.patch` — candidate (unbuilt)

Sketch to make playback follow the device's reported feedback rate smoothly (Linux-style), targeting an occasional tick. Touches the same sync-callback region as patch 3, so it **needs rebasing**, and becomes live only once the vestigial capture stream stops being auto-started (patch 3, item c).

9.1.5 Applying and building

```
cd /usr/src
patch -p1 < uaudio-clock-before-alt.c.patch
patch -p1 < uaudio-shared-clock-fix.c.patch   # applies with offsets; also on pure stock
patch -p1 < Makefile.patch                     # adds -DUSB_DEBUG

cd /usr/src/sys/modules/sound/driver/uaudio
make clean && make
```

Install the module (back up stock first, refresh the backup after every OS update so the revert path matches the running ABI):

```
OBJ=/usr/obj/usr/src/amd64.amd64/sys/modules/sound/driver/uaudio/snd_uaudio.ko
sudo cp -f /boot/kernel/snd_uaudio.ko /boot/kernel/snd_uaudio.ko.orig
sudo service musicpd stop                 # release the DAC
sudo cp -f "$OBJ" /boot/kernel/snd_uaudio.ko
sudo kldunload snd_uaudio                 # devd auto-reloads on attach
UG=$(usbconfig | awk '/DAC8STEREO/{print $1}' | tr -d ':')
sudo usbconfig -d "$UG" reset             # clean re-enumeration
sudo sysctl -f /etc/sysctl.conf           # restore buffer_ms baseline
sudo service musicpd start
```

Verify: `grep pcm0 /dev/sndstat` shows `(play/rec)`; `sysctl hw.usb.uaudio.clock_settle_ms` exists; `sysctl dev.pcm.$(readlink /dev/dsp.dac | tr -dc 0-9).feedback_rate` tracks the playback rate. Then run the listening matrix (both crystal directions, same-family changes, MPD-direct mixed-rate queue, browsers) — machine signals cannot verify these fixes; only listening counts. Revert with the `.orig` copy + kldunload/kldload.

9.2 virtual_oss / cuse patches (`freebsd-virtual-oss-patch/`)

Two related bug clusters, both root-caused live on this box.

9.2.1 1. Runtime livelock: `virtual_oss-settrigger-sync-deadlock.patch`

Symptom. Minutes after a chain (re)start, playback freezes: MPD stops advancing, BruteFIR starves, `virtual_oss` burns 150–200% CPU, both clients stuck in `cuse-cli` waits. Stopping anything from this state walks into the teardown minefield below.

Root cause (captured with procstat + gdb on the live process): a pair of userland bugs in `usr.sbin/virtual_oss`:

1. `SNDCTL_DSP_SETTRIGGER` ignores the fd's open mode: BruteFIR's OSS layer triggers both directions with one call (harmless on kernel pcm), flipping `tx_enabled = 1` on the read-only `/dev/dsp.loop` fd that can never write.
2. The synchronized-loopback engine wait loops check `tx_enabled` **once, outside the wait**, and the trigger/halt ioctl paths never wake the engine — so once the engine parks waiting for play data from a client that will never write, nothing ever re-evaluates the premise. The arming window is microseconds wide against a 200 ms block cadence — hence intermittent.

The fix (four changes, upstreamable): SETTRIGGER honours the open mode (`fflags`); trigger/halt ioctls `atomic_wakeup()` the engine; the sync wait loops re-check `tx_enabled/rx_enabled`; and a new `tx_written` latch so the engine never sleeps waiting for a client that has never written.

9.2.2 2. Teardown deadlock: userland device destroy + kernel refleak

Symptom. Stopping `virtual_oss` intermittently wedges it forever in `D<E` / `MWCHAN W` (SIGKILL-immune), pins `cuse.ko` (kldunload hangs too), a new virtual_oss cannot recreate the devices — reboot required. Every DRC rate change was a reboot risk.

Two layers:

- **Userland**: `virtual_oss` never destroyed its cuse devices on exit, so the kernel's `cuse_server_free()` busy-waited in `pause("W", hz)` for client refs that were never released. Patches `virtual_oss-teardown-int.h.patch` + `virtual_oss-teardown-main.c.patch` keep each `cuse_dev_create()` handle and call `cuse_dev_destroy()` on all DSP/WAV/loopback devices before exit. (Upstream committed an equivalent as `0bd5ef6b4363`.)
- **Kernel** (the real fix — bug #296291): a regression from commit `634e578ac7b0` (Nov 2025, in 15.1). In `cuse_client_open()`, the `is_closing` / `si_drv1==NULL` error path returns with `pcs->refs` incremented, the client left linked in `hcli`, and the destructor never registered — **every open racing into the teardown window leaks one server ref**, and `cuse_server_free()` waits forever. Proven live with a diagnostic `cuse.ko` + dtrace + kgdb (refs frozen at 4, three leaked clients on `dsp.loop`). Fix: `cuse-client-open-refleak-fix.patch` calls `cuse_client_free(pcc)` on both error paths; committed on branch `fix/cuse-client-open-refleak-296291`, compile-tested, `Fixes: 634e578ac7b0`, `PR: 296291`. The userland destroy mitigates the normal exit but does nothing for `kill -9` — the kernel leak stays latent without this.

- A diagnostic-only `cuse-teardown-diag.c.patch` logs the stuck refcount every ~5 s from the `pause("W")` loop (pure printf; remove once the kernel fix lands).

9.2.3 Applying and building

```
cd /usr/src
patch -p1 < virtual_oss-settrigger-sync-deadlock.patch
patch -p1 < virtual_oss-teardown-int.h.patch
patch -p1 < virtual_oss-teardown-main.c.patch
patch -p1 < cuse-teardown-diag.c.patch        # optional diagnostic

# userland:
cd /usr/src/usr.sbin/virtual_oss && make && sudo make install

# kernel module (diagnostic or refleak fix):
cd /usr/src/sys/modules/cuse && make && sudo make install
```

Reboot first if a wedged virtual_oss is pinning `cuse.ko` — a new module cannot load and the devices cannot be recreated until then. Validation: churn `drc.sh <rate>` / `off` cycles ~20x; `virtual_oss` must stay near-idle CPU and always exit cleanly (no D<E in `ps -o pid,stat,mwchan`, `cuse.ko` unloadable).

9.3 Kodi OSS sink patch (`kodi-virtual-oss-patch/`)

Problem. Kodi's audio settings only ever offered the hardware DAC. `CAESinkOSS::EnumerateDevicesEx()` counts kernel PCM cards via `SNDCTL_SYSINFO`; cuse userspace devices are not kernel cards, so `/dev/dsp.play` was never listed — and the settings filler silently *resets* any configured value it cannot match against the enumerated list, so hand-editing `guisettings.xml` never stuck.

The fix. After the kernel-card loop, parse the *"Installed devices from userspace:"* section of `/dev/sndstat` (the only place cuse devices are advertised) and add each node, probed exactly like a kernel card (`SNDCTL_ENGINEINFO` for formats/channels/rates, non-blocking open, graceful fallbacks so a busy device is still listed, dedup against kernel cards). Generic — any userspace OSS device is listed.

Result (verified on `multimedia/kodi` 22.0a3): *"dsp.play virtual_oss device"* appears in Settings > System > Audio, selecting it persists, and Kodi feeds the DRC chain. `virtual_oss` must be running when Kodi initialises audio.

Apply via the ports tree:

```
sudo cp patch-xbmc_cores_AudioEngine_Sinks_AESinkOSS.cpp \
        /usr/ports/multimedia/kodi/files/
cd /usr/ports/multimedia/kodi
make config && make patch && make build
sudo make deinstall && sudo make install   # same version -> reinstall is a no-op
strings /usr/local/lib/kodi/kodi.bin | grep -c "from userspace"   # must print 1
```

Upstreaming: to the FreeBSD port (`files/` patch, PR to the maintainer) and/or Kodi upstream (`AESinkOSS.cpp` PR; pre-empt the “why parse sndstat text” question — cuse devices are unreachable via the mixer ioctls).

10 Appendix B — The FreeBSD port plan

Status: **plan only, nothing applied.** Linux packaging is explicitly out of scope for now (one OS at a time). Source: `doc/FREEBSD-PORT-PLAN.md`.

10.1 Why the repo cannot be ported as-is

A FreeBSD port installs *identical, immutable* files on every machine, under hier(7) paths, from a versioned release tarball. The run-from-repo model violates that on every axis — deliberately, because it optimizes for a zero-config personal appliance:

1. **Files are rendered per-host**: `install.sh` bakes `config.env` values (`@AUDIO_USER@`, `@AUDIO_HOME@`, `@REPO_DIR@`) into the live files; a package must install the same bytes everywhere and configure at runtime.
2. **The tree is written at runtime**: `drc.sh` keeps `last_arg`, `last_power`, `drc.log` beside itself; `pkg check -s` flags modified packaged files — state must live in `/var/db/`.
3. **Room-specific data is mixed with software**: `configs/120.blue`, `filters/*` (~50 MB) are personal measurement products; a port must ship neutral defaults. `OMDRC_SITE_DATA_DIRS` / `OMDRC_SITE_ROOT` now resolve these in a separate checkout beside the engine.
4. **rc.d scripts shadow other ports**: `etc/rc.d/musicpd` and `upmpdcli` would replace scripts owned by `audio/musicpd` / `net/upmpdcli`; the stock scripts' rc.conf knobs must be used instead.
5. **Dependency on a personal BruteFIR fork**: `RUN_DEPENDS` must resolve to ports.
6. **Kernel/userland patches**: a port cannot patch the base system (uaudio, cuse) and should not carry patches for another port (virtual_oss) — these must land upstream first.
7. **Missing packaging basics**: no LICENSE, no tagged releases (the `v*` tag series this manual belongs to is the first step), and the tarball would ship debugging journals and kernel patches.

Run-from-repo does not have to die: the repo becomes a normal upstream that *also* supports `make install PREFIX=... DESTDIR=...`; run-from-repo stays the development/appliance mode and the port is a thin consumer of tagged releases.

10.2 Phase 0 — Upstream the out-of-tree pieces (prerequisite, in flight)

- **uaudio patches** (shared-clock fix, clock-before-alt) to FreeBSD base — in progress: bug 295933 / PR 2323.
- **cuse refleak fix** to base — in progress: bug 296291.
- **virtual_oss SETTRIGGER deadlock patch** to upstream `hselasky/virtual_oss`, so `audio/virtual_oss` inherits it.
- **BruteFIR fork** — options in order of preference: (a) upstream the delta to Anders Torger (upstream is dormant — unlikely); (b) submit the delta as `files/` patches to the existing `audio/brutefir` port (viable if the delta stays small and FreeBSD-relevant); (c) release the fork as its own project + port (`audio/brutefir-omdrc`) — most work, only if
    b. is refused.

- **kodi-virtual-oss-patch**: not shippable by this port — upstream to Kodi or to `multimedia/kodi`'s `files/`, or drop from the tarball.

10.3 Phase 1 — Make the repo package-friendly

Worth doing even if the port never happens; it makes the repo usable by anyone, not just this box/room.

- **1.1 Engine / site-data split.** Engine (shipped): drc.sh, drc-status.sh, omdrc-ctrl, video/webremote, browser-nodrc, own rc.d/devd glue, sample MPD/upmpdcli snippets, docs. Site data (not shipped): the `configs/` and `filters/` measurement products, room README sections, measurement plots — moved to `examples/rooms/` or a separate private overlay checked out beside the engine (`OMDRC_SITE_DIR`).
- **1.2 FLAT default filters** (first — small and independent): ship a `configs/flat/` geometry as default, one conf per rate, using BruteFIR's built-in identity coefficient `filename: "dirac pulse"` with `attenuation: 0.0` — no binary filters needed, and `verify-bitperfect.sh` can pass through the flat chain as a plumbing self-test. Keep `filter_length` and I/O identical to the room configs so swapping in real filters is a filename change.
- **1.3 Runtime configuration instead of render-time baking**: drc.sh and friends read a config at runtime (`$OMDRC_CONF` -> `${PREFIX}/etc/open-media-drc/omdrc.conf` -> `<script-dir>/config.env`, the last keeping run-from-repo unchanged); rc.d defaults flip to installed paths via the port's `SUB_FILES`; brutefir confs rendered on the fly to a state-dir tempfile so packaged confs are host-neutral.
- **1.4 State out of the tree**: `last_arg`, `last_power`, `drc.log` to `/var/db/omdrc/` in installed mode (beside the script as fallback); `/tmp/brutefir.out`, `/tmp/virtual_oss.pid` to the same state/run dir.
- **1.5 Install target** (`make install` with `DESTDIR`/`PREFIX`): scripts to `${PREFIX}/libexec/omdrc/` + a `${PREFIX}/bin/omdrc` wrapper; `omdrc.conf.sample` + flat configs to `${PREFIX}/etc/open-media-drc/`; devd conf; MPD/upmpdcli snippets to `share/examples/`; omdrc-ctrl to `share/omdrc-ctrl/`; docs to `share/doc/open-media-drc/`. rc.d scripts for our own services are installed by the port via `USE_RC_SUBR`.
- **1.6 Housekeeping**: pick a LICENSE (BSD-2-Clause fits the ecosystem); tagged releases whose tarballs exclude `freebsd-*-patch/`, investigation journals and site data (`git archive` + `export-ignore`); split the README into a user quickstart vs `doc/DEVELOPMENT.md`.

10.4 Phase 2 — The port itself

`audio/open-media-drc` (working name): `USE_GITHUB=yes`, tagged `DISTVERSION`, `NO_BUILD` for the shell core, `USES=python:run shebangfix` for omdrc-ctrl. `RUN_DEPENDS`: brutefir (per the Phase 0 decision), virtual_oss, musicpd, sox/soxr. `OPTIONS_DEFINE`: `CTRL` (web UI: py-flask/Markdown/numpy), `VIDEO` (webremote: mpv), `UPNP` (upmpdcli). `USE_RC_SUBR` for `omdrc_audio` (plus `omdrcctrl` with the CTRL option) — **not** musicpd or upmpdcli; a pkg-message documents the rc.conf lines pointing the stock scripts at our configs. `@sample` entries for every config. Validate with `portlint -AC`, `portclippy`, `poudriere testport`, and `pkg check -s` after a service run (catches leftover in-tree writes).

10.5 Phase 3 — Submission and maintenance

Submit as a Bugzilla PR (Ports & Packages), `MAINTAINER= delleceste@gmail.com`. Niche integration ports are accepted when clean and maintained — the bar is quality, not popularity. Expect review rounds on rc script style, sample handling, and the brutefir dependency; having the Phase 0 fixes merged upstream is the strongest argument the stack works on stock FreeBSD. Ongoing: bump the port per release, watch fallout from musicpd/upmpdcli/virtual_oss updates.

10.6 Recommended order of value

1. **Phase 0 upstreaming** — benefits every FreeBSD USB-audio user, already in flight, and a hard prerequisite anyway.
2. **Phase 1.2 (flat filters)** — small, immediate, makes the repo usable by others today even without a port.
3. **Rest of Phase 1** — worthwhile for the repo's own health regardless.
4. **Phases 2–3** — only once the BruteFIR question is settled and there is evidence of an audience; a port is a maintenance promise.

10.7 A clean installable image (planned, not built)

`doc/USB-APPLIANCE-IMAGE-PLAN.md` scopes a USB stick that installs FreeBSD with `open-media-drc` preinstalled — no room filters, no per-geometry BruteFIR configs, no personal data, no git history, no build artifacts — ready for a specific room's `configs/<geo>` + `filters/<geo>` to be dropped in afterward, the way a fresh port install would be configured. Target hardware is a fanless Intel/AMD-integrated-audio box dedicated to the appliance; **bee is explicitly out of scope**, staying the dev/measurement machine with its own disk/filesystem issues (root UFS, 99% full, a legacy partition layout sharing the disk).

The plan builds on work already mid-flight rather than starting over: `OMDRC_SITE_DATA_DIRS` + `GEOMETRY=flat` already gives a built-in generic mode (BruteFIR's identity coefficient, no filter files needed); `.gitattributes export-ignore` already strips filters, per-room configs, the FreeBSD patch trees and the investigation-journal Markdown from `git archive` output on a tag; and Phase 1.5 of Appendix B's own port plan (`make install DESTDIR=... PREFIX=...`, reading config at runtime rather than baking `@AUDIO_USER@` in at render time) is what should land on the image rather than bee's live `install.sh` + `config.env` flow. Status is plan-only: nothing has been built from it yet.

11 Appendix C — Bit-perfect test assets and cross-OS comparison

The Tools chapter covers the single-host proof (the page, its implementation). This appendix documents the test assets and the cross-OS procedure that proves the Linux and FreeBSD boxes send the DAC the *very same bytes*.

11.1 Test assets (`tests/`)

Two deterministic, near-silent (~ -90 dBFS) signals; every sample is uniquely determined, so any truncation, dither, volume change, resampling or channel swap shows up immediately.

- **Short asset (committed)** — `bitperfect-test-44100-s32-stereo.wav`: S32_LE, 2 ch, 44100 Hz, 100000 frames (~2.27 s). Per-sample counter in the low 16 bits, `L = i & 0xFFFF`, `R = (i*40503) & 0xFFFF`. Its PCM payload is byte-identical to the reference `.raw` (the WAV is the same bytes plus a 44-byte header — MPD cannot play headerless raw).
- **Cross-OS asset (generated, not committed)** — `bitperfect-test-44100-s32-stereo-30s.wav`: S32_LE, 2 ch, 44100 Hz, 1323000 frames (30 s), 10.6 MB. Here *every* `(L,R)` pair is unique over the whole file (`R = (i*40503 + (i >> 16)) & 0xFFFF` folds the block index in, breaking the 65536-frame period), so capture alignment is unambiguous at any length. It is too large to commit; regenerate it byte-identically on any OS:

```
python3 tests/gen-bitperfect-wav.py \
    tests/bitperfect-test-44100-s32-stereo-30s.wav
# sha256 88d365eeaccb1fa830bb1a2726b0f29bb545885824351080e0c5b4cbc9602348
```

11.2 Other rates and sample widths

The generator takes `--rate`, `--bits` (16/24/32) and `--frames` (= seconds x rate), so any format the DAC advertises can be tested. Both tap scripts read rate and width from the WAV header — feeding them a different file is the whole configuration:

```
python3 tests/gen-bitperfect-wav.py --rate 192000 --bits 24 --frames 5760000 \
    tests/bitperfect-test-192000-s24-stereo-30s.wav
./scripts/bitperfect-tap-linux.sh tests/bitperfect-test-192000-s24-stereo-30s.wav
```

The sample *values* are the same counter at every width (they never exceed `0xFFFF`), so only the container changes — which means a 16/24-bit asset additionally exercises the **lossless promotion to the 32-bit USB wire container** (<<8 for 24-bit, <<16 for 16-bit) that any bit-perfect player must perform for a DAC accepting only 32-bit containers. A 16/24-bit input therefore does not change the playback path at all: `prep` promotes first and the player always emits S32_LE.

Two consequences: **each format has its own sha256** (a 24-bit file stores 3 bytes per sample, so it is a different file), and a cross-OS comparison must use the **same width on both machines** — differently shifted wire values never byte-match. The report's `ref bytes` hash is that of the promoted stream, so it too differs per width at the same rate.

Rate	Bits	Frames	Size	sha256 (first 16)
44100	32	1323000 (30 s)	10584044	`88d365eeaccb1fa8`
44100	24	1323000 (30 s)	7938044	`e2702c119606cdf8`
192000	24	5760000 (30 s)	34560044	`58dd87f3560334fb`
192000	24	1920000 (10 s)	11520044	`01317af6523ec67f`
96000	24	960000 (10 s)	5760044	`b572faabdee3b623`

Any other combination is equally valid: the generator prints the sha256 of whatever it writes — generate once, note the hash, match it on the other machine (full hashes in `tests/README.md`). `.gitignore` excludes the generated assets by name (the 44100/32-bit and 44100/24-bit 30 s files and the 192000/24-bit 10 s one), so add any further WAV you keep, or drop it under `bp-results/` (ignored except for `*.txt`).

11.3 Verification status

Both taps are executed and passing — the FreeBSD side is no longer a written-but-unrun script:

Host	Runs	Result
Linux (DacMagic 100, kernel 7.1.5-arch1)	44100/32-bit x 30 s, 44100/24-bit x 30 s, 192000/24-bit x 10 s	all **BIT-PERFECT**, 0 truncated events, 0 usbmon drops
FreeBSD 15.1-RELEASE (same DAC, `usbus0` devaddr 2)	same three	all **BIT-PERFECT** (exit 0); the 192 kHz run confirms the DAC clock followed (`dev.pcm.0.feedback_rate` = 191994) and costs ~24 s wall clock

`bitperfect-compare.py` reports **MATCH** across the two hosts for the 44100/32-bit asset; the 24-bit pairs have no committed Linux counterpart yet, so those stand as local per-host proofs. The comparator itself has been exercised on every path (wav/wav, wav/txt, txt/txt, refusal of raw/txt) plus a deliberately bit-flipped payload, which it reports as MISMATCH at the exact offset.

The FreeBSD tap's first run exposed a real defect — a capture truncated by ~17 ms because `usbdump` discards its unflushed buffer on exit. The fix is a 500 ms **silence pad**: the player plays `ref.raw` plus half a second of zeros, while the verdict still compares the unpadded reference, so the loss lands in silence nothing depends on. The pad is removed by *arithmetic* (take `len(ref)` bytes from the alignment point), never by silence detection — the test signal is itself near-silent, so a zero-seeking trim would eat real payload.

11.4 Cross-OS byte comparison

Each OS taps its own USB isochronous OUT endpoint while playing the (locally regenerated) common WAV, then the reports are compared. Only the tiny `bp-results/*.txt` reports are committed — they carry the tap payload's length and sha256, which proves byte-identity without moving the 10 MB streams through git.

```
# Linux box:
./scripts/bitperfect-tap-linux.sh \
    tests/bitperfect-test-44100-s32-stereo-30s.wav
git add bp-results/*-linux.txt && git commit && git push

# FreeBSD box (free the DAC first: ./drc.sh off):
./scripts/bitperfect-tap-freebsd.sh \
    tests/bitperfect-test-44100-s32-stereo-30s.wav
git add bp-results/*-freebsd.txt && git commit && git push

# then on either box:
git pull
./scripts/bitperfect-compare.py \
    bp-results/bitperfect-test-44100-s32-stereo-30s-linux.txt \
    bp-results/bitperfect-test-44100-s32-stereo-30s-freebsd.txt
```

Identical length and sha256 on both reports proves the two operating systems deliver bit-identical audio to the DAC. Step-by-step commands and the mismatch-forensics path are in `scripts/README.md` and `doc/BIT-PERFECT-VERIFICATION.md` (*Cross-OS comparison*).

A single run already proves the local path. Each tap compares its capture against the reference derived from the input file *on that machine* and exits 0 on **BIT-PERFECT** — a complete file -> USB proof by itself. `bitperfect-compare.py` is a separate, optional step answering the further question of whether two hosts agree.

What the report records. Each `PREFIX.txt` names and hashes every stage (`input file`, `ref bytes`, `wire raw`, `tap wav`, `verdict`), so a run is auditable from the ~600-byte report alone. Only three of those are reproducible: `input file`, `ref bytes` and `tap wav`. **`wire raw` is not** — it is the untrimmed capture, so its length varies between otherwise identical runs (a few packets more or fewer recorded before the tap stops), and Linux and FreeBSD captures of the same input legitimately differ (10584816 vs 10772776 bytes at 44100/32-bit). It is provenance only; the field the comparator uses is `tap wav`. Note also that `PREFIX.wav` equals the input WAV only for a **32-bit** input: for 16/24-bit it carries the promoted 32-bit container, so it is longer and differently valued — the invariant that always holds is the `tap wav` payload hash, not the file hash.

What surrounds the audio. The capture is always longer than the reference, and everything outside is measurably all-zero: a head of stream-priming zeros — **exactly 16 ms at both 44100 and 192000 Hz**, a fixed-duration buffer prime rather than a timing accident — and a tail of the 500 ms pad plus ~19 ms the kernel keeps transmitting after the writer closes. On a **BIT-PERFECT** verdict both capture boundaries necessarily fell outside the audio; when they fall inside, the tool names it (`HEAD LOST` at the start, `INCOMPLETE` at the end) rather than hiding it. One qualification: "inaudible" describes the sample *values*. Opening or closing an isochronous stream, and any rate change around it, can still produce an audible artifact from the DAC's analogue side (mute relay, PLL relock — see `OKTO-DAC8-FreeBSD-44k1-flicker.md`); that comes from stream start/stop, not from the zeros.

Start with the canonical 44100 Hz asset on FreeBSD. The FreeBSD tap decodes `usbdump -vv` *text*, so parsing cost scales with the capture: 30 s at 44100 Hz is ~10.5 MB of payload arriving as tens of MB of hex-dump text (fine), while 30 s at 192 kHz is ~46 MB of payload as several hundred MB of text — slow, and it stresses the pcap capture too. Prove the path at 44100 first, then shorten the high-rate run (`--frames 1920000` = 10 s at 192 kHz). The Linux side reads usbmon's binary interface and has no such limit.

12 Appendix D — Source document index

This manual is generated from the repository's source files — the Markdown docs below plus the CMake build for the Installation chapter. For the full detail behind each section:

Topic	Source document
Chain overview, install, drc.sh, hotplug	`README.md`
Install / build (CMake superproject, host values)	`CMakeLists.txt`, `host.cmake.sample`
Build modules: DRC engine + site data	`cmake/core-drc.cmake`
Build modules: DAC-hotplug + brutefir services	`cmake/hotplug.cmake`
Build modules: MPD + upmpdcli renderers	`cmake/renderers.cmake`
Build modules: runtime dependency audit	`cmake/dependencies.cmake`
Build modules: per-user setup (`make user-install`)	`cmake/user-install.sh.in`
Web-UI subproject builds	`omdrc-ctrl/CMakeLists.txt`, `video/webremote/CMakeLists.txt`
Filter/config layout, drc.sh modes, agent rules	`FILTERS_AND_DRC.md`
Filter provenance, hashes, verification, the design scripts	`doc/FILTER_PROVENANCE_AND_RESPONSE.md`
Site-data split (`OMDRC_SITE_DATA_DIRS` / `OMDRC_SITE_ROOT`)	`scripts/README.md`, `host.cmake.sample`, `cmake/core-drc.cmake`
Helper scripts	`scripts/README.md`, `README.md`
Web control panel	`omdrc-ctrl/README.md`
CD / S-PDIF input bridge (FreeBSD)	`cdin/README.md`
CD / S-PDIF input bridge (Linux)	`doc/CDIN-LINUX.md`
ESI U24 XL configuration and traps	`cdin/ESI-U24XL.md`
Web control panel: `/configuration` page	`omdrc-ctrl/README.md`, `omdrc-ctrl/src/configuration.py`
USB appliance image plan	`doc/USB-APPLIANCE-IMAGE-PLAN.md`
Stable sound-device names and lifecycle	`etc/rc.d/omdrc_audio`, `etc/devd/omdrc-audio.conf`
Spectrum analyzer	`omdrc-ctrl/SPECTRUM_ANALYZER.md`
Video playback + Blu-ray	`video/README.md`
A/V sync delay derivation	`video/AV-SYNC-DELAY.md`
Web remote (install/API)	`video/webremote/README.md`
Web remote (design)	`video/webremote/ARCHITECTURE.md`
Glitch detection	`doc/GLITCH-DETECTION.md`
Bit-perfect verification, the `/bitperfect` page and its implementation	`doc/BIT-PERFECT-VERIFICATION.md`, `scripts/README.md`, `omdrc-ctrl/README.md`

Topic	Source document
Test signal	`tests/README.md`
FreeBSD port plan	`doc/FREEBSD-PORT-PLAN.md`
uaudio patches (index + install)	`freebsd-uaudio-patch/README.md`
44.1 kHz flicker analysis	`freebsd-uaudio-patch/FreeBSD-uaudio-shared-clock-bug.md`
Shared-clock fix design	`freebsd-uaudio-patch/uaudio-shared-clock-fix.md`
Clock-before-alt analysis	`freebsd-uaudio-patch/uaudio-clock-before-alt.md`
Feedback-follow audit	`freebsd-uaudio-patch/uaudio-feedback-follow.md`
Residual 44.1 kHz silent-open audit (patch 3)	`freebsd-uaudio-patch/uaudio-clock-transaction.md`
Upstream follow-up submission for #295933	`freebsd-uaudio-patch/SUBMISSION-295933.md`
virtual_oss patches (index)	`freebsd-virtual-oss-patch/README.md`
SETTRIGGER livelock root cause	`freebsd-virtual-oss-patch/ROOTCAUSE-settrigger-sync-engine-deadlock.md`
cuse refleak root cause	`freebsd-virtual-oss-patch/ROOTCAUSE-cuse_client_open-refleak.md`
cuse teardown bug report	`VIRTUAL_OSS_CUSE_DEADLOCK.md`
Cold-open silence audit	`OKTO-DAC8-silent-first-open.md`
44.1 kHz flicker observations	`OKTO-DAC8-FreeBSD-44k1-flicker.md`
MPD/curl CPU spin	`MPD-CURL-CPU-SPIN-FreeBSD.md`
Kodi OSS sink patch	`kodi-virtual-oss-patch/README.md`
VBA vs all-pass comparison	`doc/xtras/FVBA.vs.ALLPASS.md`

12.1 Keeping this manual up to date

This manual is a **synthesis** of the source documents above, not a transclusion: rebuilding the PDF does *not* pull in changes to the source `.md` files automatically. When a source document changes:

1. Update the corresponding section of the master document, `doc/pdf/open-media-drc-manual.md` (the table above is the section-to-source mapping).
2. If the chain topology changed, adjust the graphviz sources in `doc/pdf/diagrams/*.dot`.
3. Re-run `doc/pdf/build-pdf.sh` (requires pandoc, pdflatex, graphviz) to regenerate `doc/open-media-drc-manual.pdf`.

Editing constraints for the master document (pdflatex): keep it ASCII — no box-drawing characters or Unicode arrows/symbols; diagrams are added as `![caption](build/<name>.svg){width=NN%}`. Full details in `doc/pdf/README.md`.